

Provelt

A detailed codebase walkthrough

Architecture, proof boundaries, build topology,
and a subject-by-subject guide

Lean workspace

Facades, semantic cores, executable decision procedures, and kernel-reduced certificates.

Rocq federation

Independent ports, complementary developments, vendored proof libraries, and assumption audits.

Certificate pipelines

Generated candidates, literal data modules, small checkers, and explicit trust boundaries.

Snapshot

675d0a2ff0fc624fba199e006049016ba806f40a

13 August 2026

All repository hyperlinks in this document are pinned to this commit.

About this walkthrough

This document explains *how the ProveIt repository is put together*: what each major subtree proves, where its public surface lives, how its files depend on one another, where computation enters a proof, how the two proof assistants relate, and how to build, audit, debug, or extend a development without losing track of its logical status.

It is pinned to commit `675d0a2ff0fc624fba199e006049016ba806f40a`. That is important for a repository whose active branches can change both mathematical status and physical layout quickly. Paths, module inventories, and statements below refer to that exact tree; every path rendered in blue is a clickable commit-pinned source link.

The inspection used GitHub’s repository and Git-object APIs to read the exact commit, its trees, manifests, selected implementation modules, audit modules, support programs, and project READMEs. A complete checkout and full multi-hour/multi-toolchain build of ProveIt was *not* performed in the environment used to prepare this text. Consequently, this is an architectural and source walkthrough, not an independent re-certification of every theorem. The PDF itself was compiled and visually checked from the accompanying $\text{\LaTeX}$ source.

Orientation

The repository is easiest to understand along three independent axes.

1. **Mathematical subject:** algebra, analysis, combinatorics, computability, logic, number theory, and set theory.
2. **Proof shape:** a short direct theorem, a from-scratch semantic development, a finite certificate, a generated algebraic certificate stack, or a port/coverage project.
3. **Evidence boundary:** kernel-checked proof terms, kernel computation, VM/native computation, generated candidate data, external source transcription, or an explicitly retained hypothesis.

Folder names mostly encode the first axis. Facades, audit modules, status tables, and file naming conventions expose the other two.

How to use the document

Readers new to the repository should read Chapters 1 to 3 before choosing a subject chapter. A contributor modifying one project can jump directly to its subject section and then use Chapters 13 to 15. Readers evaluating theorem status should begin with Chapter 3 and consult the status/frontier catalogue in Chapter 16 rather than inferring status from a theorem name or file size.

The terms *Coq* and *Rocq* both appear because the repository is in a transition period reflected in filenames, logical roots, packages, and command names. This document uses *Rocq/Coq* for the prover family, but preserves literal names such as `_CoqProject`, `coqc`, and `Coq/` when describing repository artifacts.

Contents

List of figures	13
List of tables	14
I. The repository as a system	15
1. Orientation: what is Provelt?	16
1.1. A formal-mathematics monorepo, not a single library	16
1.2. Snapshot and scale	16
1.3. Top-level map	17
1.4. Five recurring species of project	18
1.4.1. The microproject	18
1.4.2. The hand-written semantic stack	18
1.4.3. The finite certificate project	18
1.4.4. The generated algebraic certificate pipeline	18
1.4.5. The port and coverage project	19
1.5. The fastest first inspection	19
2. Import surfaces and build topology	20
2.1. The root Lean facade	20
2.2. The root Lake workspace	20
2.2.1. Default targets versus registered libraries	21
2.2.2. Registered Lean libraries	21
2.3. Pinned Lean dependency graph	22
2.4. Rocq as a federation of logical roots	22
2.5. Submodules and vendored trees	23
2.6. Why no single build command tells the whole story	23
3. Trust, computation, and theorem status	25
3.1. The repository's central engineering question	25
3.2. A layered evidence model	25
3.2.1. Kernel-checked proof terms	25
3.2.2. Kernel reduction with <code>decide</code>	26
3.2.3. VM and native computation	26
3.2.4. External generators	26
3.3. Audit modules as executable documentation	27
3.4. Conditional endpoints and named open claims	27
3.4.1. Hypothesis in the theorem type	27
3.4.2. Named proposition for an unformalized search	27
3.4.3. Deliberate nonclaims	27
3.5. Independent proofs, ports, and parity	28
3.6. Source transcription as a separate boundary	28
3.7. A review checklist	28
4. Recurring module architectures	30
4.1. Facade, implementation, endpoint, audit	30
4.2. The explicit semantic ladder	30
4.3. Data/checker/meaning for finite certificates	31
4.4. Generated sharding	31
4.5. Compatibility facades	32
4.6. Parallel but noncomposable semantics	32
4.7. Status documentation as code architecture	32
4.8. Resource controls are part of module design	33
4.9. Naming as a navigation API	33

II. Subject-by-subject walkthrough	35
5. Algebra	36
5.1. Domain overview	36
5.2. JacobianConjecture	36
5.2.1. Public claim and mathematical object	36
5.2.2. Core definitions	37
5.2.3. Determinant path	37
5.2.4. Collision path	37
5.2.5. Supporting modules	38
5.2.6. Recommended reading order	38
5.3. PolynomialFormulas	39
5.3.1. Why this project needs a map before a file list	39
5.3.2. Low-degree formulas: relations before branches	39
5.3.3. Gaussian-rational radical syntax	40
5.3.4. Abel–Ruffini surfaces	40
5.3.5. Deciding radical solvability of a concrete quintic	40
5.3.6. The sextic decision	41
5.3.7. The Lazard/Dummit semantic core	41
5.3.8. A concrete generated certificate chain	41
5.3.9. Generator programs and reproducibility	42
5.3.10. Checkpoint status at the pinned commit	42
5.3.11. How to read the large project without drowning	43
5.3.12. Maintenance hazards	43
6. Analysis	44
6.1. Overview	44
6.2. ExponentialIdentities	44
6.2.1. Step 1: an exact rational enclosure for $\log 10$	44
6.2.2. Step 2: propagate the tiny increment	45
6.2.3. Step 3: convert a logarithmic increment to a floor	45
6.3. TrigonometricIdentities	45
6.4. How the analysis projects fit the repository	45
7. Combinatorics	47
7.1. Overview	47
7.2. Expression enumeration	47
7.2.1. PowerTowers	47
7.2.2. A198683: principal values of towers of i	47
7.2.3. Generated data and the $n=12$ frontier	48
7.2.4. A158415 and derivative expansions	48
7.3. SquaredSquare	48
7.3.1. What is proved	48
7.3.2. Geometry reduced to integer data	49
7.3.3. The elementary lower bound	49
7.3.4. Sharp minimality remains a named claim	49
7.3.5. Module path	49
7.4. KlarnerConstant	50
7.4.1. Headline result	50
7.4.2. The five layers	50
7.4.3. The geometry module families	51
7.4.4. Asymptotic endpoint	51
7.4.5. Trust and transcription	51
7.5. Reading routes	52
8. Computability	53
8.1. Overview	53
8.2. BusyBeaver	53
8.2.1. The facade is intentionally small	53

8.2.2.	Machine model	53
8.2.3.	Score is not runtime	54
8.2.4.	Classification and certificate modules	54
8.2.5.	Mathlib and Rocq bridges	54
8.3.	CombinatoryLogic	54
8.3.1.	Semantic families	54
8.3.2.	Lambda to SK: bracket abstraction	55
8.3.3.	SK, SKI, and Iota	55
8.3.4.	Confluence and evaluation	55
8.3.5.	Partial-recursive compilation	56
8.3.6.	What is and is not claimed	56
8.4.	TuringDegrees	56
8.4.1.	Why Lean and Rocq differ	56
8.4.2.	Lean architecture	56
8.4.3.	Rocq architecture	57
8.4.4.	The coverage ledger	57
8.4.5.	Build boundary	57
8.5.	Comparative map	57
9.	Logic	59
9.1.	Overview	59
9.2.	Propositional and equational microprojects	59
9.2.1.	Sheffer stroke and Boolean algebra	59
9.2.2.	Natural deduction and entailment	60
9.2.3.	Finite matrix noncharacterizability	60
9.3.	QuantifierCommutation	60
9.4.	FirstOrder	60
9.4.1.	A deliberately fixed language	60
9.4.2.	Module ladder	61
9.4.3.	The Henkin path in more detail	61
9.4.4.	Two compactness projects, two provenances	62
9.5.	FoundationPort and Modal	62
9.5.1.	Pinned source, independent target	62
9.5.2.	Declaration-free facades still count	62
9.5.3.	The Modal port	62
9.6.	Interpretability	63
9.6.1.	PA/HF	63
9.6.2.	ZFCinPA: target and status	63
9.6.3.	Five architectural layers	63
9.6.4.	Specification hardening	64
9.6.5.	Lake target discovery gotcha	64
9.7.	Peano arithmetic	64
9.7.1.	ListCoding	64
9.7.2.	Bounded PA consistency	65
9.7.3.	NoFiniteModel	65
9.7.4.	Finite-basis and undecidability projects	65
9.8.	PresburgerArithmetic	66
9.8.1.	Lean end-to-end decision procedure	66
9.8.2.	Rocq scope	66
9.9.	How to navigate Logic safely	66
10.	Number theory	67
10.1.	Overview	67
10.2.	DiophantineEquations and Fermat exponent four	67
10.3.	IntegerSums	67
10.4.	RationalEnumeration	67
10.5.	RiemannHypothesis	68

11. Set theory	69
11.1. Overview	69
11.2. ZF	69
11.2.1. Public surface and internal scale	69
11.2.2. Why one substantial module can be reasonable	69
11.3. ClosureAxiomatization	69
11.3.1. Alternative axiom	69
11.3.2. Build and exposition	70
11.4. Bounded ZFC consistency	70
11.4.1. The exact theorem	70
11.4.2. Proof route	70
11.4.3. All-occurrences restriction	71
11.4.4. Discarded routes documented	71
11.4.5. Relation to ZFCinPA	71
11.5. Set-theory dependency flow	71
11.6. Recommended reading order	71
12. Ancillary directories, vendored code, and tooling	73
12.1. Why these directories matter	73
12.2. lib: the only vendored-code home	73
12.2.1. Coq Library of Undecidability	73
12.2.2. FormalizedFormalLogic/Foundation	73
12.2.3. MathComp-Abel	73
12.2.4. Synthetic Computability	73
12.2.5. Coq-BB5	74
12.3. Tools/Rocq92Compat	74
12.4. Oeis	74
12.5. Shenanigans	74
12.6. Articles and reports	75
12.7. Repository hygiene files	75
12.8. Ownership map	75
III. Building, debugging, and extending Provelt	76
13. Working with the repository	77
13.1. Start from a pinned tree	77
13.2. Lean environment	77
13.2.1. Toolchain and cache	77
13.2.2. Broad validation	77
13.2.3. Direct module elaboration	77
13.2.4. Threading and memory	78
13.3. Focused Lean validation recipe	78
13.4. Rocq environment	78
13.4.1. Version and package roots	78
13.4.2. Root build	79
13.4.3. Focused sequential build	79
13.5. Audit workflow	79
13.5.1. Lean	79
13.5.2. Rocq	80
13.6. Generator workflow	80
13.7. Reading build output correctly	80
13.8. A clean validation record	81
14. Debugging proof and build failures	82
14.1. Classify the failure before changing the proof	82
14.2. Target-discovery failures	82
14.3. Import-resolution failures	83
14.3.1. Lean logical name versus physical path	83

14.3.2. Rocq logical roots	83
14.3.3. Submodule state	83
14.4. Elaboration and specification failures	83
14.4.1. Unexpected implicit parameters	83
14.4.2. Coercion and notation drift	84
14.4.3. Quotients and representative changes	84
14.5. Tactic and normalization failures	84
14.5.1. Do not respond first by raising all limits	84
14.5.2. Simplifier loops and giant goals	84
14.6. Kernel-certificate failures	85
14.6.1. A bisection strategy for term/tail shards	85
14.6.2. Recursion depth versus heartbeats	85
14.6.3. Native replay disagrees with kernel replay	85
14.7. Semantic-bridge failures	85
14.8. Cross-prover mismatches	86
14.9. Rocq environment and compatibility failures	86
14.10. Minimal reproductions inside the monorepo	86
14.11. When to suspect the statement	86
15. Extending a project without blurring its boundary	88
15.1. Choose the project shape first	88
15.2. Adding a direct Lean theorem	88
15.2.1. Place the theorem at the lowest reusable layer	88
15.2.2. Generalize only where it lowers dependencies	88
15.2.3. Add an audit entry	89
15.3. Adding a semantic layer	89
15.3.1. Define the contract	89
15.3.2. Prove substitution/renaming early	89
15.3.3. Keep data-carrying and Prop-valued relations distinct	89
15.4. Adding a finite certificate	89
15.4.1. Start with the mathematical predicate	89
15.4.2. Choose an evaluator deliberately	90
15.4.3. Separate placement from meaning	90
15.5. Adding or changing a generator	90
15.5.1. Treat output format as an API	90
15.5.2. Emit a manifest	91
15.5.3. Regenerate all or prove locality	91
15.5.4. Keep generator tests outside theorem claims	91
15.6. Adding a Rocq port	91
15.6.1. Choose the parity level	91
15.6.2. Map dependencies, not just declarations	91
15.6.3. Audit and kernel-check the target	92
15.7. Registering a new project	92
15.8. Writing status documentation	92
15.9. Commit and review hygiene	93
15.10. Pre-merge checklist	93
16. Status and active frontiers at the pinned commit	94
16.1. Three different meanings of “unfinished”	94
16.2. Snapshot catalogue	94
16.3. How to read a checkpoint commit	97
16.4. Status can regress without a theorem becoming false	97
16.5. Recommended status vocabulary	97
17. Architectural assessment and technical recommendations	99
17.1. What the repository already does exceptionally well	99
17.1.1. It names proof boundaries	99
17.1.2. It avoids false cross-prover symmetry	99
17.1.3. It scales exact computation by sharding	99

17.1.4.	It separates reusable conditional theorems from unconditional endpoints	99
17.1.5.	It treats status ledgers as artifacts	99
17.2.	Structural risks	99
17.2.1.	Generated-source discoverability	99
17.2.2.	Silent build no-ops	100
17.2.3.	Documentation can drift faster than theorem types	100
17.2.4.	Resource feasibility is implicit	100
17.2.5.	Root validation is not visibly automated	100
17.2.6.	Submodule and compatibility complexity	100
17.3.	Recommendation 1: a machine-readable project registry	100
17.4.	Recommendation 2: target preflight	101
17.5.	Recommendation 3: tiered validation	101
17.6.	Recommendation 4: generated artifact manifests	102
17.7.	Recommendation 5: generated status from literal theorem types	102
17.8.	Recommendation 6: cross-prover coverage schema	102
17.9.	Recommendation 7: source-transcription checksums	103
17.10.	Recommendation 8: resource contracts	103
17.11.	Recommendation 9: stable public-facade tests	103
17.12.	Recommendation 10: preserve the current honesty	103
17.13.	Overall assessment	104
18.	Reading itineraries	105
18.1.	A one-hour architectural tour	105
18.2.	To audit one headline theorem	105
18.3.	To understand generated algebra	105
18.4.	To understand formal metamathematics	106
18.5.	To learn certificate engineering	106
18.6.	To compare Lean and Rocq	106
18.7.	To contribute an independent result	106
18.8.	To review mathematical status rather than code style	107
IV.	Reference appendices	108
A.	Root Lean workspace registry	109
B.	Project and artifact index	113
C.	Command cookbook	115
C.1.	Repository state	115
C.2.	Lean bootstrap and broad builds	115
C.3.	Focused Lean	115
C.4.	Lean inspection	115
C.5.	Lean resource controls	115
C.6.	Rocq focused build	116
C.7.	Rocq root build	116
C.8.	Rocq inspection	116
C.9.	Foundation port coverage	116
C.10.	Generated output comparison	116
C.11.	Find reachability	117
C.12.	Clean rebuilds	117
D.	Trust and status checklist	118
D.1.	For a direct theorem	118
D.2.	For a computed theorem	118
D.3.	For a generated certificate	118
D.4.	For a conditional theorem	118
D.5.	For a Lean/Rocq pair	118
D.6.	For a source transcription	119

E. Glossary	120
F. Snapshot methodology and source notes	122
F.1. Pinned repository state	122
F.2. Inspection method	122
F.3. What was and was not independently checked	122
F.4. Current-event context	123
F.5. Path and typography conventions	123
F.6. How to update this walkthrough	123
F.7. Closing perspective	123

List of Figures

1.1.	The physical top level is a subject taxonomy. Proof shape and trust boundary cut across it. . . .	16
2.1.	Build topology. Lean and Rocq developments share a repository and often a theorem surface, but not proof artifacts or one common package graph.	24
3.1.	Evidence layers. External generation can sit below a verified checker; an unresolved mathematical premise must instead remain in the theorem type or status document.	25
4.1.	The ordinary hand-written project shape. A facade may import every stage directly, but the conceptual dependency direction remains useful.	30
5.1.	Generated certificate path in PolynomialFormulas.	42
7.1.	KlarnerConstant proof flow. The final numeric certificate is small; proving that the actual geometric counts satisfy its recurrences is the bulk of the development.	50
9.1.	The first-order dependency ladder.	61
9.2.	Two independent uniformity gaps in bounded consistency.	65
11.1.	The repository-local set-theory stack and its main consumers.	71

List of Tables

1.1.	Three different notions of “what belongs to ProveIt.”	16
1.2.	GitHub language accounting at the pinned commit.	17
1.3.	Top-level directories at the pinned commit.	17
2.1.	Root Lean libraries and their physical roles.	21
3.1.	Common cross-prover relationships.	28
3.2.	Trust-oriented questions for a headline theorem.	28
4.1.	Frequent naming signals.	33
5.1.	JacobianConjecture Lean module responsibilities.	38
5.2.	Conceptual strata of the PolynomialFormulas facade.	39
7.1.	SquaredSquare modules.	49
7.2.	Klarner geometric families.	51
8.1.	The three computability projects compared.	57
9.1.	Logic project families.	59
9.2.	ZFCinPA status at the pinned commit.	63
12.1.	Where an artifact should live.	75
14.1.	Failure classes and first diagnostics.	82
16.1.	Headline status at commit 675d0a2f.	95
16.2.	Precise status terms.	97
17.1.	Suggested validation tiers.	101
A.1.	Root Lean library registry	110
B.1.	Where to start for each major project.	113

Part I.

The repository as a system

1. Orientation: what is ProveIt?

1.1. A formal-mathematics monorepo, not a single library

ProveIt is a monorepo of formalized mathematics. It has a shared Lean workspace and a repository-wide Rocq build description, but it does not have one domain model, one executable entry point, or one uniform proof strategy. A two-line propositional theorem, a from-scratch completeness proof, a verified decision procedure, a finite geometric certificate, and a generated multi-thousand-module invariant-theory calculation can all be first-class projects in the same tree.

The top-level organization is mathematical:

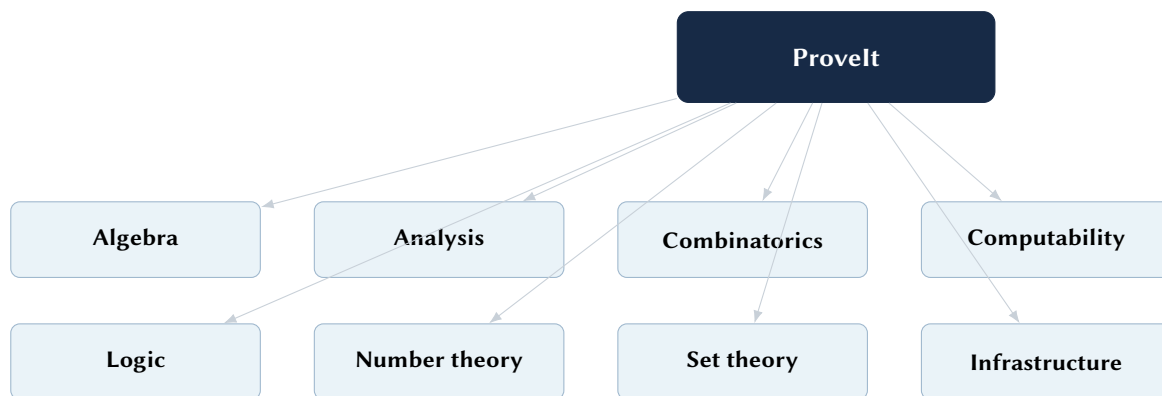


Figure 1.1.: The physical top level is a subject taxonomy. Proof shape and trust boundary cut across it.

The root Lean facade, [ProveIt.lean](#), creates a convenient broad import surface over selected projects. The root Lake file, [lakefile.toml](#), registers many more libraries and gives some heavyweight and audit libraries explicit default-target status. On the Rocq side, [_CoqProject](#) is both a logical-path map and an ordered source registry. These three files answer different questions:

Table 1.1.: Three different notions of “what belongs to ProveIt.”

Artifact	Meaning
ProveIt.lean	The intentionally broad <i>user-facing Lean import</i> . It omits several expensive or status-sensitive modules even when their libraries are registered and buildable.
lakefile.toml	The Lean workspace registry: logical library names, physical source roots, facade defaults, and additional default targets.
_CoqProject	Rocq logical roots, vendored roots, and the source set whose dependency graph is handed to rocq makefile.

A frequent mistake is to treat these surfaces as synonyms. For example, `import ProveIt` does not import the expensive Busy Beaver classification certificates, while the corresponding project is fully present and has separate build targets. Conversely, a project-local Lean directory may have a standalone `lakefile.toml` even though the root workspace also maps the same logical library.

1.2. Snapshot and scale

The inspected snapshot is commit `675d0a2ff0fc624fba199e006049016ba806f40a`, dated 13 August 2026. Its GitHub language accounting is shown in Table 1.2. Byte counts are useful for estimating build and storage shape,

Table 1.2.: GitHub language accounting at the pinned commit.

Language	Bytes	Share
Lean	139,510,311	77.60%
Rocq Prover	39,617,616	22.04%
Python	380,455	0.21%
T _E X	198,247	0.11%
Wolfram Language	51,635	0.03%
PowerShell	19,257	0.01%
C	2,195	< 0.01%
Shell	2,193	< 0.01%
Total	179,781,909	100%

but not for measuring mathematical originality: generated literal certificates and vendored sources dominate raw volume.

GitHub code search reports roughly 30,464 Lean files and 2,968 Rocq .v files in this snapshot. Those numbers are startling until one separates the following populations:

- handwritten semantic and theorem modules;
- generated data and certificate shards, especially under **PolynomialFormulas**;
- copied or vendored proof libraries under **lib**;
- audit facades and compatibility facades;
- independent Lean and Rocq versions of the same mathematical project.

The repository therefore has a *small conceptual core relative to its physical size*. A productive walkthrough identifies the semantic core and the certificate checker before opening the largest files.

1.3. Top-level map

Table 1.3.: Top-level directories at the pinned commit.

Directory	Main contents	Typical proof shape
Algebra	Jacobian counterexamples; low-degree polynomial formulas; Abel–Ruffini, quintic and sextic radical decision procedures; Lazard/Dummit certificate stacks.	Direct algebra, imported theory, and very large generated certificates.
Analysis	Exact trigonometric and exponential identities, including the tiny exponent tower floor calculation.	Short analytic developments with exact rational bounds.
Combinatorics	Expression enumeration, polyomino growth, and squared-square dissections.	Finite certificates, explicit injections, and asymptotic arguments.
Computability	Busy Beaver, combinatory logic, and Turing degrees.	Executable semantics, simulation theorems, classification certificates, and cross-prover coverage.
Logic	Propositional/equational microprojects, first-order completeness, Foundation ports, modal logic, interpretability, Peano arithmetic, and Presburger arithmetic.	From-scratch calculi, arithmetization, ports, decision procedures, and status-heavy metamathematics.

Directory	Main contents	Typical proof shape
NumberTheory	Fermat exponent four, integer sums, Calkin–Wilf enumeration, and a PA statement of the Riemann hypothesis.	Small wrappers plus independent paired proofs.
SetTheory	A lightweight ZF development, closure axiomatization, and bounded-complexity consistency for ZFC.	Internal model mathematics and partial-satisfaction constructions.
Oeis	An auxiliary/legacy home for sequence material such as A290268.	Sequence-specific formalization.
Shenanigans	A redirect to a separate repository rather than an active development.	Documentation only.
Tools	The Rocq 9.2 compatibility overlay.	Toolchain engineering, not theorem content.
lib	Vendored and submodule dependencies, with provenance and licensing information.	Read-only/reference or locally hardened third-party proof code.

The domain READMEs—for example [Algebra/README.md](#), [Computability/README.md](#), and [Logic/README.md](#)—are useful tables of contents, but project READMEs often contain the more important information: exact theorem status, intentional nonclaims, build commands, trust boundaries, and module-by-module roles.

1.4. Five recurring species of project

1.4.1. The microproject

A microproject has a facade, one or a few proof modules, and an audit. Examples include **QuantifierCommutation**, **ShefferStroke**, and several analysis and number-theory topics. These are the best places to learn local naming and audit conventions because their dependency graph fits in one view.

1.4.2. The hand-written semantic stack

Here the repository defines syntax, semantics, a proof system, metatheory, a public textbook theorem, and an assumption audit. The independent first-order completeness project is the canonical example: $\text{Fol} \rightarrow \text{Calculus} \rightarrow \text{Completeness} \rightarrow \text{ClassicalCompleteness/Compactness} \rightarrow \text{Audit}$.

1.4.3. The finite certificate project

A finite object is supplied as data and checked by a small executable predicate whose correctness is proved. Dijkstra’s squared-square placement and the Busy Beaver small-state classifications use this shape. The crucial question is not “was a computer used?” but “which computation is replayed inside the proof assistant, and by which evaluator?”

1.4.4. The generated algebraic certificate pipeline

The largest Lazard and Dummit developments go further. External programs construct and shard enormous exact polynomial identities; generated Lean or Rocq modules contain literal normal forms; small certificate modules prove that a formally defined substitution, multiplication, or normalization operation accepts them. The generator is replaceable. The formal checker and the semantic theorem connecting acceptance to the intended mathematics are the proof boundary.

1.4.5. The port and coverage project

Some projects independently recreate a theorem family in both proof assistants. Others, notably **FoundationPort** and **Modal**, track a pinned source library and record which declarations have been ported, partially ported, or remain outside scope. Here a status ledger is part of the architecture: a green module build is not automatically a parity claim.

1.5. The fastest first inspection

Reading route

For an unfamiliar project X, use this sequence.

1. Read the nearest project `README.md`, especially headings named *Status*, *Trust boundary*, *Deliberate boundary*, *Layout*, and *Building*.
2. Open the facade `Lean/X.lean` or its equivalent. Its import list identifies the public dependency cut.
3. Open `Audit.lean` or `Audit.v`. The declarations audited there are usually a better public-API inventory than filename size.
4. Follow one headline theorem backward until definitions stop being project-specific.
5. If names contain `Data`, `Certificate`, `Shard`, `Tail`, or `Support`, identify the checker before reading the literal data.
6. Compare the root build target with any project-local `Lake` or `Rocq` instructions. They can differ intentionally.

Primary files

`README.md`; `ProveIt.lean`; `lakefile.toml`; `_CoqProject`; `lean-toolchain`; `lake-manifest.json`; `.gitmodules`; `.gitattributes`.

2. Import surfaces and build topology

2.1. The root Lean facade

The root module is intentionally declarative: it contains imports, not a repository-wide namespace or executable. At the pinned commit its public surface is equivalent to the following list.

Listing 2.1: Root facade import surface, normalized for presentation.

```
import JacobianConjecture
import PolynomialFormulas
import TrigonometricIdentities
import ExponentialIdentities
import DiophantineEquations
import IntegerSums
import RationalEnumeration
import ShefferStroke
import NaturalDeduction
import FiniteMatrixNoncharacterizability
import MonotonicityOfEntailment
import PrincipleOfExplosion
import EquationalLogic
import BooleanAlgebra
import PowerTowers
import A158415
import A290268
import SquaredSquare
import KlarnerConstant
import FirstOrder
import ArbitraryLanguageCompactness
import QuantifierCommutation
import PAHF
import NoFiniteModel
import PAFiniteBasisReduction
import PresburgerArithmetic
import RiemannHypothesis
import ZF
import ClosureAxiomatization
import CombinatoryLogic
import BusyBeaver
import TuringDegrees
```

The comment in [ProveIt.lean](#) is a design contract: the facade is broad, but the expensive BB2/BB3 certificate modules are not imported by the **BusyBeaver** facade. Similar choices elsewhere prevent a convenient top-level import from becoming a request to elaborate every generated certificate in the repository.

Orientation

A facade answers “which modules should a consumer see by default?” It does not answer “which modules exist?”, “which modules are default build targets?”, or “which theorem is unconditional?” Those questions belong to the Lake registry, audits, and project status documents respectively.

2.2. The root Lake workspace

The root [lakefile.toml](#) maps logical library names to physical source directories. This lets a project keep a mathematically natural location such as

Logic/PeanoArithmetic/BoundedConsistency/Lean/

while exposing a short import root such as `BoundedPAConsistency`. The workspace also maps the vendored Foundation tree as a Lean library, so repository-authored developments can import the pinned source without copying it.

2.2.1. Default targets versus registered libraries

The default targets include the root facade and several libraries that are deliberately not imported by it:

Listing 2.2: Conceptual form of the root default-target set.

```
defaultTargets := [
  `ProveIt,
  `PAListCoding,
  `PAListCoding.Audit,
  `PAUndecidable,
  `PAUndecidable.Audit,
  `BoundedPAConsistency,
  `BoundedPAConsistency.Audit
]
```

This is a subtle but valuable separation.

- **Facade inclusion** controls downstream import cost and namespace exposure.
- **Default-target inclusion** controls what a broad workspace build is expected to validate.
- **Library registration** controls what can be requested explicitly from the root workspace.

Thus **PAUndecidable** can be part of the ordinary validation policy without becoming part of every import ProveIt environment.

2.2.2. Registered Lean libraries

Table 2.1.: Root Lean libraries and their physical roles.

Logical library	Physical project or role
JacobianConjecture	Algebraic counterexample project.
PolynomialFormulas	Formula, solvability, and Lazard/Dummit project.
ProveIt	Root facade.
TrigonometricIdentities	Exact trigonometric identities.
ExponentialIdentities	Exact exponential identities and bounds.
PowerTowers	Parenthesized power-tower enumeration.
A158415	Radical-expression enumeration.
A290268	Derivative-expansion sequence development.
SquaredSquare	Perfect squared-square existence and lower bound.
KlarnnerConstant	Polyomino growth upper bound.
BusyBeaver	Machine semantics and opt-in classifications.
CombinatoryLogic	Lambda/SK/SKI/Iota simulations and universality.
TuringDegrees	Oracle reducibility and degree structure.
FirstOrder	Dependency-free fixed-language first-order metatheory.
ArbitraryLanguageCompactness	Mathlib-backed arbitrary-language compactness facade.
QuantifierCommutation	Constructive commutation and counterexamples.
ShefferStroke	Propositional Sheffer-stroke development.
NaturalDeduction	Propositional natural deduction.
FiniteMatrixNoncharacterizability	Propositional noncharacterizability result.
MonotonicityOfEntailment	Entailment monotonicity microproject.
PrincipleOfExplosion	Explosion microproject.
EquationalLogic	Shared equational logic.
BooleanAlgebra	Boolean-algebra development.
PAHF	Peano arithmetic / hereditarily finite set interpretation.
NoFiniteModel	No finite model of PA.
BoundedPAConsistency	Bounded-complexity consistency for PA.
PAListCoding	PA-definable list and number-theory predicates.

Logical library	Physical project or role
PAFiniteBasisReduction	Finite-axiomatizability reduction.
Foundation	Pinned vendored Foundation Lean source.
PAUndecidable	Undecidability of PA.
PresburgerArithmetic	Cooper elimination and decision.
DiophantineEquations	Diophantine-equation results, including FLT exponent four.
IntegerSums	Integer-sum identities.
RationalEnumeration	Calkin–Wilf enumeration.
RiemannHypothesis	Arithmetic statement of RH.
ZF	Repository-local lightweight ZF base.
BoundedZFCConsistency	Numeralwise bounded consistency for ZFC.
ZFCinPA	Uniform PA-provability project, with a conditional endpoint.
ClosureAxiomatization	Alternative closure axiom and equivalence with ZF.

Several subprojects also carry their own `lakefile.toml`. The local project is useful when it has a smaller dependency closure, is intentionally Mathlib-free, or needs different target discovery. The root workspace is useful for integration and shared cached dependencies. When documentation offers both commands, that is generally an intentional choice, not duplicate boilerplate.

2.3. Pinned Lean dependency graph

`lean-toolchain` selects `leanprover/lean4:v4.32.0`. The root manifest pins Mathlib and its transitive Lake dependencies. At the inspected snapshot those include **Plausible**, **LeanSearchClient**, **importGraph**, **ProofWidgets**, **Aesop**, **Qq**, **Batteries**, and **Cli**. The manifest is the reproducibility boundary for the Lean package graph; a source module’s imports are only the human-readable view of that graph.

The ordinary cold-checkout bootstrap is:

Listing 2.3: Root Lean bootstrap and broad build.

```
lake exe cache get
lake build
```

The broad build is intentionally expensive. Many project READMEs recommend setting `LEAN_NUM_THREADS=0` to disable Lake’s worker pool when individual modules consume several gigabytes, while some projects that rely on a worker require the ambient value to be at least one. Treat the project-local command as authoritative.

2.4. Rocq as a federation of logical roots

The root `_CoqProject` begins with logical-path declarations, for example:

Listing 2.4: Representative Rocq logical roots.

```
-Q Algebra/JacobianConjecture/Coq JacobianConjecture
-Q Algebra/PolynomialFormulas/Coq PolynomialFormulas
-R lib/MathComp-Abel/theories Abel
-Q Computability/CombinatoryLogic/Coq CombinatoryLogic
-Q Logic/FirstOrder/Coq FirstOrder
-Q Logic/Modal/Coq FoundationModal
-Q NumberTheory/RationalEnumeration/Coq RationalEnumeration
-Q SetTheory/ZF/Coq ZF
-Q lib/Coq-Synthetic-Computability/theories SyntheticComputability
-Q Computability/TuringDegrees/Coq TuringDegrees
```

It then enumerates the source files to be handed to the generated dependency graph. The **PolynomialFormulas** portion alone includes a long sequence of core definitions, invariant-theory bridges, generated data shards,

and certificate modules. The order is not meant to be read as a hand-maintained linear execution script; `rocq` `makefile` resolves declared imports, while the registry states the intended source universe.

A root build follows the repository’s pinned vendored dependencies:

Listing 2.5: Repository-wide Rocq setup and build.

```
git submodule update --init `
  lib/Coq-Synthetic-Computability `
  lib/MathComp-Abel

opam install --yes --deps-only `
  ./lib/MathComp-Abel/coq-mathcomp-abel.opam

pwsh -NoProfile -File `
  Computability/TuringDegrees/Coq/BuildSyntheticComputability.ps1

rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq
```

Focused, sequential `coqc` commands are common in project READMEs. They serve three purposes: smaller feedback loops, explicit import order for dependency-light developments, and avoidance of platform-specific parallel build problems. Many projects finish with `coqchk` or `rocqchk` on the public modules.

2.5. Submodules and vendored trees

The root `.gitmodules` records four submodules:

1. the current repository fork/snapshot of the Coq Library of Undecidability;
2. **FormalizedFormalLogic/Foundation**, used as a pinned Lean dependency and a source reference for independent ports;
3. **MathComp-Abel**, used by the Rocq Abel–Ruffini development;
4. **coq-synthetic-computability**, used by Turing-degree wrappers.

In addition, selected Busy Beaver Rocq certificates are vendored as ordinary repository content. `lib/README.md` is the provenance index and states a strong locality rule: `lib` is the only home for vendored code; repository-authored wrappers, patches, bridges, and models live in the subject tree that owns them.

2.6. Why no single build command tells the whole story

A successful `lake build` establishes that the selected Lean targets elaborate under the pinned package graph. It does not run Rocq. A successful root Rocq `make` does not validate support scripts, Wolfram replays, article PDFs, or every local executable generator unless they are registered sources. A focused project command may intentionally validate more audit modules than a facade import, or less generated data than a repository-wide build.

The repository handles this by putting validation instructions next to the project’s status claims. For a serious review, the README command, the facade, the audit, and the root registry should be read as a four-part specification.

Primary files

`ProveIt.lean`; `lakefile.toml`; `lake-manifest.json`; `lean-toolchain`; `_CoqProject`; `.gitmodules`; `lib/README.md`.

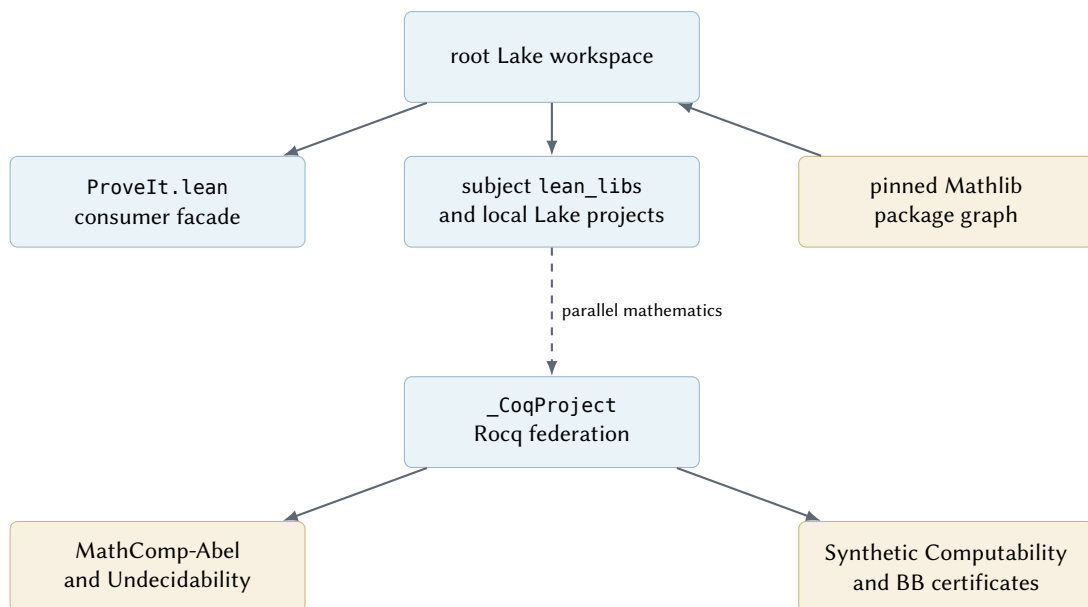


Figure 2.1.: Build topology. Lean and Rocq developments share a repository and often a theorem surface, but not proof artifacts or one common package graph.

3. Trust, computation, and theorem status

3.1. The repository’s central engineering question

Formal mathematics repositories often say that a result is “machine-checked.” ProveIt usually goes further and records *which machine-checked object closes the theorem* and *what remains outside that object*. This matters because the repository contains all of the following:

- hand-written proof terms;
- elaborator tactics whose output is rechecked by the kernel;
- Boolean decision procedures reduced by the kernel;
- virtual-machine and native evaluators;
- external Python or Wolfram programs that generate exact candidates;
- manually transcribed finite data from papers;
- imported theorems from Mathlib, MathComp-Abel, Foundation, or other pinned libraries;
- conditional endpoints with explicit mathematical hypotheses.

These mechanisms are not interchangeable. A useful code walkthrough follows the evidence path from a public theorem down to the smallest trusted checker or imported theorem, rather than stopping at the tactic used in the last line.

3.2. A layered evidence model

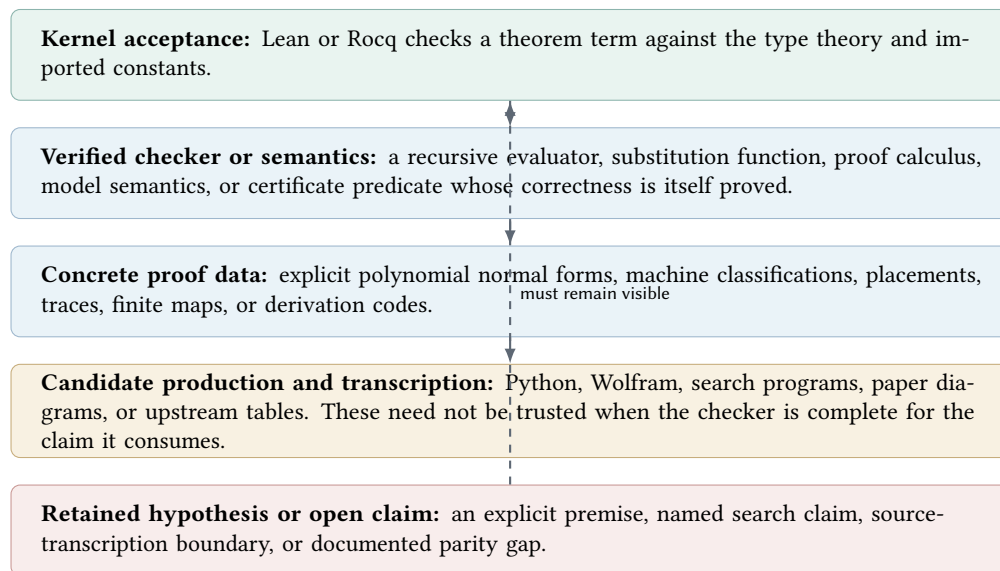


Figure 3.1.: Evidence layers. External generation can sit below a verified checker; an unresolved mathematical premise must instead remain in the theorem type or status document.

3.2.1. Kernel-checked proof terms

The ordinary case is a theorem whose elaborated term is accepted by the Lean or Rocq kernel. Tactics such as `ring`, `norm_num`, and `linarith` are proof producers: their implementation can be complicated, but a bug normally causes them to produce no term or an invalid term, not a silently accepted theorem.

Imported constants are part of this boundary. A Lean wrapper around a Mathlib theorem is fully kernel-checked, but its proof provenance is “specialization of imported theorem,” not “independent local proof.” The repository usually states that distinction explicitly, as in arbitrary-language compactness and the Lean proof of Fermat’s Last Theorem for exponent four.

3.2.2. Kernel reduction with decide

When a proposition has a decidable instance, a theorem such as

```
theorem finite_certificate_is_valid :
  checkCertificate certificate = true := by
  decide
```

can be reduced inside Lean’s kernel. Some modules use `decide +kernel` to make this intent especially explicit. This is appropriate for finite integer grids, sparse-polynomial list equality, and bounded machine searches when the reduction remains feasible.

The cost is elaboration time, recursion depth, and memory. Large certificate modules often raise `maxRecDepth` and `maxHeartbeats`, or split one calculation into term, row, tail, and merge certificates so each kernel reduction stays bounded.

3.2.3. VM and native computation

Rocq projects often use `vm_compute`. Lean can use `native_decide`. Both accelerate executable reflection by moving computation away from the slowest kernel reduction path, but they change the operational trust story. ProveIt project READMEs commonly state which evaluator is used.

This is not a binary “formal/not formal” distinction. The theorem is still accepted by the proof assistant, and the checker can still be proved correct. The distinction concerns the evaluator and compiler/runtime relied upon to produce the proof witness. For example:

- **SquaredSquare** advertises Lean `decide +kernel` and Rocq `vm_compute`;
- several projects explicitly say that no `native_decide` boundary is present;
- the root README separates kernel proofs from `native_decide`/VM computation rather than hiding the difference behind a generic “computed” label.

3.2.4. External generators

A generator can enumerate a finite search space, derive a Groebner-style normal form, split a giant expression, or emit source modules. The generator is outside the proof boundary when all of the following hold:

1. its output is represented as ordinary formal data;
2. a formal checker recomputes the relevant relation;
3. a theorem proves that checker acceptance implies the mathematical statement;
4. the generated module is actually imported into the endpoint.

A Python script that merely prints “verified” does not satisfy this pattern. A script that emits an exact sparse polynomial subsequently normalized by formally defined Lean operations does.

Proof boundary

The right review question is: *Could a malicious or buggy generator make the proof assistant accept a false theorem?* If the generator can only choose candidate data and the formal checker is sound for arbitrary data, the answer is no. If the script computes a fact that is merely asserted as an axiom or unverified constant, the script belongs to the trusted base.

3.3. Audit modules as executable documentation

Lean audit modules typically contain two kinds of command:

Listing 3.1: Typical Lean audit pattern.

```
#check headline_theorem
#print axioms headline_theorem
```

The first catches accidental API drift and documents the intended declaration. The second prints the constants that Lean treats axiomatically in the theorem’s transitive proof term. For Mathlib-backed classical mathematics, the common footprint is `propext`, `Classical.choice`, and `Quot.sound`. A smaller constructive project may print no assumptions.

Rocq audits use `Print Assumptions`; project build recipes may additionally run `coqchk` or `rocqchk` over the public modules. An audit is most valuable when it names the *headline endpoints*, not merely internal lemmas. The Jacobian audit, for example, checks determinant, collision, noninjectivity, no-polynomial-inverse, dimension-specific, stabilization, scaling, and simplified-counterexample declarations.

Audits also expose a subtle failure mode: a theorem can have no axiom dependency yet state the wrong proposition. The **ZFCinPA** README records an instance where an out-of-scope identifier became an accidental implicit variable. The project responded by placing `set_option autoImplicit false` in statement-bearing modules and auditing the literal endpoint. Assumption audits verify proof dependencies; they do not replace specification review.

3.4. Conditional endpoints and named open claims

A conditional theorem is not a defect when the condition is the intended formal boundary and is documented accurately. ProveIt uses several patterns.

3.4.1. Hypothesis in the theorem type

The reusable **KlarnerConstant** theorem in `Main.lean` accepts abstract recurrence and domination hypotheses. The concrete endpoint in `GeometricComplete.lean` supplies those hypotheses for actual polyomino counts, so the user-facing bound is unconditional. Here the conditional intermediate theorem is an abstraction boundary.

By contrast, the current **ZFCinPA** endpoint keeps `ZFCSuccessorImplicationsInAllModels` as a premise. Its README states that nonstandard-index successor implications remain open. The reduction is proved to be an equivalence, so the premise is not disguised as a harmless implementation detail.

3.4.2. Named proposition for an unformalized search

SquaredSquare defines `DuijvestijnSearchClaim` for the exact statement that every perfect squared square has at least 21 pieces. It proves that sharp minimality is equivalent to this claim and proves the unconditional bracket $7 \leq \text{minimum order} \leq 21$. No axiom is introduced for the historical exhaustive search.

This naming style is especially honest: users can import and reason about the reduction without accidentally receiving the open search result as a theorem.

3.4.3. Deliberate nonclaims

The Turing-degree project enumerates deep existence and classification theorems it has not formalized. The A198683 project withholds $n = 12$ from the accepted OEIS data field while a proof-quality certificate is missing. The Busy Beaver facade excludes expensive classifications, and the topic README distinguishes a checked score theorem from a conditional root equality. In each case, omission is an architectural status choice rather than an absence readers must infer.

3.5. Independent proofs, ports, and parity

The Lean and Rocq directories often prove similarly named theorems, but their relationship varies:

Table 3.1.: Common cross-prover relationships.

Relationship	Meaning
Independent reproof	Both sides define comparable semantics and prove a theorem separately, as in quantifier commutation or first-order completeness.
Contract-level parity	The numeric representation differs, but both sides prove the same public round-trip/functionality contract, as in PA list coding.
Complementary developments	One side provides results absent on the other because its imported library or constructive setting differs. Turing degrees is the clearest example.
Imported specialization versus local proof	One side wraps an upstream theorem while the other reconstructs an argument, as with exponent-four Fermat.
Source port	A pinned source module is used as the declaration inventory, while the target proof assistant independently recreates the mathematical surface. FoundationPort and Modal use this pattern.
Parallel endpoint with a parity gap	Most layers correspond, but one endpoint remains conditional or absent in one assistant. Bounded PA consistency documents such a gap for the strongest uniform sentence.

The phrase “independent” should therefore be read locally. A Rocq port can be independent of a Lean implementation while still importing a pinned Rocq library. A Lean theorem can be independent of a Coq proof while relying on Mathlib. The relevant provenance appears in the project README and import graph.

3.6. Source transcription as a separate boundary

Some finite formal objects originate in diagrams or tables that a kernel cannot visually interpret. The **KlarnerConstant** patterns were manually compared against Bui’s neighborhood diagrams. The squared-square placement comes from a Bouwkamp code. Once coordinates are entered, all local partitions, injections, coverage checks, and arithmetic can be formal, but “these coordinates faithfully transcribe that page” remains a human source boundary unless a parser for the source artifact is itself verified and used.

The repository’s strongest documentation names this boundary rather than confusing it with mathematical proof. A reviewer can then distinguish:

1. correctness of the formal theorem relative to entered data;
2. correctness of the data transcription;
3. novelty or historical attribution;
4. reproducibility of the candidate-generation process.

Only the first is necessarily a kernel theorem. All four can be important.

3.7. A review checklist

Table 3.2.: Trust-oriented questions for a headline theorem.

Question	Where to answer it
What is the literal theorem type?	The endpoint module and its audit, not the README paraphrase alone.

Question	Where to answer it
Is the result imported, re-proved, or computed?	Imports, proof body, and project provenance notes.
Which evaluator closes finite computation?	The proof term (<code>decide</code> , <code>native_decide</code> , <code>vm_compute</code>) and trust-boundary section.
Can generated data be adversarial?	The checker correctness theorem and its quantification over candidate data.
Does a premise survive?	The theorem type, <code>\#check</code> , and status table.
Are all intended modules built?	Facade imports, Lake globs/targets, or Rocq source registry.
Does an audit cover the endpoint?	<code>Audit.lean/Audit.v</code> ; check the exact declaration name.
Do Lean and Rocq prove the same statement?	Coverage ledger, encoding contract, and hypothesis lists.
Is the source transcription formalized?	Project README's trust-boundary or provenance section.
Is an omitted theorem intentionally omitted?	"Deliberate boundary," "nonclaims," or "residue" documentation.

Primary files

[README.md](#); [Algebra/JacobianConjecture/Lean/JacobianConjecture/Audit.lean](#); [Combinatorics/SquaredSquare/README.md](#); [Combinatorics/Polyominoes/KlarnConstant/README.md](#); [Computability/TuringDegrees/COVERAGE.md](#); [Logic/Interpretability/ZFCinPA/README.md](#).

4. Recurring module architectures

4.1. Facade, implementation, endpoint, audit

The most common project shape is:

facade $\longrightarrow$ definitions and lemmas $\longrightarrow$ headline endpoint $\longrightarrow$ audit.

The arrow here means “imports or exposes,” not necessarily one file per stage. A facade normally has little or no theorem body. The implementation modules are organized by mathematical responsibility. An endpoint module assembles reusable lemmas into the theorem promised by the README. The audit imports the public surface and asks the prover to display types and assumptions.

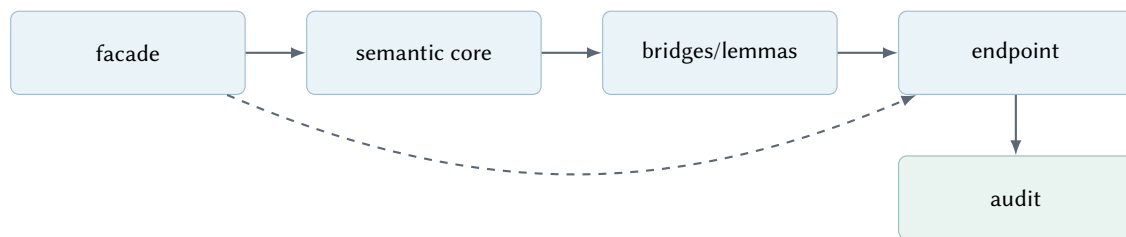


Figure 4.1.: The ordinary hand-written project shape. A facade may import every stage directly, but the conceptual dependency direction remains useful.

Examples:

- **FirstOrder:** $\text{Fol} \rightarrow \text{Calculus} \rightarrow \text{Completeness} \rightarrow \text{ClassicalCompleteness/Compactness} \rightarrow \text{Audit}$.
- **PresburgerArithmetic:** $\text{Syntax} \rightarrow \text{NormalForm} \rightarrow \text{Cooper} \rightarrow \text{Elimination} \rightarrow \text{Decision} \rightarrow \text{Audit}$.
- **JacobianConjecture:** shared definitions and the main map, collision/scaling/equivariance/simplification/stabilization modules, then a broad audit.

This pattern gives a reliable debugging rule: an error in a facade import is usually downstream of the true source. Build or elaborate the lowest module that still reproduces the problem.

4.2. The explicit semantic ladder

Metatheory projects make levels that ordinary application code often merges:

1. object-language syntax;
2. variable operations such as shift, substitution, or renaming;
3. interpretation in a structure under an assignment;
4. derivation objects or a provability relation;
5. soundness;
6. model construction or decision;
7. completeness/compactness/consistency endpoint.

The separation is not ceremonial. A theorem about syntactic consequence can be wrong even if the semantic theorem it resembles is true. A coded proof predicate can inspect only conclusions even though the intended statement bounds every formula occurrence. A substitution theorem can hold on standard quoted terms while failing for nonstandard codes in a nonstandard model.

The code therefore contains many “bridge” modules whose only purpose is to connect adjacent levels: quotation adequacy, decoder soundness, erasure from a data-carrying proof tree to a Prop-valued derivation, translation identity, truth lemmas, or model-relative satisfaction specifications.

4.3. Data/checker/meaning for finite certificates

A robust finite certificate has three logically distinct components.

Data

The concrete list, table, trace, placement, machine, sparse polynomial, or decomposition.

Checker

An executable predicate or function that computes whether the data satisfy a finite condition.

Meaning theorem

A proof that acceptance implies the mathematical property stated by the endpoint.

For a squared-square dissection, data are integer rectangles; the checker establishes bounds, separation, distinct sides, and exact unit-cell coverage; analytic lemmas lift that certificate to closed squares in the real plane. For Busy Beaver, data are machine traces or classification tables; semantics connect trace replay to halting score. For Dummit coefficients, data are sparse polynomial normal forms; substitution and normalization semantics connect list equality to the polynomial identity.

Proof boundary

A checker that validates only a *necessary* condition cannot justify an endpoint that needs a sufficient condition. Completeness of the checker for the consumed claim is a theorem obligation, not a property conferred by calling the data a certificate.

4.4. Generated sharding

Large exact expressions are split for both computational and software reasons. Common names reveal the split:

- Data: literal generated definitions;
- Certificate: equality or acceptance proofs;
- TermNN: one source term’s contribution;
- TailNN: an inductive suffix of a long sum;
- RowData: stable shared table input;
- Support: helper definitions or finite support;
- Shard, Part, or numbered coefficient modules: bounded pieces of one global identity;
- Core, Common, Bridge: hand-written semantic infrastructure intended to remain small.

A typical tail certificate has the following shape:

Listing 4.1: Representative generated-certificate composition.

```
import PolynomialFormulas...Tail22Data

set_option maxRecDepth 1000000
set_option maxHeartbeats 20000000

theorem table0_tail22_merge_certificate :
  table0Tail22Candidate = table0Tail22Normal := by
  decide

theorem table0_tail22_certificate :
  SparsePolynomial.substitute table0Tail22 elementaryPolynomials =
```

```

    table0Tail22Normal := by
change add (substitute term88 ...) (
  add (substitute term89 ...) (
    add (substitute term90 ...) (
      add (substitute term91 ...)
        (substitute table0Tail23 ...)))) = table0Tail22Normal
rw [table0_term88_certificate,
    table0_term89_certificate,
    table0_term90_certificate,
    table0_term91_certificate,
    table0_tail23_certificate]
exact table0_tail22_merge_certificate

```

The long `change` statement exposes definitional structure so rewriting can compose small certificates. The final `decide` compares concrete normalized lists. A reviewer should read the sparse data type and substitution semantics first, then one representative term certificate and one tail certificate, and only then inspect generator code.

4.5. Compatibility facades

Refactoring large developments often leaves small modules that preserve an old import surface. The **KlarnetConstant** README names `GeometricPPartition.lean`, `GeometricQ.lean`, and `GeometricVW.lean` as compatibility facades over newer multi-stage chains. These files are not dead weight: they decouple downstream imports from an internal split and make git history easier to follow.

Likewise, the root `ProveIt.lean` is a compatibility and convenience surface over a physical taxonomy. Removing facades merely because they have no declarations would make the import graph more brittle.

4.6. Parallel but noncomposable semantics

Two formalizations can both be correct and still be unsafe to compose without a bridge. **CombinatoryLogic** calls this out explicitly: its local lambda calculus and the lambda semantics used by the vendored Undecidability library have different operational presentations. Theorems are not silently chained across them.

The same warning applies to:

- quotient versus setoid presentations of Turing degrees;
- different natural-number encodings of lists;
- the repository-local fixed first-order language versus Mathlib’s arbitrary signatures;
- Foundation’s set-theory language versus the repository-local `Form` used by the lightweight ZF project;
- external standard numerals versus internal elements of a nonstandard model.

A bridge needs a translation and a theorem preserving the relevant semantics or derivability. Similar notation and matching informal intent are not enough.

4.7. Status documentation as code architecture

Some README sections function almost like interfaces:

Status table

Lists exact endpoints and whether each is proved, conditional, open, or proved only at standard indices.

Coverage ledger

Maps source claims to Lean/Rocq declaration names and states missing claims.

Residue

Orders remaining obligations by dependency.

Trust boundary

Separates kernel facts, computation modes, generated candidates, and transcription.

Deliberate boundary/nonclaims

Prevents a partial development from being mistaken for a theorem inventory.

For fast-moving formalization, this documentation is not secondary. It is the human-readable counterpart of theorem types and build targets. The strongest examples are [Computability/TuringDegrees/COVERAGE.md](#), [Logic/Interpretability/ZFCinPA/README.md](#), and the bounded-consistency READMEs.

4.8. Resource controls are part of module design

Options such as `maxHeartbeats` and `maxRecDepth` are common in generated certificate modules. Their presence says something architectural:

- the theorem is expected to close by reduction or large rewriting;
- the shard size was chosen against elaborator limits;
- moving a definition from reducible data to an opaque theorem can alter performance drastically;
- a “cleanup” that merges modules may make the proof infeasible even if the mathematical term is equivalent.

Worker settings in build instructions similarly reflect memory behavior. One large module per process can be safer than many concurrent modest modules. Generated certificate topology is therefore partly a scheduling design.

4.9. Naming as a navigation API

Names generally tell a reviewer where to look:

Table 4.1.: Frequent naming signals.

Name fragment	Likely role
Basic, Core, Defs	Fundamental datatypes and semantics.
Common, Support	Shared technical lemmas, often not a public endpoint.
Syntax, Coding	Object syntax or numeric encodings.
Evaluation, Semantics, Soundness	Meaning bridge.
Compiler, Translation, Adapter, Bridge	Preservation across representations.
Completeness, Endpoint, Main	Assembly toward a headline theorem.
Audit	Public declarations and assumption reporting.
Data, Table, Matrix	Literal or generated finite objects.
Certificate, Check	Acceptance proof for data.
Research, Article	Exposition/provenance; not automatically built theorem code.
Generate, verify_	External or executable support tooling.

These conventions are strong enough that a reader can often reconstruct the intended dataflow from a directory listing before opening a proof.

Primary files

Logic/FirstOrder/README.md; Logic/PresburgerArithmetic/README.md; Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitKernelCertificateTable0Tail22Certificate.lean; Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitKernelCertificateTable0Tail22Data.lean; Combinatorics/Polyominoes/KlarnerConstant/README.md; Computability/CombinatoryLogic/README.md.

Part II.

Subject-by-subject walkthrough

5. Algebra

5.1. Domain overview

[Algebra](#) contains only two project directories, but they occupy very different architectural scales.

JacobianConjecture

A compact explicit polynomial-map construction. Its main facts are determinant and collision identities, followed by invariant-preserving transformations such as scaling, stabilization, and degree-lowering shears.

PolynomialFormulas

A broad theory and computation project: low-degree formulas, algebraic radical syntax, numerical approximation, Abel–Ruffini, per-polynomial solvability decisions, and the very large Lazard/Dummit invariant-theory certificate program.

The first project demonstrates the ordinary definitions–calculation–endpoint–audit pattern. The second demonstrates how that pattern scales when one theorem consumes hundreds or thousands of exact generated sub-certificates.

5.2. JacobianConjecture

5.2.1. Public claim and mathematical object

The central module [Algebra/JacobianConjecture/Lean/JacobianConjecture/Counterexample.lean](#) defines an n -dimensional polynomial map over a commutative semiring as

$$\text{PolynomialMap}(n, R) = \text{Fin}(n) \rightarrow R[X_0, \dots, X_{n-1}],$$

then defines coordinatewise point evaluation, the formal Jacobian matrix $\partial f_i / \partial X_j$, its determinant, the Keller condition “nonzero constant Jacobian,” and polynomial invertibility via the induced maps on points.

The displayed three-variable map is

$$\begin{aligned} P(x, y, z) &= (1 + xy)^3 z + y^2(1 + xy)(4 + 3xy), \\ Q(x, y, z) &= y + 3x(1 + xy)^2 z + 3xy^2(4 + 3xy), \\ R(x, y, z) &= 2x - 3x^2 y - x^3 z. \end{aligned}$$

The formal determinant is proved equal to -2 . The module also proves that the distinct integral points

$$(-1, 1, 5), \quad (0, -2, -16)$$

both evaluate to $(0, -2, 0)$. It therefore derives noninjectivity, absence of a pointwise left inverse, absence of a polynomial two-sided inverse, and the negation of the repository’s fixed-dimensional and all-dimensional Jacobian-conjecture predicates over characteristic-zero fields.

This was an exceptionally recent mathematical development at the snapshot: the explicit map was publicly announced in July 2026. The code walkthrough does not rely on community-status language; the local formal evidence is particularly transparent because the two decisive claims are finite polynomial identities.

5.2.2. Core definitions

The public API in `Counterexample.lean` is intentionally small:

Listing 5.1: Normalized outline of the Jacobian core definitions.

```
abbrev PolynomialMap (n : Nat) (R : Type u) [CommSemiring R] :=
  Fin n -> MvPolynomial (Fin n) R

noncomputable def PolynomialMap.evalMap
  (f : PolynomialMap n R) (a : Fin n -> R) : Fin n -> R :=
  fun i => MvPolynomial.eval a (f i)

noncomputable def PolynomialMap.jacobian
  (f : PolynomialMap n R) :
  Matrix (Fin n) (Fin n) (MvPolynomial (Fin n) R) :=
  fun i j => pderiv j (f i)

noncomputable def PolynomialMap.jacobianDet
  (f : PolynomialMap n R) : MvPolynomial (Fin n) R :=
  (jacobian f).det

def PolynomialMap.HasNonzeroConstantJacobian (f : PolynomialMap n R) : Prop :=
  exists c : R, c != 0 /\ jacobianDet f = C c

def PolynomialMap.HasPolynomialInverse (f : PolynomialMap n R) : Prop :=
  exists g, LeftInverse (evalMap g) (evalMap f) /\
    RightInverse (evalMap g) (evalMap f)
```

The inverse predicate is stated extensionally on points. This is sufficient for the refutation: any two-sided polynomial inverse supplies a left inverse, and a left inverse forces injectivity. The module never needs to formalize polynomial composition equality in order to conclude that the displayed map has no polynomial inverse.

5.2.3. Determinant path

Rather than asking the general matrix determinant simplifier to normalize the entire expression opaquely, the file defines the closed 3×3 determinant formula `det3` and proves it equals `Mathlib's Matrix.det`. It then:

1. unfolds the three coordinates and formal partial derivatives;
2. supplies a helper proving that numeric literals have zero `pderiv`;
3. normalizes the resulting polynomial identity with `ring`;
4. transports the result through `det3_eq_det`;
5. rewrites the constant polynomial into `C (-2)`.

The expensive local lemma receives scoped `maxHeartbeats 1600000` and `maxRecDepth 8000`. This keeps the resource concession adjacent to the calculation that needs it rather than globally weakening safeguards.

The reusable theorem `det_eq_det_toBlocks11` handles a block lower-triangular matrix with zero upper-right block and identity lower-right block. Later five-variable and stabilized determinant arguments reduce to this lemma. This is a good example of extracting the mathematical structure from what could otherwise become repeated large ring calls.

5.2.4. Collision path

The collision proof is intentionally elementary:

1. define the two points and common value as `Fin 3 -> R` vectors;
2. prove each coordinate equality by function extensionality;
3. split the finite coordinate index with `fin_cases`;

4. evaluate with `norm_num`;
5. prove the points distinct by comparing coordinate zero;
6. contradict injectivity.

The determinant theorem is polymorphic over every commutative ring. The Keller condition adds `Field` and `CharZero` only where $-2 \neq 0$ is needed. The collision theorem needs merely a nontrivial commutative ring. This stratification makes the algebraic content sharper than a proof written only over $\mathbb{C}$.

5.2.5. Supporting modules

Table 5.1.: JacobianConjecture Lean module responsibilities.

Module	Role
Counterexample	Definitions, determinant, integral collision, noninjectivity, and the dimension-three/all-dimensions endpoints.
CollisionFamily	A parameterized family of collisions and useful specializations.
Equivariance	Symmetries of the map and mirrored/rational collision consequences.
Scaling	Source/target scalings, composition laws, and scaled collision families.
SimplerCounterexample	A stably equivalent degree-lowered presentation, including total-degree bounds and its own determinant/collision endpoint.
Stabilization	Adjoins identity coordinates, transports evaluation and determinant, and extends the counterexample to every dimension at least three.
Audit	Checks and prints axioms for the public declarations across all of the above.

The audit is unusually comprehensive. It does not stop at one final negation; it enumerates determinant identities, all collision witnesses, equivariance, scaling laws, degree profiles, simplified maps, and stabilization bridges. That makes [Algebra/JacobianConjecture/Lean/JacobianConjecture/Audit.lean](#) the fastest public-API index.

The Rocq side uses a small polynomial-expression syntax and interprets it over Coqueliot's complex numbers. Its source sequence mirrors the main themes: `Common.v`, `Counterexample.v`, `Scaling.v`, `CollisionFamily.v`, `SimplerCounterexample.v`, and `Audit.v`. It is an independent proof artifact rather than an extraction of the Lean theorem.

5.2.6. Recommended reading order

Reading route

1. [Algebra/JacobianConjecture/README.md](#) for status and build.
2. The definitions and two decisive lemmas in [Algebra/JacobianConjecture/Lean/JacobianConjecture/Counterexample.lean](#).
3. `Stabilization.lean` to see how a dimension-specific witness is transported.
4. `SimplerCounterexample.lean` for the stable degree-lowering construction.
5. `Audit.lean` as the declaration checklist.
6. The Rocq modules only after the Lean object and theorem surface are clear; their representation is intentionally different.

5.3. PolynomialFormulas

5.3.1. Why this project needs a map before a file list

PolynomialFormulas spans several mathematically distinct layers:

1. exact formulas for degrees one through four;
2. an algebraic language of radicals and exact root collections;
3. executable Gaussian-rational approximation;
4. impossibility theorems such as Abel–Ruffini;
5. per-polynomial radical-solvability decision procedures;
6. general resolvent and invariant-theory infrastructure;
7. explicit quintic constructions in the style of Lazard;
8. generated sparse-polynomial tables and kernel certificates.

A flat alphabetical listing hides this progression. The facade [Algebra/PolynomialFormulas/Lean/PolynomialFormulas.lean](#) provides a much better overview. Its imports can be grouped as follows.

Table 5.2.: Conceptual strata of the PolynomialFormulas facade.

Stratum	Representative imports	Responsibility
Classical formulas	Basic, Cubic, Quartic	Low-degree equations, algebraic branch conditions, and root exhaustiveness.
Exact radical execution	GaussianPolynomialSolver, GaussianRadicalBounds, GaussianPolynomialApproximation	Finite radical-expression syntax over Gaussian rationals, evaluation relations, and certified approximations.
Quintic constructions	LazardQuintic, LazardQuinticFourier, LazardQuinticSectionFiveCombinedAction, GaussianQuinticSolver	Resolvents, Fourier/invariant coordinates, conditional solver packaging, and executable fallback behavior.
Impossibility and decision	AbelRuffini, QuinticRadicalDecision, SexticRadicalDecision, SelmerAbelRuffini, GenericAbelRuffini	Generic and explicit nonsolvability plus decidability for concrete low-degree integer polynomials.
Examples	Examples	Small end-to-end specializations demonstrating the public API.

The facade does not import every internal Lazard certificate module directly; higher-level modules aggregate them. Consequently the import graph, not the directory’s file count, defines the current endpoint.

5.3.2. Low-degree formulas: relations before branches

The degree-two, degree-three, and degree-four developments avoid treating a computer-language square root as if it were an algebraic branch-independent operation. A radical witness is characterized by an equation such as $s^2 = \Delta$, and the theorem states what follows for any witness satisfying that relation. This has several benefits:

- the core theorem works over fields/rings with the stated algebraic properties rather than only over $\mathbb{C}$;
- branch conventions do not leak into purely algebraic identities;
- degeneracies and characteristic restrictions remain visible;
- an executable complex solver can later choose a principal branch and prove it instantiates the abstract relation.

Root collections are proved sound and exhaustive where the hypotheses permit it. This is stronger than showing that substituting one displayed formula yields zero: it explains how multiplicities, zero discriminants, and branch families cover every root.

5.3.3. Gaussian-rational radical syntax

For executable exact work, the project defines finite expression trees whose leaves are Gaussian rationals and whose nodes represent arithmetic and selected radicals. A solver can return such a tree without pretending that its transcendental numerical rendering is exact. Separate modules prove:

- algebraic evaluation or root correctness;
- bounds on radical magnitude and propagation of approximation error;
- a computable approximation procedure to a requested precision;
- compatibility between exact syntax and approximate complex values.

This separation is important for the quintic layer. The theorem that a returned expression denotes a root is exact; approximation is an interface for observing or testing the expression, not the proof of roothood.

5.3.4. Abel–Ruffini surfaces

The project contains several related but nonidentical impossibility results.

Explicit example

A concrete quintic, such as $X^5 - 4X + 2$, is shown not solvable by radicals through imported and locally bridged Galois theory.

Rootwise form

The statement is expressed in terms of which individual roots can lie in radical towers, not merely whether a splitting field is solvable.

Selmer family

$X^n - X - 1$ supplies a family across arbitrary degrees under the relevant hypotheses.

Generic form

The symmetric rational-function extension gives the usual generic obstruction and connects it to explicit resolvents.

On the Rocq side, the MathComp-Abel submodule provides the deep Galois-theory base. Local modules adapt its theorem surface to the project’s polynomial definitions and examples. A reviewer should therefore distinguish the local bridge proof from the upstream Abel theorem itself.

5.3.5. Deciding radical solvability of a concrete quintic

The decision project is not a universal formula for quintic roots. It takes a specific integer/rational quintic and decides whether its Galois group is solvable using finite exact data. Conceptually, the Lean path is:

1. normalize the polynomial and split reducible cases by bounded factor search;
2. handle low-degree factors with known radical formulas;
3. for the irreducible quintic, compute exact resolvent data;
4. use Frobenius/Dummit-style criteria and a Chapman correction to classify the relevant transitive group cases;
5. turn the finite decision into a `ComputablePred`;
6. connect the algorithm to the repository’s Turing-machine interface.

The bounded factor search is primitive recursive. The resulting quintic decision is packaged strongly enough to obtain a verified machine-level computability result. “Decidable” here means an executable theorem about each concrete polynomial, not that the generic quintic has a radical formula.

5.3.6. The sextic decision

The sextic project generalizes the strategy but changes the recursion classification. It uses degree-15 and degree-10 resolvents and a finite classification of transitive subgroups of S_6 . The repository claims a recursive/computable decision, not the stronger primitive-recursive packaging advertised for the quintic. Lean connects the procedure to a verified Turing machine; the Rocq version uses its own μ -recursive infrastructure.

This distinction is a good example of accurate theorem granularity: computability, primitive recursiveness, and practical runtime are three different claims.

5.3.7. The Lazard/Dummit semantic core

Before the generated modules, a small hand-written representation defines five-variable sparse monomials:

Listing 5.2: Core sparse representation used by Dummit certificates.

```
structure Powers where
  p0 : Nat
  p1 : Nat
  p2 : Nat
  p3 : Nat
  p4 : Nat
deriving Repr, DecidableEq

structure SparseTerm where
  coeff : Int
  powers : Powers
deriving Repr, DecidableEq

abbrev SparsePolynomial := List SparseTerm

def SparseTerm.eval [CommRing R]
  (t : SparseTerm) (x : Fin 5 -> R) : R := ...

def SparsePolynomial.eval [CommRing R] :
  SparsePolynomial -> (Fin 5 -> R) -> R
| [], _ => 0
| t :: q, x => t.eval x + SparsePolynomial.eval q x
```

The same core supplies addition, multiplication, substitution, normalization, and evaluation lemmas. A seven-entry `dummitTable` stores coefficient polynomials in ascending sextic degree. This representation is deliberately simple: lists and integer exponent tuples have decidable equality and reduce predictably. Sophisticated invariant theory is proved *around* a computation-friendly certificate language.

5.3.8. A concrete generated certificate chain

At the pinned commit, [Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitKernelCertificateTable0Tail22Data.lean](#) imports shared row data, the next tail certificate, and certificates for terms 88 through 91. It defines a very large literal normalized sparse polynomial. The adjacent [Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitKernelCertificateTable0Tail22Certificate.lean](#) then proves two facts:

1. the assembled candidate list equals the stored normal list, by kernel `decide`;
2. formal substitution of the source tail equals that normal list, by rewriting the four term certificates and the next-tail certificate.

The full proof path is therefore:

A bug in the generator should cause a certificate theorem to fail. A review of the generator is still valuable for reproducibility and shard completeness, but the theorem's soundness is meant to rest on the formal operations and their semantic lemmas.

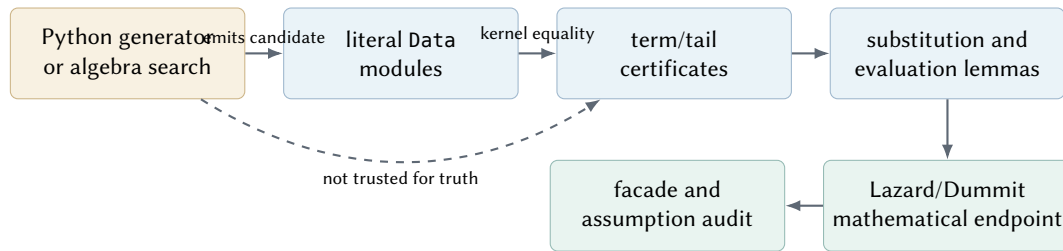


Figure 5.1.: Generated certificate path in PolynomialFormulas.

5.3.9. Generator programs and reproducibility

The support directory contains substantial generators, including:

- `generate_dummit_kernel_certificates.py`;
- `generate_fin5_transitive_classification_certificates.py`;
- `generate_lazard_modular_product_tail_certificates.py`;
- `generate_lazard_root_fourier_numerator_p2_sparse.py`.

They do more than print constants: they choose shard boundaries, canonicalize supports, name dependencies, and emit source whose import graph composes the certificate. Consequently they are build-system generators as well as mathematical calculators. Deterministic output and a clean regeneration diff are important maintenance properties even though the generated truth is rechecked.

The repository also keeps expository and crosswalk artifacts such as [Algebra/PolynomialFormulas/LazardQuintic Formalization.tex](#) and the paper-claim crosswalk. These explain which formal module discharges which step of a mathematical exposition. For a development of this size, the crosswalk is often a more useful index than the file tree.

5.3.10. Checkpoint status at the pinned commit

The commit message for the inspected snapshot records a concrete build checkpoint rather than claiming the entire long-term Lazard program finished. Among the recorded milestones were:

- a bounded eight-double-coset certificate stack;
- a focused-kernel-green five-class S_5 classifier and four-class corollary;
- the bridge from classification to resolvent criteria;
- Dummit table-zero certificates through tail 22 and a specified term frontier;
- a deterministic 563-module root plan with three bounded product shards;
- a regenerated 28-page formalization report.

The same checkpoint still identifies pending monolithic root-state data and later terms. These statements are best read as build-plan coordinates: specific module families were green, while the overall construction retained a known frontier.

5.3.11. How to read the large project without drowning

Reading route

1. Start with [Algebra/PolynomialFormulas/README.md](#) and the facade [Algebra/PolynomialFormulas/Lean/PolynomialFormulas.lean](#).
2. Read `Basic`, `Cubic`, and `Quartic` to learn the abstract radical convention.
3. Choose one executable path: `GaussianPolynomialSolver` plus approximation modules.
4. Read `AbelRuffini` and one explicit example to understand imported Galois-theory dependencies.
5. For decision procedures, trace factor search → resolvent criterion → computability wrapper.
6. Before any generated file, read `ComputableDummitCoefficientsCore.lean`.
7. Inspect one `Term...Certificate` and one `Tail...Certificate`; use them as templates for the rest.
8. Use the formalization report/crosswalk to locate the current mathematical endpoint and exact residue.

5.3.12. Maintenance hazards

Gotcha

The biggest hazards in this subtree are architectural rather than algebraic.

- A generated module can exist but be unreachable from the facade or intended build target.
- Reordering or merging shards can exceed recursion/heartbeat limits.
- A generator can regenerate a syntactically different but extensionally equal normal form, producing enormous noisy diffs.
- A small bridge theorem can carry the real mathematical meaning while hundreds of data modules merely support equality.
- “The file compiles” is weaker than “the endpoint imports the file.”
- A fast native replay and a kernel certificate are different validation tiers and should not be substituted silently.

Primary files

[Algebra/README.md](#); [Algebra/JacobianConjecture/README.md](#); [Algebra/JacobianConjecture/Lean/JacobianConjecture/Counterexample.lean](#); [Algebra/JacobianConjecture/Lean/JacobianConjecture/Audit.lean](#); [Algebra/PolynomialFormulas/README.md](#); [Algebra/PolynomialFormulas/Lean/PolynomialFormulas.lean](#); [Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitCoefficientsCore.lean](#); [Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitKernelCertificateTable0Tail22Data.lean](#); [Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitKernelCertificateTable0Tail22Certificate.lean](#); [Algebra/PolynomialFormulas/LazardQuinticFormalization.tex](#).

6. Analysis

6.1. Overview

The analysis subtree is compact enough to read end to end. It contains **ExponentialIdentities** and **TrigonometricIdentities**, each with a small facade, a handful of focused proof modules, and paired Lean/Rocq work where appropriate. Unlike the certificate-heavy combinatorics and algebra projects, these developments are dominated by analytic inequalities and exact symbolic transformations rather than generated source.

6.2. ExponentialIdentities

The standout module is [Analysis/ExponentialIdentities/Lean/ExponentialIdentities/TinyExponentTower.lean](#). It proves the exact floor identity

$$\left\lfloor 10^{10^{10^{10^{10^{-10^{10}}}}} \right\rfloor - 10^{10^{10}} = 2\,811\,012\,357\,389.$$

The expression is visually enormous but analytically close to a first-order perturbation. The file introduces

$$N = 10^{10}, \quad t = 10^{-N}, \quad L = \log 10,$$

then names the successive tower layers u, v, c and the final real value. Its proof strategy is more instructive than a direct interval evaluation.

6.2.1. Step 1: an exact rational enclosure for $\log 10$

The module proves 10^{-20} -scale bounds for $\log 2$ and $\log(5/4)$ using Mathlib's finite logarithmic-series remainder theorem. It combines

$$\log 10 = 3 \log 2 + \log(5/4)$$

to obtain a narrow rational interval around

$$\frac{230258509299404568403}{10^{20}}.$$

This interval is then raised to the fourth power with monotonicity lemmas. The key first derivative quantity is shown to satisfy

$$M + \frac{2}{5} < 10^{11} L^4 < M + \frac{1}{2}, \quad M = 2\,811\,012\,357\,389.$$

The integer in the final answer is therefore not guessed from a floating-point tower; it is isolated by a rational inequality for the derivative coefficient.

6.2.2. Step 2: propagate the tiny increment

For $0 \leq x \leq 1$, the module derives

$$0 \leq e^x - 1 - x \leq x^2.$$

A reusable `exp_layer` lemma says that if an increment w is bounded by Kt , then one additional exponentiation has a nonnegative quadratic remainder and a new increment bounded by $4Kt$. The three inner exponential layers instantiate this lemma with moderate constants. Since $t = 10^{-10^{10}}$, even deliberately crude constants annihilate all higher-order terms.

The proof never attempts to materialize $10^{10^{10}}$ as a natural number. It works with real exponentiation, logarithmic identities, and order properties; the gigantic scale appears symbolically in powers whose monotonicity is easy to prove.

6.2.3. Step 3: convert a logarithmic increment to a floor

The final layer is written as a base point 10^N multiplied by an exponential perturbation. Lower and upper inequalities show that the excess lies strictly between M and $M + 1$. Standard floor lemmas then give the integer identity.

Orientation

The module is a model for formalizing “numerically obvious but astronomically large” identities: isolate a small parameter, prove a coarse analytic remainder bound, reserve high precision only for one moderate constant, and avoid evaluating the large baseline altogether.

The Rocq counterpart uses Coquelicot/Interval infrastructure for the same exact claim. The two implementations have comparable mathematical decomposition but different interval machinery and trust profiles.

6.3. TrigonometricIdentities

The trigonometric project contains exact identities such as an arctangent square relation and golden-ratio trigonometry. Representative modules include `ArctanSquareIdentity.lean` and `TrigGoldenRatio.lean`, with Rocq counterparts.

The recurring pattern is:

1. reduce the displayed identity to algebraic statements about $\sin$, $\cos$, or $\tan$;
2. prove domain and sign conditions needed by inverse trigonometric functions;
3. use exact polynomial identities for the special constants;
4. invoke injectivity or interval-restricted inverse lemmas to recover the target angle.

The sign and interval obligations are the nontrivial formalization cost. A paper proof may silently select a quadrant after squaring or applying $\tan$; the formal proof makes that branch selection explicit.

6.4. How the analysis projects fit the repository

These projects are useful reference points because they contain almost none of the monorepo’s generated-certificate complexity. Their build graph is close to their mathematical argument, and their source files demonstrate:

- local helper lemmas for exact constants;
- careful control of inverse-function side conditions;

- rational enclosure rather than floating-point evidence;
- scoped resource options rather than global relaxation;
- paired proof-assistant results without a generator layer.

Reading route

Read `TinyExponentTower.lean` in four passes: declarations and theorem statement; logarithm enclosures; generic exponential-layer lemmas; final floor argument. Reading it linearly on the first pass obscures the fact that the proof is one reusable perturbation lemma instantiated three times.

Primary files

[Analysis/README.md](#); [Analysis/ExponentialIdentities](#); [Analysis/ExponentialIdentities/Lean/ExponentialIdentities/TinyExponentTower.lean](#); [Analysis/TrigonometricIdentities](#); [Analysis/TrigonometricIdentities/Lean/TrigonometricIdentities/ArctanSquareIdentity.lean](#); [Analysis/TrigonometricIdentities/Lean/TrigonometricIdentities/TrigGoldenRatio.lean](#).

7. Combinatorics

7.1. Overview

The combinatorics subtree contains four different kinds of finite reasoning:

1. recursive syntax and value-set enumeration for expression trees;
2. exact finite geometric certificates;
3. finite injections that prove coefficient recurrences;
4. asymptotic arguments that convert recurrences or supermultiplicativity into growth bounds.

The main projects are expression enumeration (power towers and radical expressions), derivative-expansion sequences, perfect squared squares, and Klarner’s polyomino growth constant.

7.2. Expression enumeration

7.2.1. PowerTowers

The facade [Combinatorics/ExpressionEnumeration/PowerTowers/Lean/PowerTowers.lean](#) imports a shared core and sequence-specific developments for A000081, A002845, A199812, and A198683. It also imports selected small-value certificates and a substantial $n = 12$ analysis for A198683.

The shared core separates:

- binary parenthesization syntax;
- structural size and recursive decomposition;
- interpretation under a chosen base and exponentiation convention;
- finite sets or multisets of attainable values;
- counting quotients where distinct syntax can denote the same value.

This distinction is essential. Counting binary trees alone is a Catalan-style problem; counting *distinct values* requires a semantic equality analysis whose difficulty depends heavily on the base.

7.2.2. A198683: principal values of towers of i

A198683 counts distinct principal complex values of all binary parenthesizations of $i^i \cdots^i$. The project defines `principalPow` through the principal complex logarithm and recursively collects value sets. Small cases combine finite-set decomposition with analytic proofs that candidate values are distinct.

For $n = 7$, for example, the proof constructs several explicitly separated families. Injectivity of maps such as $z \mapsto i^z$ is established on carefully bounded real strips; signs of real or imaginary parts separate image families; finite cardinalities then supply the lower bound matching the computed upper bound.

This architecture avoids a fragile list of pairwise “not equal” proofs. A geometric invariant—strip injectivity, norm, or sign—separates whole families at once.

7.2.3. Generated data and the $n = 12$ frontier

The physical layout under A198683 is informative:

- `SmallValues` contains hand-guided and bounded certificates;
- `DataCertificates` contains imported/generated rows and replay lemmas;
- `N12` contains probe, magnitude, overflow, symbolic, certificate, endpoint, and bound modules.

The main A198683.lean comment says that the accepted OEIS data field runs through $n = 11$ and deliberately withholds $n = 12$ while a proof-quality final certificate is outstanding. At the same snapshot, the separate $n = 12$ subtree has made substantial progress: endpoint estimates for a near-1 split are proved unconditionally by rational boxing, monotone transport, and Taylor bounds. The endpoint module even records two earlier false numerical constants and proves their falsity before replacing them.

This is an excellent status pattern: a difficult intermediate can become fully formal without silently upgrading the final sequence-value claim.

Proof boundary

For A198683, floating-point clustering is a discovery aid only. Distinctness must ultimately come from exact symbolic relations or certified analytic separation, because principal complex exponentiation can bring enormous intermediate magnitudes back near one another.

7.2.4. A158415 and derivative expansions

The radical-expression project for A158415 has Lean and Rocq trees plus research/support material. Its architecture parallels power towers: recursive expression syntax, normalization/evaluation, finite enumeration, and certificates for initial sequence values.

The derivative-expansion work around A290268 lives under [Combinatorics/DerivativeExpansions](#) with an auxiliary top-level [Oeis/A290268](#) surface. It formalizes the combinatorics of iterated derivatives and the resulting coefficient sequence. These are smaller projects, but they reuse the same “syntax first, semantic quotient second” discipline.

7.3. SquaredSquare

7.3.1. What is proved

The project formalizes A. J. W. Duijvestijn’s order-21 dissection of a square of side 112 into pairwise noncongruent integer-sided squares. The side lengths are

2, 4, 6, 7, 8, 9, 11, 15, 16, 17, 18, 19, 24, 25, 27, 29, 33, 35, 37, 42, 50.

The Lean and Rocq developments independently prove:

- the listed placements cover the 112×112 square exactly;
- open interiors are pairwise disjoint;
- all side lengths are positive and distinct;
- distinct positive side lengths imply noncongruence;
- scaling yields a 21-piece perfect dissection of every positive-side square;
- every perfect squared square has at least seven pieces.

Therefore the minimum possible order is unconditionally bracketed by

$$7 \leq m \leq 21.$$

7.3.2. Geometry reduced to integer data

`Basic.lean` defines an axis-parallel closed square $[x, x + s] \times [y, y + s]$, its open box, dissections, and congruence. It proves that the explicit open box is the topological interior of the closed square. Congruence is handled through squared Euclidean distance: a positive-side square has squared diameter $2s^2$, so congruent squares have equal sides; equal sides are congruent by translation.

The finite placement proof then works over integers:

1. every piece lies inside the bounding grid;
2. every pair of open rectangles is separated;
3. the side list has no duplicates;
4. every unit grid cell belongs to exactly one piece.

Lean discharges the certificate with `decide +kernel`; Rocq uses `vm_compute`. Elementary floor and integer-part lemmas lift cell coverage to exact coverage of the real square.

7.3.3. The elementary lower bound

`Minimality.lean` formalizes the classical corner/edge argument.

1. Each corner of the outer square is covered by a unique corner tile.
2. No piece can have the full outer side, and the four corner tiles are pairwise distinct; hence at least four pieces.
3. Tiles meeting one edge partition that edge, so their side lengths sum to the outer side. The reusable one-dimensional theorem lives in `Intervals.lean`.
4. With at most two non-corner pieces, edge-incidence cases force either equal corner sides or a sum of positive lengths equal to zero.

The result excludes orders 1 through 6. It does not replay Duijvestijn’s historical exhaustive network search excluding orders 7 through 20.

7.3.4. Sharp minimality remains a named claim

`MinimalOrder.lean` defines `DuijvestijnSearchClaim` and proves the sharp statement “the least achievable order is 21” equivalent to that claim. The existence half and the lower bound seven are already formal; the complete planar-network enumeration is not.

The README spells out what a full formal search would require: commensurability, a complete enumeration of relevant planar structures, and a verified linear-algebra check for each structure. This is a precise future proof obligation, not an informal “computer search omitted” note.

7.3.5. Module path

Table 7.1.: SquaredSquare modules.

Module	Responsibility
<code>Basic</code>	Geometric definitions, interiors, diameter, and congruence.
<code>Duijvestijn</code>	Concrete 21-piece placement and kernel certificate.
<code>Scaling</code>	Dilation closure for arbitrary positive side.
<code>Intervals</code>	One-dimensional interval-cover sum theorem.
<code>Minimality</code>	Exclusion of perfect dissections with at most six pieces.
<code>MinimalOrder</code>	Least-order statement and reduction to the search claim.
<code>Audit / Coq/Audit.v</code>	Assumption reporting.
<code>Support/bouwkamp_decode.py</code>	Bouwkamp-code decoding and data checks only.

7.4. KlarnerConstant

7.4.1. Headline result

The project proves the exact upper bound

$$\lambda \leq \frac{9047}{2000} = 4.5235$$

for Klarner’s polyomino growth constant. The numerical improvement changes no geometric recurrence from the cited seventeen-neighborhood system; it supplies a new exact rational supersolution at $\zeta = 2000/9047$. All seventeen coordinates share denominator 10^7 .

The formal project goes well beyond checking those seventeen inequalities. It defines the actual fixed-polyomino counts, proves all required geometric recurrences by finite injections, establishes supermultiplicativity, and connects the pointwise exponential estimate to the conventional asymptotic growth limit.

7.4.2. The five layers

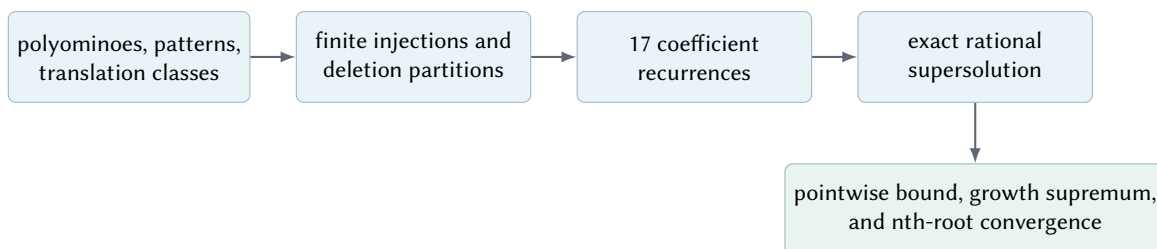


Figure 7.1.: KlarnerConstant proof flow. The final numeric certificate is small; proving that the actual geometric counts satisfy its recurrences is the bulk of the development.

Objects and counting. `Polyomino.lean`, `Patterns.lean`, and `Counting.lean` define finite edge-connected lattice animals, southwest normalization, required and forbidden offset patterns, and marked-occurrence counts. The seventeen coefficient sequences are therefore concrete finite cardinalities, not uninterpreted sequences postulated to satisfy a recurrence.

Translation quotient. `TranslationClasses.lean` constructs translation equivalence as a quotient, proves that each class has a unique southwest-normalized representative, and identifies the fixed count with the quotient cardinality.

Connected decomposition. `SeededPartition.lean` grows disjoint nonempty seeds inside a connected cell set into disjoint connected territories covering the original. The geometry modules use these territories as target factors in explicit injections.

Recurrence algebra. `Convolution.lean`, `Recurrence.lean`, `BuiSystem.lean`, `CoefficientSystem.lean`, and `PublishedSystem.lean` distinguish the paper’s literal indexing convention from an all-index algebraic encoding. They prove adapters between the two instead of silently changing base cases.

Exact certificate. `Certificate.lean` defines the seventeen-variable polynomial map and checks the rational supersolution with ordinary kernel-reduced `norm_num`. Python and Wolfram scripts independently replay the same rational inequalities, but Lean’s certificate is the theorem boundary.

7.4.3. The geometry module families

The recurrence proof is split by neighborhood type and proof mechanism.

Table 7.2.: Klarner geometric families.

Family	Role
<code>GeometricLinear</code>	Same-size occupancy partitions for the simplest coefficient identities.
<code>GeometricDeletion</code>	Injective marked-leaf deletion for several one-cell inequalities.
<code>GeometricPBasics/Core/Geometry/Endpoint</code>	Multi-branch P decomposition, recovery, and endpoint.
<code>GeometricQCore/Geometry/Endpoint</code>	Analogous Q chain.
<code>GeometricFourFive...</code>	The multi-stage S inequality.
<code>GeometricU...</code> , <code>GeometricV...</code> , <code>GeometricW...</code>	Separate connected-decomposition chains for the corresponding types.
<code>GeometricTwoDeletion</code>	One- and two-cell deletion maps for X, Y, Z , retaining enough coordinates for reconstruction.
<code>GeometricComplete</code>	Assembles all seventeen concrete recurrences and applies the certificate without recurrence or domination hypotheses in the final statements.

The recurring proof obligation is injectivity. It is not enough to map a marked polyomino to several connected territories; the target tuple must retain enough marks, offsets, and normalization data to reconstruct the source uniquely. “Geometry” modules define the map; “Core” and “Endpoint” splits keep reconstruction and cardinal arithmetic manageable.

7.4.4. Asymptotic endpoint

`Concatenation.lean` vertically stacks normalized polyominoes injectively, proving supermultiplicativity of the fixed count. `Asymptotic.lean` applies Fekete’s lemma to negative logarithms, identifies the limit with the supremum of positive-index n th roots, and derives convergence of the standard real roots.

`Growth.lean` separately proves the generic passage from a pointwise exponential bound to the growth supremum. `Main.lean` remains reusable and accepts abstract recurrence/domination hypotheses. The unconditional composition is `GeometricComplete.lean`.

7.4.5. Trust and transcription

The formal argument has no project axioms, sorry, unsafe definitions, or native-decision boundary according to the project README and audit. However, the seventeen finite coordinate tables in `Patterns.lean` were transcribed from source diagrams. Lean proves the theorem relative to those tables; the human comparison with the paper’s page is a separate provenance step.

This is exactly the right place for independent support scripts: they can replay arithmetic and detect transcription inconsistencies without being confused with the proof term.

7.5. Reading routes

Reading route

To understand finite certificate style: read **SquaredSquare** in the order Basic → Duijvestijn → Scaling → Minimality.

To understand finite-injection recurrence style: read **KlarnerConstant** in the order Polyomino/Patterns → SeededPartition → one small geometry family → CoefficientSystem → Certificate → GeometricComplete.

To understand analytic distinctness around generated values: read **PowerTowers** Core, one small A198683 value proof, then the $n = 12$ symbolic and endpoint modules.

Primary files

Combinatorics/README.md; Combinatorics/ExpressionEnumeration/PowerTowers/Lean/PowerTowers.lean; Combinatorics/ExpressionEnumeration/PowerTowers/Lean/PowerTowers/A198683.lean; Combinatorics/ExpressionEnumeration/PowerTowers/Lean/PowerTowers/A198683/N12/N12Endpoints.lean; Combinatorics/SquaredSquare/README.md; Combinatorics/Polyominoes/KlarnerConstant/README.md.

8. Computability

8.1. Overview

The computability subtree contains three projects with almost disjoint mathematical styles:

BusyBeaver

Executable finite-state machine semantics, trace/classification certificates, and explicit control of expensive build targets.

CombinatoryLogic

Hand-written operational semantics, compilers among lambda, SK, SKI, and Iota calculi, and universality simulations.

TuringDegrees

Higher computability theory over oracle reducibility, with intentionally different Lean and Rocq presentations and a claim-by-claim coverage ledger.

Together they show why “computability formalization” is not one architecture: finite execution, simulation of partial recursive functions, and abstract degree theory require different interfaces.

8.2. BusyBeaver

8.2.1. The facade is intentionally small

The Lean facade `BusyBeaver.lean` imports only `BusyBeaver.Core` and `BusyBeaver.KnownValues`. A source comment states that the expensive BB2 and BB3 certificates are opt-in.

This is a deliberate answer to a monorepo problem: users who need the machine model or known-value API should not pay for all classification shards. Separate libraries and executable generators remain available for focused builds.

8.2.2. Machine model

`Core.lean` lives in namespace `SetTheory.BusyBeaver`. It defines:

- head moves and tape actions;
- a machine transition table indexed by state and read symbol;
- typed and untyped machine presentations;
- a finite-support tape represented by lists around the head;
- configurations, one-step execution, and multi-step traces;
- halting and score functions;
- compiler/bridge interfaces used by classification code.

The tape representation is optimized for formal execution rather than for extensional infinite sequences. Blank tails are implicit. Read, write, and shift operations have executable definitions whose semantic lemmas are small enough to reuse in trace proofs.

8.2.3. Score is not runtime

The project distinguishes at least two natural Busy Beaver quantities:

- **halting time**: number of transitions before halt;
- **score**: number of marked symbols (or the project-specific output measure) at halt.

A machine can maximize one without maximizing the other. The theorem names and README keep these measures separate, and classification certificates must state which optimization they establish. This is particularly important when importing upstream BB certificates whose machine convention or objective may differ.

8.2.4. Classification and certificate modules

The internal Lean tree includes BB2, BB3, and BB4 families, with generated shards and audit modules. Small-state classification typically proceeds by:

1. normalize machines modulo explicit symmetries;
2. partition the finite machine space;
3. associate each representative with a halting trace, a domination argument, or a nonhalting classification;
4. replay the finite certificate with kernel decide;
5. lift representative results through the symmetry/normalization theorem;
6. derive the score bound and exhibit a matching champion.

Executable Lean programs such as `GenerateBB4R05A14Guided.lean` and `GenerateBB4R13C14Guided.lean`, plus PowerShell generation scripts, produce candidate certificate material. They are support tools; the checked trace/classification theorems are the intended boundary.

The project status distinguishes completed kernel classifications from conditional or incomplete BB4 root claims. A facade that omits a module is not itself evidence that the result is open; the project README and audits state the exact boundary.

8.2.5. Mathlib and Rocq bridges

`Mathlib.lean` connects the repository-local machine model to relevant Mathlib computability notions. Separate audit modules can report assumptions for local theorems and imported Mathlib facts independently.

The Rocq side combines repository-authored models/bridges with selected certificates vendored under `lib/Coq-BB5`. The vendored README records the upstream commit and local kernel-hardening changes. Repository-authored work remains under `Computability/BusyBeaver`, preserving provenance.

Reading route

Read `Core.lean` before any certificate. Then inspect one smallest classification path end to end: machine normalization, one concrete trace, certificate checker, and final bound. Only after that should the generated BB3/BB4 shard tree be read; otherwise the data volume hides the machine semantics that gives it meaning.

8.3. CombinatoryLogic

8.3.1. Semantic families

The project independently formalizes several calculi:

- a scoped untyped lambda calculus with capture-avoiding substitution;
- pure SK terms and reduction;

- SKI terms and reduction;
- pure Iota terms;
- arithmetic and coding inside SKI;
- external computability interfaces for partial recursive functions.

The Lean module list makes the intended proof graph visible: `Lambda`, `StrongLambda`, `Reduction`, `SK`, `SKI`, `Iota`, compilation modules, coding/evaluation modules, confluence modules, partial-recursive bridges, `Universality`, and `Audit`.

8.3.2. Lambda to SK: bracket abstraction

`LambdaToSK.lean` implements bracket abstraction. A free variable is eliminated structurally:

- the variable itself maps to I ;
- a term not containing the variable maps to Kt ;
- an application maps to S applied to the abstractions of its two children.

The key theorem is semantic, not just syntactic: applying the compiled SK term to an argument simulates substitution in the original lambda term. The compiler for closed lambda terms follows by iterating the abstraction step.

The scoped representation is important. It allows the substitution theorem to state exactly which variables are available at each term and prevents capture by construction or by proved renaming discipline.

8.3.3. SK, SKI, and Iota

The standard one-combinator Iota encoding is recorded explicitly. In one common notation the derived combinators are:

$$I = u, \quad K = \iota(\iota(u)), \quad S = \iota(\iota(\iota(u))).$$

The project proves positive simulations among pure Iota, SK, and SKI rather than identifying their syntax. It also proves useful injectivity properties of compilers, ensuring that a target encoding does not collapse distinct source terms accidentally.

`IotaToLambda.lean` gives exact one-, two-, and three-step beta head traces for the derived combinators, then lifts them through contexts and reflexive/transitive closures. This is stronger and more reusable than a single theorem saying “Iota is universal”: downstream proofs can account for operational cost and reduction context.

8.3.4. Confluence and evaluation

The SKI subtree distinguishes raw one-step reduction, parallel or Church-style reduction, confluence, and executable evaluation. Confluence justifies treating normal forms as observationally stable where they exist; an evaluator theorem connects an implementation to the inductive reduction relation.

Coding modules assign natural numbers to syntax and prove primitive-recursive properties of constructors, decoders, and reduction predicates. Arithmetic modules construct numerals and operations inside the calculus. These are the ingredients needed to connect a high-level universality proof to a standard partial-recursive/Turing-machine notion.

8.3.5. Partial-recursive compilation

`PartrecToSKI.lean` and `ExactPartrecToSKI.lean` construct SKI terms representing partial recursive functions. The exact version tracks the observation relation carefully enough to state successful and divergent behavior rather than only an extensional existence theorem.

The Rocq side also connects combinatory logic to the vendored Library of Undecidability. It uses that library's own lambda semantics and machine interfaces. The project README explicitly refuses to compose this theorem with the local Iota embedding without an operational bridge.

Gotcha

Do not infer a commuting square merely because all four corners are called “lambda,” “SKI,” or “Turing complete.” A compiler theorem preserves a specific reduction or observation relation. Different libraries can use weak-head, call-by-value, full beta, fuelled evaluation, or relational semantics. The missing bridge can be the hardest theorem in the square.

8.3.6. What is and is not claimed

The compilers are not claimed to be mutual inverses, reduction-reflecting in every direction, or fully abstract for an observational equivalence. Positive simulation and universality are sufficient for the public endpoint. Stating these nonclaims prevents a user from treating an encoding theorem as an isomorphism of calculi.

8.4. TuringDegrees

8.4.1. Why Lean and Rocq differ

The Lean project builds on Mathlib's oracle-computability infrastructure and uses a quotient of set representatives by mutual Turing reducibility. The Rocq project uses predicates plus a setoid/equivalence presentation because its imported Synthetic Computability stack exposes different abstractions.

Both are legitimate degree constructions, but theorem transfer requires an explicit representation bridge. The repository therefore maintains a coverage ledger instead of forcing line-by-line syntactic parity.

8.4.2. Lean architecture

The Lean development includes:

- oracle reducibility and equivalence over sets/predicates;
- the quotient type of Turing degrees;
- order and lattice-like operations;
- zero degree and its characterization by computability;
- an exact even/odd join construction;
- complement invariance;
- the halting degree $0'$ and c.e.-completeness;
- structural/cardinality results, including continuum many degrees and countable lower cones or equivalence classes where stated.

An “exact join” theorem matters more than a mere upper bound: it proves that the even/odd tagged union is the least upper bound under reducibility, allowing the quotient order to inherit familiar degree operations.

The quotient presentation makes equality pleasant for consumers but can hide representative-level computation. Internal lemmas therefore descend and lift properties carefully through `Quot.sound`.

8.4.3. Rocq architecture

The Rocq development is deeper in several classical recursion-theory directions. According to its README and coverage ledger, it includes components around:

- the Turing jump and jump monotonicity;
- Kleene–Post constructions of incomparable degrees;
- a limit lemma and hierarchy machinery;
- versions of Post’s theorem;
- a conditional or assumption-parameterized treatment of Post’s problem;
- compatibility wrappers over Synthetic Computability.

Several hypotheses—Markov principles, enumeration principles, or effective presentation assumptions—are visible in theorem statements rather than folded into a global classical axiom. That makes the Rocq side mathematically richer in some directions but not a drop-in replacement for Lean’s quotient API.

8.4.4. The coverage ledger

[Computability/TuringDegrees/COVERAGE.md](#) maps informal claims to specific Lean and Rocq declarations and lists their hypotheses. It also records deliberate omissions and corrects source-level shorthand.

One example is a recursive approximation: a correct formal definition needs both the input and stage arguments, even when prose writes a one-argument limit notation. Another is pair coding: exact pairing/unpairing statements are preferable to an informal “computable coding exists” phrase because later reductions consume their equations.

The ledger’s discipline is conservative. A mathematically related theorem elsewhere in the repository does not count as a port until its statement and dependency boundary have been compared.

8.4.5. Build boundary

TuringDegrees has both a Lean audit and a Rocq setup script that builds the pinned Synthetic Computability dependency with a repository-maintained compatibility patch. The patch lives beside the wrapper project, not inside the submodule. This preserves the submodule as an auditable upstream snapshot while making the local toolchain reproducible.

8.5. Comparative map

Table 8.1.: The three computability projects compared.

Project	Primary objects	Main proof mechanism	Main boundary
BusyBeaver	Finite transition tables and traces	Executable replay and finite classification	Generator output, evaluator choice, and opt-in certificate targets
CombinatoryLogic	Terms, reductions, compilers, observations	Induction, simulation, confluence, coding	Operational semantics of each source/target calculus
TuringDegrees	Oracles, reductions, equivalence classes/degrees	Abstract reducibility, quotient/setoid reasoning, priority-style constructions	Cross-prover representation and explicit logical principles

Primary files

Computability/README.md; Computability/BusyBeaver/README.md; Computability/BusyBeaver/Lean/BusyBeaver.lean; Computability/BusyBeaver/Lean/BusyBeaver/Core.lean; Computability/CombinatoryLogic/README.md; Computability/CombinatoryLogic/Lean/CombinatoryLogic; Computability/TuringDegrees/README.md; Computability/TuringDegrees/COVERAGE.md.

9. Logic

9.1. Overview

Logic is the broadest top-level domain. It ranges from tiny constructive propositional lemmas to first-order completeness, modal-system libraries, arithmetized syntax, interpretability, bounded consistency, and verified decision procedures.

A useful internal taxonomy is shown in Table 9.1.

Table 9.1.: Logic project families.

Family	Representative projects	Architectural emphasis
Propositional/equational microprojects	ShefferStroke, NaturalDeduction, MonotonicityOfEntailment, PrincipleOfExplosion, EquationalLogic, BooleanAlgebra, FiniteMatrixNon-characterizability	Small syntax/semantics developments, often paired and constructive.
First-order metatheory	FirstOrder, ArbitraryLanguage-Compactness	From-scratch fixed-language calculus versus a Mathlib-backed generic facade.
Source ports and modal logic	FoundationPort, Modal	Pinned source inventory, independent Rocq modules, and conservative coverage ledgers.
Interpretability	PAHF, ZFCinPA	Translations among arithmetic/set theories, model-relative coding, and explicit conditional endpoints.
Peano arithmetic	ListCoding, BoundedConsistency, NoFiniteModel, NotFinitelyAxiomatizable/finite-basis reduction, Undecidable	Arithmetized syntax, representability, partial truth, and proof predicates.
Decision procedures	PresburgerArithmetic	Syntax normalization, Cooper elimination, executable decision, and correctness.
Binder semantics	QuantifierCommutation	Constructive positive laws and finite countermodels for plausible false laws.

9.2. Propositional and equational microprojects

These projects are compact laboratories for repository conventions. Most have a Lean facade, one or more definition/proof modules, a Rocq counterpart, and an audit.

9.2.1. Sheffer stroke and Boolean algebra

The Sheffer-stroke project develops a functionally complete single connective and translates ordinary Boolean operations into it. The Boolean-algebra project supplies algebraic semantics and equational reasoning. Together with **EquationalLogic**, they separate syntactic derivability from semantic validity and give small soundness/-completeness examples.

9.2.2. Natural deduction and entailment

NaturalDeduction defines proof objects or derivability rules for a propositional language and proves the expected structural/metatheoretic properties. **MonotonicityOfEntailment** isolates the weakening fact that adding hypotheses preserves entailment. **PrincipleOfExplosion** isolates derivability from contradiction.

The existence of separate microprojects for seemingly elementary facts is useful: they avoid importing a large logic framework when testing a particular encoding or cross-prover statement.

9.2.3. Finite matrix noncharacterizability

FiniteMatrixNoncharacterizability is a more substantial propositional result. Its presence beside the microprojects shows that the top-level taxonomy is by subject, not by scale. Readers should use its facade and audit to find the actual public theorem rather than infer the argument from the directory name alone.

9.3. QuantifierCommutation

This dependency-free project proves constructively that adjacent universal quantifiers commute and adjacent existential quantifiers commute:

$$\begin{aligned}(\forall x \forall y Rxy) &\leftrightarrow (\forall y \forall x Rxy), \\ (\exists x \exists y Rxy) &\leftrightarrow (\exists y \exists x Rxy).\end{aligned}$$

It then gives finite counterexamples for two superficially similar binders: nested “there does not exist” and nested unique existence.

The first counterexample relies on parsing. The nested expression

$$\nexists x (\nexists y Rxy)$$

is not the same as one outer negation of a two-variable existential block. A two-element relation can have a witness in every row but an empty column, separating the two binder orders constructively.

For unique existence, a three-element relation is chosen with exactly one row containing exactly one related element, while every column contains exactly one. Thus the outer uniqueness count changes when binders are swapped.

The Lean core defines unique existence explicitly because dependency-free Lean does not supply the notation in that environment. Rocq uses its standard `exists!`; the README verifies that the semantic contract matches. No classical axiom is needed.

This project is an excellent template for a small paired development: statement, countermodel, audit, focused build, and `coqchk` all fit in one README.

9.4. FirstOrder

9.4.1. A deliberately fixed language

The repository-local first-order project uses a countable language with equality and one distinguished binary relation, `fMem`. That relation can later serve as set membership, but the completeness proof itself treats it abstractly.

Fixing the language keeps the development dependency-light and makes every encoding explicit. It is not intended to replace Mathlib’s generic first-order logic. Instead, the root also exposes **ArbitraryLanguageCompactness**, a separate audited facade over Mathlib’s arbitrary-signature theorem.

9.4.2. Module ladder

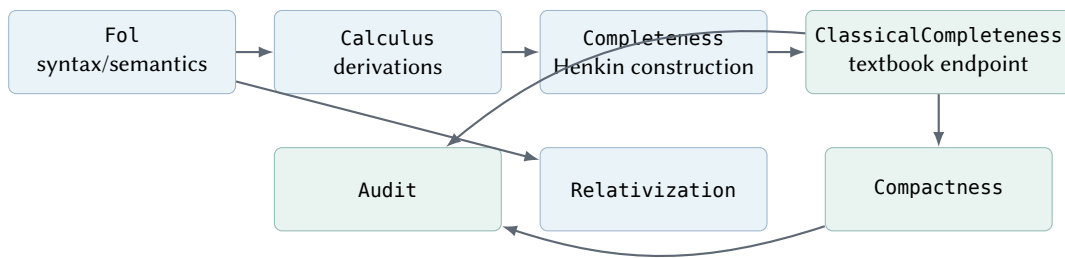


Figure 9.1.: The first-order dependency ladder.

Fol. Defines terms, formulas, free-variable operations, structures, assignments, term evaluation, and formula satisfaction. Basic substitution and assignment lemmas establish the semantic infrastructure consumed by soundness and the truth lemma.

Calculus. Defines the proof system and derivability from theories. Structural rules, substitution, equality behavior, and soundness are proved before any model construction begins.

Completeness. Builds a maximal consistent Henkin extension, constructs the term model, quotients terms by provable equality, proves the truth lemma, and obtains a model of a consistent theory.

ClassicalCompleteness. Packages the construction as the familiar semantic-entailment-implies-derivability theorem and related consistency/model-existence equivalences.

Compactness. Derives compactness from completeness by finite inconsistency/derivation support.

Relativization. Provides the formulas and semantic lemmas needed to restrict quantifiers to a definable domain, a key dependency for internal model constructions in set theory and interpretability.

9.4.3. The Henkin path in more detail

The completeness proof follows the standard mathematics but must expose several hidden paper steps:

1. enumerate formulas and add witnesses while preserving consistency;
2. extend the resulting Henkin theory to a complete/maximal consistent theory;
3. define a congruence on closed terms by provable equality;
4. show function/relation interpretation is well defined on equivalence classes;
5. prove the truth lemma by induction on formulas;
6. derive satisfaction of the original theory;
7. turn semantic entailment into a contradiction if the target formula were not derivable.

The quotient step introduces `Quot.sound` and classical selection into the usual Lean assumption footprint. The audit makes this visible rather than describing the project simply as “classical.”

9.4.4. Two compactness projects, two provenances

FirstOrder.Compactness is a corollary of the repository’s own fixed-language completeness proof. **Arbitrary-LanguageCompactness** re-exports or specializes Mathlib’s general result and audits its assumptions. The root facade imports both.

This is not duplication: one is an independently inspectable metatheory development used by repository-local ZF and coding projects; the other gives consumers a generic language interface without rebuilding Mathlib.

9.5. FoundationPort and Modal

9.5.1. Pinned source, independent target

The submodule `lib/FormalizedFormalLogic-Foundation` is a pinned, read-only Lean source reference. `Logic/FoundationPort` tracks an independent Rocq port. “Complete” is defined conservatively: every source module must be accounted for, not merely every theorem that happened to be needed by a local endpoint.

`ported.tsv` stores reviewed mappings. The script `Logic/FoundationPort/coverage.sh` reads the pinned tree, validates each mapping, and emits one row for each of the 455 Lean modules. Status is one of:

ported

The theorem surface has an independently checked Rocq representation, possibly more general or idiomatic.

partial

A meaningful slice exists, but declarations or a dependency layer remain.

unported

No module-level parity claim has been reviewed.

The generated full report is deliberately not committed; the reviewed mapping and pinned gitlink are the compact sources of truth.

9.5.2. Declaration-free facades still count

Foundation contains modules whose entire role is importing a family of modules. The port ledger records exact or partial declaration-free Rocq facades for those source modules. This avoids a common coverage bug: counting only declarations misses import surfaces that are part of a library’s public API.

9.5.3. The Modal port

`Logic/Modal` is a large independent Rocq development corresponding to high-value parts of Foundation’s modal tree. Its scope includes:

- generic Kripke frames, models, generated submodels, and p-morphisms;
- modal syntax, substitutions, normal forms, and semantics;
- Hilbert systems and canonical-model completeness;
- systems such as K, T, D, B, K4, K5, S4, S5, GL, Grz, and extensions;
- filtration and finite-model arguments;
- frame correspondences and definability;
- algebraic semantics;
- boxdot and translation constructions;
- hierarchy and incompleteness-related components.

The README/module table names partial boundaries instead of claiming blanket parity. Certain systems or specialized source modules remain explicitly outside complete coverage. The port is not imported into the Lean source and does not alter the Foundation submodule.

Orientation

FoundationPort treats coverage metadata as a theorem-library artifact. A build answers whether existing Rocq files check; the ledger answers whether those files cover the pinned Lean source. Both are necessary.

9.6. Interpretability

9.6.1. PA/HF

PAHF is a compact paired Lean/Rocq development relating Peano arithmetic and hereditarily finite set theory. It supplies translations and model constructions showing how arithmetic data can be represented in finite set-theoretic structure and conversely.

Its topic-level documentation is lighter than the neighboring ZFC project, so the facade and audit are the best guides. This asymmetry is real: not every project in the monorepo has been normalized to one README template.

9.6.2. ZFCinPA: target and status

The ambitious target is a PA theorem asserting, internally and uniformly, that for every complexity bound n , ZFC has no proof of contradiction whose formulas stay within that bound. Informally:

$$\text{PA} \vdash \forall n \text{Prov}_{\text{ZFC}}(\ulcorner \text{Con}_n(\text{ZFC}) \urcorner).$$

At the pinned snapshot the development does *not* present this as an unconditional theorem. Its status can be summarized as follows.

Table 9.2.: ZFCinPA status at the pinned commit.

Layer	Status
For each external standard numeral n , ZFC proves the corresponding bounded-consistency sentence	kernel-checked in BoundedZFC-Consistency .
The repository-local Foundation/ZFC layer proves the required instance	kernel-checked .
The uniform endpoint is reduced to an eight-field internal certificate	kernel-checked equivalence/reduction.
Successor implications at standard indices	kernel-checked .
Successor implications for arbitrary, possibly non-standard model elements	open premise.
Final PA uniform theorem	conditional ON ZFCSuccessorImplicationsInAllModels.

9.6.3. Five architectural layers

The README divides the project into a useful ladder.

1. **Definability:** encode syntax, bounded satisfaction, proof predicates, and the relevant functions in arithmetic.
2. **Statement:** define the exact PA sentence and model-relative semantic version.
3. **Eight-field certificate:** package the local implications needed to step the construction through all indices.
4. **Compiler/translation layer:** turn source derivations and semantic facts into the coded target claims.

5. **Successor/evaluation bridge:** prove the model-internal successor step that must work even at nonstandard indices.

An earlier five-field certificate was proved nontransferable to the desired models. The replacement adds two Tarski-style truth fields and a semantic precheck, making the missing obligation explicit. Recording a failed abstraction this precisely is valuable: future work is directed at the actual nonstandard-model bridge, not at polishing an insufficient package.

9.6.4. Specification hardening

Statement-bearing modules generally use

```
set_option autoImplicit false
```

so an out-of-scope identifier cannot silently become an additional theorem parameter. A small number of legacy exceptions are documented. Companion audit modules inspect the endpoint of each layer.

This policy arose from a real specification risk, not stylistic preference. Metamathematical theorems with many codes, models, and translations are especially vulnerable to a theorem that is easy only because an intended constant became a free parameter.

9.6.5. Lake target discovery gotcha

The root ZFCinPA library has an explicit facade and no automatic glob that turns every source file into a named Lake target. Therefore a command such as

```
lake build +ZFCinPA.Numerals0mega
```

can be a silent no-op if that module is not reachable from a registered target. The reliable alternatives are to import it from the facade, add a target, or elaborate directly:

```
lake env lean Logic/Interpretability/ZFCinPA/Lean/ZFCinPA/Numerals0mega.lean
```

This is a general monorepo lesson: a successful command that schedules zero jobs proves nothing. Inspect the target graph or build the source directly.

9.7. Peano arithmetic

9.7.1. ListCoding

PAListCoding independently develops Lean and Rocq formulas representing finite lists and related arithmetic predicates. The audited surface contains thirty formulas: twenty-five standard list/number predicates and five formulas used for hereditary Cantor-normal-form notation below ε_0 .

Representative capabilities include:

- validity of a list code, length, and element lookup;
- singleton, append/concatenation, and flattening;
- occurrence counting, permutation, and subsequence relations;
- sum, product, extrema, and simple statistics;
- primality, powers, factorization, digits, and divisors;
- Diophantine graphs for exponentiation, tetration, and higher hyperoperators;
- hereditary notation operations used in ordinal-analysis examples.

The public contract is representation and functionality, not equality of numeric codes across proof assistants. Lean uses an Ackermann/graph-of- function approach integrated with Foundation infrastructure; Rocq uses a nested polynomial code convenient for direct arithmetic proofs.

Two conventions are easy to miss: list positions are zero-based, while the “ n th prime” interface is one-based. The README and audit make those choices explicit.

9.7.2. Bounded PA consistency

BoundedConsistency formalizes a scheme saying, for each external complexity bound, that no PA proof of contradiction exists if *every formula occurrence in the proof* stays within that bound.

Four distinctions govern the whole project:

1. **External scheme versus internal universal sentence.** Proving a theorem for each meta-level numeral n is weaker than one PA derivation of $\forall n \text{Con}_n$.
2. **All occurrences versus conclusion only.** Intermediate formulas in a proof tree must satisfy the bound.
3. **Standard codes versus nonstandard model elements.** A decoder theorem for quoted standard syntax may not control an internally represented nonstandard proof code.
4. **Formula complexity versus proof length.** The bound is on formula rank/complexity, not the number of inference steps.

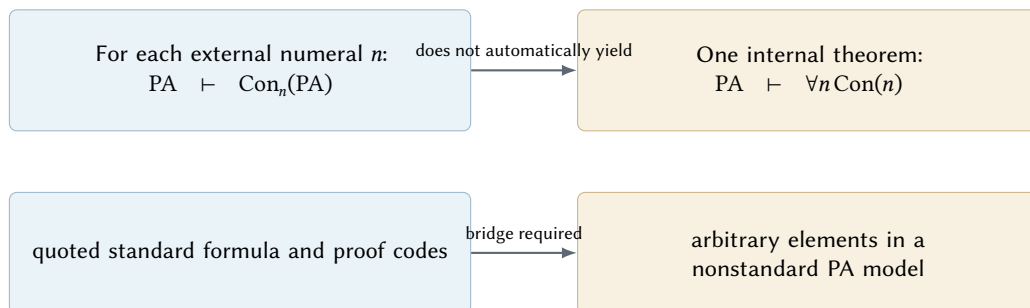


Figure 9.2.: Two independent uniformity gaps in bounded consistency.

The implementation defines coded formulas, complexity, a seventeen-rule proof tree, a verifier that checks every node, partial truth predicates, and soundness of bounded derivations. The Lean and Rocq numeralwise schemes are complete. The strongest uniform sentence is unconditional in Lean at this snapshot, while the Rocq counterpart remains conditional on explicit compiler premises; the README states that asymmetry rather than claiming parity.

9.7.3. NoFiniteModel

NoFiniteModel proves that a structure satisfying the chosen PA axiom system cannot be finite. The usual mathematical core is the injective but non-surjective successor function. Formalization requires aligning the object-language axioms with the semantic structure and extracting the successor properties under the exact axiom encoding.

9.7.4. Finite-basis and undecidability projects

The finite-basis reduction formalizes a route from finite axiomatizability assumptions to the relevant contradiction/nonaxiomatizability theorem. The undecidability project connects PA derivability or theoremhood to a standard computability obstruction. Their inclusion as root default targets means the workspace treats their audits as integration-critical even though the root facade does not expose every internal module.

For both, read the statement module before the reduction machinery: “undecidable” can refer to provability, validity, consistency, or a coded decision problem, and “not finitely axiomatizable” depends on whether equivalence is semantic or deductive and over which class of models.

9.8. PresburgerArithmetic

9.8.1. Lean end-to-end decision procedure

The Lean development implements Cooper elimination with the following module flow:

Syntax → NormalForm → Cooper → Elimination → Decision → Audit.

Syntax defines the arithmetic language and semantics. Normalization moves formulas into the restricted linear form required by elimination. Cooper handles one quantified variable by collecting coefficients, least common multiples, divisibility atoms, and finite residue cases. Elimination iterates the step. Decision evaluates the resulting quantifier-free formula.

Correctness is proved at each boundary, so the final Boolean procedure is sound and complete for the input sentence. No external solver or oracle is called.

9.8.2. Rocq scope

The Rocq side independently verifies the decisive normalized one-variable Cooper step and its supporting arithmetic. Its theorem surface is not identical to the Lean end-to-end executable decision. This is documented as a coverage boundary, not concealed by identical project names.

9.9. How to navigate Logic safely

Reading route

1. For a microproject, read facade → definitions → audit.
2. For metatheory, draw the syntax–semantics–derivation–model ladder before opening proofs.
3. For arithmetization, write every quantifier at the meta/object/model level and label standard versus nonstandard variables.
4. For a source port, consult the coverage ledger before the module tree.
5. For a decision procedure, identify the executable function, the semantic specification, and the theorem connecting them.
6. For interpretability, locate the translation identity and the model-relative satisfaction theorem; theorem-name similarity is not a bridge.

Primary files

Logic/README.md; Logic/QuantifierCommutation/README.md; Logic/FirstOrder/README.md; Logic/FoundationPort/README.md; Logic/Modal; Logic/Interpretability/ZFCinPA/README.md; Logic/PeanoArithmetic/ListCoding/README.md; Logic/PeanoArithmetic/BoundedConsistency/README.md; Logic/PresburgerArithmetic/README.md.

10. Number theory

10.1. Overview

The number-theory subtree is modest in byte size but deliberately varied in proof provenance. Its projects are:

- a formal exponent-four case of Fermat’s Last Theorem;
- integer-sum identities;
- Calkin–Wilf enumeration of positive rationals;
- an arithmetic sentence expressing the Riemann hypothesis.

The key architectural lesson is that matching theorem names on the Lean and Rocq sides do not imply matching proofs.

10.2. DiophantineEquations and Fermat exponent four

The Lean facade imports a very small `FermatFour.lean`. Its role is to specialize and expose the relevant Mathlib theorem under the repository’s chosen statement. The formal evidence is kernel-checked, but the deep descent argument belongs to the imported library.

The Rocq side contains an independent descent development. A code reader should therefore describe the two sides as:

- **Lean**: local API and specialization over a pinned Mathlib theorem;
- **Rocq**: repository-local proof of the exponent-four descent.

This distinction matters when assessing maintainability. The Lean wrapper is small but tied to upstream theorem naming and statement shape. The Rocq proof is larger but locally inspectable and can expose intermediate number-theoretic lemmas.

10.3. IntegerSums

The integer-sums project proves exact finite-sum identities in both proof assistants. It is a useful compact example of algebraic normalization, induction over a finite range, and explicit handling of integer/natural casts.

These modules often look trivial informally but are valuable regression tests for coercion and finite-sum conventions used in larger projects.

10.4. RationalEnumeration

The Calkin–Wilf development formalizes the binary-tree/orbit enumeration of positive rational numbers. Its architecture separates:

- the successor map or left/right tree actions;
- positivity and reduced-fraction invariants;
- existence: every positive rational is reached;
- uniqueness: no positive rational is reached twice;
- the resulting enumeration/bijection statement.

Euclidean descent is the conceptual inverse: compare numerator and denominator, subtract the smaller from the larger, and record the branch. Formalization requires a termination measure and exact reconstruction theorem.

Lean and Rocq use their native rational/integer libraries, so the internal normal-form proofs differ while the public enumeration contract is parallel.

10.5. RiemannHypothesis

This project does not attempt analytic number theory inside PA. It constructs a first-order arithmetic sentence representing the Riemann hypothesis under the selected coding and proves the expected statement-level relationships.

The main review questions are therefore syntactic:

- Which external RH formulation is arithmetized?
- Which coding of finite sequences, rationals, or approximations is used?
- Is the theorem an equivalence in the standard model, a derivability statement, or merely a definition of the sentence?
- Which imported representability results connect the code to analysis?

The project belongs in number theory by intended statement and in logic by implementation technique. Its current location keeps the user-facing mathematical taxonomy primary.

Primary files

[NumberTheory/README.md](#); [NumberTheory/DiophantineEquations/README.md](#); [NumberTheory/DiophantineEquations](#); [NumberTheory/IntegerSums](#); [NumberTheory/RationalEnumeration](#); [NumberTheory/RiemannHypothesis](#).

11. Set theory

11.1. Overview

The set-theory subtree provides a lightweight internal ZF base, an alternative closure-schema axiomatization, and a bounded-complexity consistency theorem for ZFC. It also supplies dependencies consumed by the logic/interpretability projects.

These developments use the repository-local fixed first-order language rather than importing a large set-theory library. This gives tight control over formula codes, quantifier rank, relativization, and internal models—exactly the features needed for bounded consistency.

11.2. ZF

11.2.1. Public surface and internal scale

The Lean facade is tiny: it imports `ZF.Zf`. The implementation module is substantial and is paired with an audit. It defines the set-theoretic formulas and model infrastructure needed downstream, including:

- the ZF/ZFC axiom formulas in the one-relation language;
- models and satisfaction of those axioms;
- internal empty set, pairing, union, separation, power set, infinity, and replacement constructions as available under hypotheses;
- set algebra and extensionality;
- Kuratowski ordered pairs;
- a natural-number object and finite recursion;
- formulas and lemmas used by bounded-satisfaction developments.

The goal is not to reproduce a full general-purpose set-theory ecosystem. It is a compact formal base with explicit syntax and semantics. Downstream code can quote its formulas and reason about their complexity without crossing a translation from another language framework.

11.2.2. Why one substantial module can be reasonable

A single large `Zf.lean` keeps mutually recursive internal definitions and their notation close, but it raises navigation cost. The facade/audit pair and section headings inside the file therefore matter. When debugging a downstream theorem, search for the declaration named in the import error or audit rather than reading the module linearly.

The Rocq side mirrors the mathematical base with its own first-order and set representations. The pair is a contract-level parallel development, not a shared extracted kernel object.

11.3. ClosureAxiomatization

11.3.1. Alternative axiom

The project studies a theory retaining Extensionality, Regularity, Separation, and Powerset while replacing Pairing, Union, Infinity, and Replacement with a single closure schema for set-like relations. It proves equivalence with ZF.

The module organization separates two levels of equivalence.

Shallow semantic directions

Forward and Reverse construct the missing axioms from the closure schema and the closure schema from ZF inside models.

Deductive equivalence

Equivalence combines the local first-order calculus, translations, and model theory to state the theory-level equivalence in the intended formal sense.

This distinction is important. Showing that every model of one axiom list satisfies each axiom of the other is the mathematical core, but a reusable theory equivalence may also need syntax-level consequence and translation lemmas.

11.3.2. Build and exposition

The Lean project is Mathlib-free, depending on the repository's own first-order base. A paired Rocq development independently checks the directions. The directory also includes an expository article in $\text{\LaTeX}$ /PDF, useful for matching theorem modules to the mathematical narrative.

11.4. Bounded ZFC consistency

11.4.1. The exact theorem

For each external natural number n , the project proves in ZFC the consistency of ZFC proofs all of whose formula occurrences have quantifier rank at most n . Schematically:

$$\text{for every meta-level } n, \quad \text{ZFC} \vdash \text{Con}_n(\text{ZFC}).$$

This is a *numeralwise scheme*. It is not one object-language theorem $\text{ZFC} \vdash \forall n \text{Con}(n)$. The distinction protects the result from conflict with Gödel's second incompleteness theorem.

The rank used here counts every quantifier in the repository's language. It is finer than a Levy-hierarchy measure with primitive bounded quantifiers, because bounded quantification must itself be encoded.

11.4.2. Proof route

The completed proof uses externally indexed partial satisfaction over the repository's own first-order/ZF stack. At a high level:

1. define formula rank and the all-occurrences proof restriction;
2. for a fixed external n , define satisfaction for formulas up to the required rank;
3. prove Tarski-style recursion equations for that finite rank;
4. prove soundness of every inference rule for restricted proof trees;
5. show the contradiction formula is false;
6. arithmetize/quote the resulting argument as the ZFC derivation of the bounded-consistency sentence.

The external index permits the meta-theory to build one finite truth definition for each n . No uniform truth predicate for all formulas is claimed.

11.4.3. All-occurrences restriction

The proof predicate checks every formula in the derivation, not merely the conclusion. This is stronger and more stable under the soundness induction: each inference rule receives truth for exactly the formulas it mentions. A conclusion-only restriction could admit arbitrarily complex intermediate lemmas and would not support the same partial-satisfaction proof.

11.4.4. Discarded routes documented

The README records two important corrections:

- an earlier empty-context sentence did not express the intended theorem in the right theory-relative way;
- a reflection-style route was abandoned in favor of direct externally indexed partial satisfaction.

Documenting wrong turns is particularly useful in metamathematics, where a stronger-looking statement can be either false, inconsistent with known theorems, or merely different because a quantifier moved from meta-language to object language.

11.4.5. Relation to ZFCinPA

BoundedZFConsistency supplies the complete numeralwise theorem. **ZFCinPA**, located under Logic/Interpretability, tries to internalize and uniformly prove the corresponding family in PA. The latter must handle nonstandard model elements and therefore has a genuinely stronger bridge obligation. The two projects should never be described as “the same theorem in different theories.”

11.5. Set-theory dependency flow

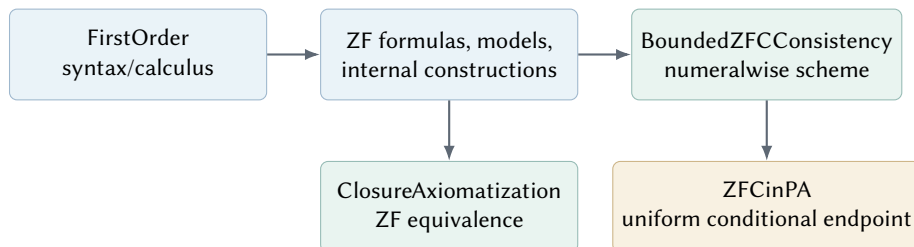


Figure 11.1.: The repository-local set-theory stack and its main consumers.

11.6. Recommended reading order

Reading route

1. Read the FirstOrder language and relativization overview first.
2. Read the ZF facade and audit, then search `Zf.lean` by public declaration rather than linearly.
3. For bounded consistency, write the exact scheme and all-occurrences predicate on paper before following the partial truth recursion.
4. Read ClosureAxiomatization’s shallow semantic directions before its deductive-equivalence wrapper.
5. Only then compare BoundedZFConsistency with ZFCinPA; the standard/nonstandard index gap will be visible.

Primary files

SetTheory/README.md; SetTheory/ZF/README.md; SetTheory/ZF/Lean/ZF; SetTheory/ClosureAxiomatization/README.md; SetTheory/BoundedConsistency/README.md; Logic/Interpretability/ZFCinPA/README.md.

12. Ancillary directories, vendored code, and tooling

12.1. Why these directories matter

A walkthrough that covers only theorem modules misses several reproducibility boundaries. ProveIt has explicit homes for vendored code, proof-assistant compatibility work, auxiliary sequence material, generated articles, and redirect-only projects. Their location communicates ownership and trust.

12.2. `lib`: the only vendored-code home

[lib/README.md](#) is the provenance index for third-party proof code. The directory contains both git submodules and selected copied snapshots.

The policy is simple:

- unmodified or locally hardened upstream source lives under `lib`;
- repository-authored mathematical wrappers live under the owning subject project;
- compatibility patches that must remain reviewable live outside the submodule or are documented as local changes;
- licenses and upstream commits are recorded near the vendored tree.

This avoids two common monorepo failures: silently editing a dependency inside a subject directory, and placing original project code in a subtree users expect to be replaceable by a fresh vendor import.

12.2.1. Coq Library of Undecidability

The pinned/forked Undecidability library supplies standard machine, lambda-calculus, and computability infrastructure. `CombinatoryLogic` uses it for external computability equivalences, but keeps local operational semantics separate.

12.2.2. FormalizedFormalLogic/Foundation

Foundation is a read-only Lean source dependency and the authoritative module inventory for `FoundationPort`. The root Lake workspace registers it as a library; the Rocq port does not modify it.

12.2.3. MathComp-Abel

MathComp-Abel provides the Galois-theory machinery used by Rocq `PolynomialFormulas`. Root Rocq setup installs its opam dependencies and maps the theory root to logical namespace `Abel`.

12.2.4. Synthetic Computability

The Turing-degrees Rocq development wraps a pinned Synthetic Computability source. A project-local build script applies or stages compatibility work needed by the current Rocq environment without turning the submodule into an opaque mutable dependency.

12.2.5. Coq-BB5

Selected Busy Beaver certificates are copied with upstream provenance and kernel-hardening notes. Their status is therefore different from a git submodule but still explicitly third-party. Repository-authored semantics and bridges remain outside `lib`.

12.3. Tools/Rocq92Compat

This directory is infrastructure, not a theorem project. Its README documents a reproducible compatibility overlay for a Windows/Rocq 9.2 environment where package splits and upstream build assumptions do not line up cleanly.

The overlay includes several kinds of adaptation:

- empty old-package-name adapters where package names changed;
- findlib aliases;
- pinned Elpi and MathComp repairs;
- proof-script ordering patches for Coquelicot and Interval;
- Windows launchers and command shims;
- MinGW/remake compatibility fixes.

A compatibility overlay carries its own trust and maintenance concerns. It can change which library source is compiled or how commands are resolved, but it is not a mathematical axiom. The important review properties are: patches are version-pinned, diffable, reproducible, and kept separate from theorem files.

Gotcha

Do not copy a compatibility-patched dependency tree back into `lib` and commit it as though it were the upstream snapshot. Keep the immutable source, patch set, and generated build tree distinguishable; otherwise provenance and regeneration become impossible to audit.

12.4. Oeis

[Oeis](#) contains auxiliary or legacy sequence material, notably an A290268 surface related to the combinatorics derivative-expansion project. New sequence-specific proof code generally lives in the relevant mathematical domain, while the OEIS directory provides sequence-oriented artifacts or compatibility paths.

The A198683 development demonstrates a stronger sequence-status convention: only values backed by the current accepted certificate surface are exposed as accepted data, even if exploratory and partial proof modules for later values already exist.

12.5. Shenanigans

The top-level [Shenanigans](#) directory is a redirect to a separate repository. It has no active proof architecture to include in the root build. Treating the redirect as documentation avoids false expectations that missing Lean/Rocq modules were omitted from this walkthrough.

12.6. Articles and reports

Several project directories contain $\text{\LaTeX}$ sources and compiled PDFs, for example the Lazard formalization report and the closure-axiomatization article. These documents are not theorem dependencies unless a build file explicitly says otherwise. They serve as:

- mathematical narrative around a module graph;
- claim-to-module crosswalks;
- status snapshots for long-running proof programs;
- source/provenance records for finite data.

A regenerated report can therefore be a useful project milestone without changing a kernel theorem. Conversely, a green article build does not imply a green Lean/Rocq build.

12.7. Repository hygiene files

`.gitattributes` normalizes text to LF while preserving binary treatment for PDFs, images, and archives. This is particularly important for generated source: newline churn can turn a deterministic regeneration into a repository-wide diff.

`.editorconfig` and `.gitignore` establish ordinary editor and build-output conventions. The pinned commit has no repository-level `.github/workflows` tree, so the source does not expose a single GitHub-Actions validation matrix. Build reproducibility is consequently documented primarily in root and project READMEs.

12.8. Ownership map

Table 12.1.: Where an artifact should live.

Artifact	Preferred home	Reason
Hand-written theorem or bridge	Owning subject project	Keeps mathematical dependencies local.
Generated formal data/certificate	Owning subject project's Lean/Rocq tree	It is part of the checked import graph.
Generator/search/replay program	Project Support, Tools, or Research subtree	Separates candidate production from proof terms.
Unmodified upstream source	lib submodule or vendored snapshot	Makes provenance and replacement obvious.
Compatibility patch/wrapper	Owning project or Tools/Rocq92Compat	Avoids mutating the pinned source invisibly.
Expository article/crosswalk	Project root or documentation subtree	Keeps narrative adjacent without making it an import.
Coverage ledger	Port project root	Status is tied to the source inventory, not one target module.

Primary files

`lib/README.md`; `.gitmodules`; `Tools/Rocq92Compat/README.md`; `Oeis`; `Shenanigans`; `.gitattributes`.

Part III.

Building, debugging, and extending Provelt

13. Working with the repository

13.1. Start from a pinned tree

For reproducible work, record the commit before running a generator or changing a theorem. The repository contains submodules, generated modules, and fast-moving status documents; “current main” is not a durable input.

Listing 13.1: Pin and initialize a checkout.

```
git clone https://github.com/VladimirReshetnikov/ProveIt.git
Set-Location ProveIt
git checkout 675d0a2ff0fc624fba199e006049016ba806f40a
git submodule update --init --recursive
git status --short
git submodule status
```

For a narrow Lean-only project, initializing every submodule may be unnecessary. For root Rocq work or projects depending on Foundation, MathComp-Abel, Undecidability, or Synthetic Computability, initialize the specific listed submodules before interpreting missing imports as source errors.

13.2. Lean environment

13.2.1. Toolchain and cache

The root `lean-toolchain` selects Lean 4.32.0, and `lake-manifest.json` pins Mathlib and transitive dependencies. With Elan and Lake available:

```
lean --version
lake --version
lake exe cache get
```

The Mathlib cache removes a large amount of baseline compilation. It does not contain repository-local generated certificates, so a first broad build can still be substantial.

13.2.2. Broad validation

```
lake build
```

This requests the root default targets. It is useful after focused work, but is a poor first diagnostic because a failure may occur far from the changed module and multiple large jobs can compete for memory.

To validate only the consumer facade:

```
lake build ProveIt
```

To validate an explicitly registered library or audit, use the exact target spelling in `lakefile.toml` or the project’s README.

13.2.3. Direct module elaboration

For a source file that is not a registered target or not reachable from a facade, use:

```
lake env lean path/to/Module.lean
```

This is the most reliable antidote to a silent target no-op. It elaborates the literal file under the root package environment and returns an error if the file is absent or invalid.

A project-local Lake environment may be preferable:

```
lake --dir Logic/QuantifierCommutation/Lean build
lake --dir Logic/FirstOrder/Lean build
```

The local manifest can intentionally omit Mathlib and provide a much smaller dependency graph.

13.2.4. Threading and memory

Use the project README's worker recommendation. Typical patterns are:

```
# Serialize heavy elaboration jobs.
$env:LEAN_NUM_THREADS = "0"
lake build SomeHeavyCertificate

# Restore ordinary worker behavior for projects that need it.
Remove-Item Env:LEAN_NUM_THREADS -ErrorAction SilentlyContinue
```

Lean's `maxHeartbeats` limits elaboration work inside one declaration; Lake worker count limits concurrent processes. Raising the former does not solve system-wide memory pressure, and serializing jobs does not fix a single nonterminating reduction.

13.3. Focused Lean validation recipe

For an ordinary project:

1. elaborate the changed leaf file directly;
2. build its facade;
3. build its audit;
4. build the root library target if the project is registered there;
5. finish with `lake build ProveIt` or the root default set as appropriate.

For a generated certificate project:

1. regenerate into a clean temporary directory;
2. compare generated paths and hashes with the checked-in tree;
3. elaborate the smallest changed Data module;
4. elaborate its immediate Certificate;
5. walk up term/tail/shard parents;
6. build the semantic bridge;
7. build facade and audit.

This order localizes both generation and reduction failures.

13.4. Rocq environment

13.4.1. Version and package roots

The root documentation targets Rocq 9.2 or later for current builds, while some MetaRocq exact-equivalence work is tied to a 9.0-era environment. Verify the executable names supplied by the installation:

```
rocq --version
coqc --version
opam switch show
opam list
```

Do not assume `coqc`, `rocq c`, and `rocq compile` are interchangeable in scripts across installations. Preserve project commands unless deliberately modernizing them.

13.4.2. Root build

The root build requires its submodule dependencies and project-specific Synthetic Computability staging:

```
git submodule update --init `
  lib/Coq-Synthetic-Computability `
  lib/MathComp-Abel

opam install --yes --deps-only `
  ./lib/MathComp-Abel/coq-mathcomp-abel.opam

pwsh -NoProfile -File `
  Computability/TuringDegrees/Coq/BuildSyntheticComputability.ps1

rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq
```

On Windows, consult [Tools/Rocq92Compat/README.md](#) before patching dependencies ad hoc.

13.4.3. Focused sequential build

A dependency-light project often documents explicit commands:

```
coqc -Q Logic/QuantifierCommutation/Coq QuantifierCommutation `
  Logic/QuantifierCommutation/Coq/Commutation.v

coqc -Q Logic/QuantifierCommutation/Coq QuantifierCommutation `
  Logic/QuantifierCommutation/Coq/Counterexamples.v

coqc -Q Logic/QuantifierCommutation/Coq QuantifierCommutation `
  Logic/QuantifierCommutation/Coq/Audit.v

coqchk -silent `
  -Q Logic/QuantifierCommutation/Coq QuantifierCommutation `
  QuantifierCommutation.Commutation `
  QuantifierCommutation.Counterexamples `
  QuantifierCommutation.Audit
```

This style makes logical roots and module order explicit. For a large dependency graph, generate a project Makefile and request the endpoint `.vo` or `audit` target rather than compiling the root universe.

13.5. Audit workflow

13.5.1. Lean

After the theorem elaborates, run the audit and read the output, not just the exit code. A new axiom can appear because an imported theorem changed even when the local proof still checks.

A useful temporary scratch module is:

```
import Project.Audit

#check Project.headlineTheorem
#print Project.headlineTheorem
#print axioms Project.headlineTheorem
```

For a polymorphic theorem, inspect the fully elaborated type. If implicit parameters are suspicious, temporarily enable detailed pretty printing:

```
set_option pp.all true in
#check Project.headlineTheorem
```

13.5.2. Rocq

Use both source-level and kernel-level checks where documented:

```
Print Assumptions HeadlineTheorem.
Check HeadlineTheorem.
```

Then run `coqchk/rocqchk` on the public module set. This catches damaged or inconsistent compiled objects and independently validates the environment assumptions accepted by the kernel checker.

13.6. Generator workflow

A generator invocation should be reproducible from a clean checkout. Record:

- exact interpreter/tool version;
- command line and working directory;
- input files and their hashes;
- deterministic seed or proof that no randomness is used;
- expected output paths;
- formatting/canonicalization version;
- the first formal certificate target consuming the output.

Prefer generation into a temporary tree followed by a directory diff:

```
$Out = Join-Path $env:TEMP "proveit-generated"
Remove-Item $Out -Recurse -Force -ErrorAction SilentlyContinue
New-Item $Out -ItemType Directory | Out-Null

python path/to/generator.py --output-root $Out

git diff --no-index -- `
  path/to/checked-in/generated `
  $Out
```

If the generator writes directly into the source tree, first ensure `git status --short` is empty and inspect every changed path before building.

13.7. Reading build output correctly

A zero exit code can mean:

- the intended target built;

- the target was already up to date;
- the command selected no target;
- a wrapper skipped optional work;
- a noisy converter returned success after producing a partial artifact.

For Lean, verify that the requested module is registered/reachable and inspect the `.olean` timestamp only as a secondary clue. For Rocq, verify the requested logical module appears in the generated Makefile and that its `.vo` is refreshed. For generated certificates, verify the endpoint imports the changed shard.

13.8. A clean validation record

For a substantial theorem or generated checkpoint, preserve a text record:

```
$ErrorActionPreference = "Stop"
$Log = "validation-675d0a2f.txt"

"Commit: $(git rev-parse HEAD)" | Tee-Object $Log
"Lean:   $(lean --version)"      | Tee-Object $Log -Append
"Lake:   $(lake --version)"      | Tee-Object $Log -Append
"Rocq:   $(rocq --version)"      | Tee-Object $Log -Append
git submodule status             | Tee-Object $Log -Append

lake build Project Project.Audit 2>&1 |
  Tee-Object $Log -Append
```

The record is not a proof artifact, but it makes an engineering claim such as “the focused checkpoint was green on this toolchain” reproducible.

Primary files

[README.md](#); [lean-toolchain](#); [lakefile.toml](#); [lake-manifest.json](#); [_CoqProject](#); [Tools/Rocq92Compat/README.md](#); [Computability/TuringDegrees/Coq/BuildSyntheticComputability.ps1](#).

14. Debugging proof and build failures

14.1. Classify the failure before changing the proof

Most wasted time in a formal monorepo comes from treating every red build as a mathematical failure. ProveIt failures fall into several distinct classes.

Table 14.1.: Failure classes and first diagnostics.

Class	Symptom	First action
Target discovery	Command succeeds instantly or reports nothing; expected module never appears.	Elaborate the literal file with <code>lake env lean</code> ; inspect facade and <code>lakefile.toml</code> .
Import resolution	Unknown module, logical path, or package.	Check root/local workspace, submodule initialization, and <code>-Q/-R</code> mapping.
Elaboration/type inference	Type mismatch, failed synthesis, unsolved metavariable, unexpected implicit.	Minimize to the leaf declaration; enable <code>pp.all</code> ; disable <code>autoImplicit</code> .
Tactic proof search	Tactic timeout, no applicable rule, normalization blow-up.	Replace the endpoint tactic with intermediate lemmas; inspect the goal after each transformation.
Kernel reduction	<code>decide</code> exceeds heartbeats/recursion/memory.	Shrink or re-shard data; expose definitional structure; avoid global option increases first.
Generated dependency	Missing shard, cyclic import, stale name, wrong tail boundary.	Compare generator manifest/import graph; build immediate data and certificate modules.
Semantic bridge	Certificate equality checks but endpoint does not follow.	Inspect the checker-correctness or evaluation theorem, not the data.
Rocq environment	Package or plugin name mismatch, Windows launcher failure.	Reproduce the documented compatibility overlay and exact opam switch.
Cross-prover parity	One assistant proves a theorem with extra premises or different encoding.	Compare literal theorem types and coverage ledger; do not patch names to match.
Specification	The theorem is easy for a suspicious reason or audits show extra parameters.	Print the fully elaborated statement; trace object/meta/model quantifiers.

14.2. Target-discovery failures

The strongest symptom is a “successful” build that touched nothing. This can happen when a module exists under a registered source root but is not reachable from a target, or when a command syntax names a target Lake does not know.

Use three tests:

1. Does the file exist at the expected physical path?
2. Can `lake env lean path/to/file.lean` elaborate it?
3. Does a registered facade import it?

If only test 2 succeeds, the source is valid but integration is missing. Choose deliberately whether to add it to a facade, create a target, or leave it as direct-build research material.

For Rocq, inspect the `_CoqProject` source list and logical mapping. A `.v` file outside the list can compile manually yet be absent from the root Makefile.

14.3. Import-resolution failures

14.3.1. Lean logical name versus physical path

An import such as

```
import BoundedPAConsistency.Audit
```

does not imply a directory named `BoundedPAConsistency` at the root. The Lake registry maps the logical prefix to a deeper source directory. When moving modules, update the source-root mapping or preserve the logical prefix; do not make import paths mirror physical taxonomy accidentally.

If the same project has a local Lake file, verify which environment the editor and command line are using. A module available in the root workspace may be unknown in the local project, or vice versa.

14.3.2. Rocq logical roots

For Rocq, a file path and module name are connected by `-Q` or `-R`. Changing one side without the other can create duplicate logical names, unbound imports, or `coqchk` failures. Use the exact mapping from the project README in focused commands.

14.3.3. Submodule state

A missing Foundation, Abel, Undecidability, or Synthetic Computability import can simply mean the gitlink was not initialized. Check `git submodule status`; a leading minus indicates an uninitialized submodule, while a plus indicates a checked-out commit different from the superproject's pin.

14.4. Elaboration and specification failures

14.4.1. Unexpected implicit parameters

Lean's automatic implicit-variable creation is convenient in ordinary mathematics and dangerous in statement-heavy metatheory. If a theorem unexpectedly generalizes over a model, code, or translation:

1. print the theorem with `pp.all`;
2. place `set_option autoImplicit false` near the declarations;
3. declare every universe, type, model, and coding parameter explicitly;
4. add an audit `\#check` with the intended type.

A theorem that proves too easily after a refactor should be treated as a specification regression until its fully elaborated type is reviewed.

14.4.2. Coercion and notation drift

Large projects cross $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, polynomial constants, finite indices, and quoted object-language numerals. Tactic failures often come from a changed coercion rather than a false lemma.

Normalize one boundary at a time:

- expose `Nat.cast`, `Int.cast`, or polynomial `C`;
- use explicit type annotations on numerals;
- prove a local cast lemma instead of adding more simplifier lemmas globally;
- distinguish an object-language numeral constructor from the meta-language number.

14.4.3. Quotients and representative changes

FirstOrder term models and Turing degrees use quotient constructions. A definition over representatives must prove compatibility before descending. When a quotient theorem fails after an upstream API change, look for:

- a missing congruence theorem;
- a representative-level equality weakened to equivalence;
- an accidental use of definitional equality where only quotient equality is available;
- changed simplification around `Quot.sound`.

14.5. Tactic and normalization failures

14.5.1. Do not respond first by raising all limits

A timeout can signal that the proof lost structure. Before increasing heartbeats:

1. identify the last fast intermediate state;
2. split a global `simp/ring/norm_num` into semantic rewrites followed by algebraic normalization;
3. make finite indices explicit with `fin_cases`;
4. isolate one polynomial coordinate or one matrix block;
5. mark a large reusable equality as a theorem and rewrite by it;
6. check whether a reducible definition was made opaque or vice versa.

The Jacobian determinant proof is a good model: reduce the general determinant to a 3×3 closed formula, unfold derivatives, then call `ring`.

14.5.2. Simplifier loops and giant goals

Generated algebra code can make `simp` expand a large table repeatedly. Use restricted simplification sets, unfold only the top-level function, and prefer a named semantic lemma over exposing every definition.

In a certificate module, change is often intentional: it reveals a specific list-addition spine so previously proved term certificates can rewrite it. Replacing that with indiscriminate unfolding can create an unmanageable expression.

14.6. Kernel-certificate failures

14.6.1. A bisect strategy for term/tail shards

Suppose `table0_tail22_certificate` no longer closes.

1. Build/import `Tail22Data` alone. A failure here is generated syntax, type, or dependency.
2. Check the immediate `merge_certificate`. If `decide` fails, the candidate and normal literal differ.
3. Check each term certificate 88–91 and tail 23 independently.
4. Expand only the top-level tail definition and verify the expected association of `add`.
5. Compare generator output for the first mismatching term, not the entire normal polynomial.

A binary-search or Merkle-style manifest over generated pieces can shorten this process further; see the recommendations in Chapter 17.

14.6.2. Recursion depth versus heartbeats

`maxRecDepth` failures indicate deeply nested term reduction or elaborator recursion. `maxHeartbeats` failures indicate too much total work. Increasing the wrong option merely changes the error. Long list literals and right-associated additions tend to stress depth; large normalization across many monomials stresses work and memory.

14.6.3. Native replay disagrees with kernel replay

If an external or native checker says a certificate is valid but kernel `decide` does not:

- confirm both evaluate the same serialized data and normalization convention;
- check integer overflow or machine-word arithmetic in the fast tool;
- ensure sorting and duplicate-combination rules match;
- verify signs and exponent tuple order;
- reduce the disagreement to the smallest term and compare exact output.

The kernel result is authoritative for the kernel theorem. The fast replay is a diagnostic oracle, not a reason to replace the theorem with an assertion.

14.7. Semantic-bridge failures

A certificate can be perfectly valid for the wrong predicate. Typical bridge bugs include:

- a coverage checker validates every unit cell but the real-lifting theorem assumes boundary behavior not proved;
- a machine trace checker proves halting but the endpoint needs maximal score among all machines;
- a sparse polynomial list equality is proved but evaluation preservation under normalization is missing;
- a coded proof verifier checks conclusions but the partial-truth induction requires all formula occurrences bounded;
- a compiler simulates successful evaluation but the universality theorem needs divergence preservation.

Debug by writing the exact implication:

$$\text{check}(d) = \text{true} \implies \text{headline mathematical property.}$$

Then inspect every premise of the theorem implementing that arrow.

14.8. Cross-prover mismatches

When Lean and Rocq theorem names align but statements do not, avoid forcing a syntactic port. Compare:

1. universe/type assumptions;
2. constructive versus classical principles;
3. quotient versus setoid equality;
4. encoding and indexing conventions;
5. standard versus nonstandard quantification;
6. total versus partial computation relations;
7. imported theorem provenance.

Update the coverage ledger with the semantic relationship. A more general Rocq theorem or a more executable Lean theorem can both be correct without one being “behind.”

14.9. Rocq environment and compatibility failures

The compatibility overlay should be reproduced before editing proof scripts. Package-name adapters and proof-order patches can make the same source parse or resolve differently.

Useful diagnostics include:

```
where.exe coqc
where.exe rocq
opam var lib
ocamlfind list
rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq -n Target.vo
```

The dry run reveals which compiler command and logical paths the Makefile will use. If a patched library is involved, record its exact source commit and patch hash.

14.10. Minimal reproductions inside the monorepo

A good minimal reproduction remains in the same package environment but imports as little as possible:

```
import Project.SmallestDependency

namespace Scratch

-- Copy only the failing statement and the few definitions it needs.

end Scratch
```

Place it temporarily beside the project or under a scratch path excluded from the final commit. Do not copy giant generated definitions manually; import the data module and isolate one theorem.

For Rocq, create a temporary `Scratch.v` with the same `-Q/-R` flags and only the failing `Require Imports`.

14.11. When to suspect the statement

The following are red flags:

- a difficult theorem becomes trivial after moving a variable;

- a bound changes from external n to internal $\forall n$ without a new uniformity argument;
- a theorem over standard syntax is applied to arbitrary model elements;
- the audit prints an unexpected parameter or no longer names a crucial premise;
- a translation theorem loses its satisfaction-preservation hypothesis;
- a final theorem imports a `Placeholder`, `Assumption`, or `Conditional` module not mentioned in status docs.

Stop optimizing the proof and review the specification. In formal metamathematics, the wrong theorem can be easier than the right one.

Primary files

[Logic/Interpretability/ZFCinPA/README.md](#); [Logic/PeanoArithmetic/BoundedConsistency/README.md](#); [Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitKernelCertificateTable0Tail22Certificate.lean](#); [Tools/Rocq92Compat/README.md](#); [Computability/TuringDegrees/COVERAGE.md](#).

15. Extending a project without blurring its boundary

15.1. Choose the project shape first

Before adding files, decide which of the repository's project species the work belongs to:

1. a direct theorem over an existing semantic API;
2. a new semantic layer or calculus;
3. a finite certificate for an existing checker;
4. a new checker plus generated certificates;
5. an independent Rocq/Lean port;
6. a status/coverage extension without a new endpoint.

This choice determines where definitions live, what the facade should import, which audits are needed, and whether support tooling is inside or outside the proof boundary.

15.2. Adding a direct Lean theorem

Suppose an existing project already has the right definitions.

15.2.1. Place the theorem at the lowest reusable layer

If the theorem is a general property of the core datatype, place it near the core or in a focused lemma module. If it assembles project-specific results, place it in an endpoint module. Avoid adding a deep theorem directly to a facade.

A typical sequence is:

```
-- Project/NewProperty.lean
import Project.Core
import Project.ExistingBridge

namespace Project

theorem newProperty ... : ... := by
  ...

end Project
```

Then import `Project.NewProperty` from the facade only if the theorem is part of the consumer surface.

15.2.2. Generalize only where it lowers dependencies

Prove determinant identities over a commutative ring and add characteristic-zero only at nonvanishing. Prove finite combinatorial maps over finite types and add concrete coordinates later. But do not generalize a statement into a forest of typeclasses that hides its intended semantics.

A useful test is whether the generalized lemma removes an import or serves at least two endpoint paths. Otherwise a concrete theorem can be clearer.

15.2.3. Add an audit entry

The audit should contain the literal public theorem:

```
#check Project.newProperty
#print axioms Project.newProperty
```

If a theorem is intended to remain constructive, add an explanatory comment about the expected empty/classical assumption footprint. If it deliberately uses an imported classical theorem, name that provenance.

15.3. Adding a semantic layer

A new proof system, machine model, or object-language encoding needs more than one endpoint theorem.

15.3.1. Define the contract

Write down:

- syntax and well-formedness;
- semantics or observation relation;
- executable operations, if any;
- equivalence/congruence needed for quotients;
- public theorem relating syntax and semantics;
- exact imported assumptions.

Then split the files along those responsibilities. The FirstOrder and Presburger projects are good templates.

15.3.2. Prove substitution/renaming early

Many later metatheorems depend on substitution and assignment lemmas. If these are postponed, every semantic induction accumulates ad hoc rewrites. Establish identity, composition, and semantic substitution before soundness or completeness.

15.3.3. Keep data-carrying and Prop-valued relations distinct

An inductive proof tree carrying premises is useful for executable verification and all-occurrences bounds. A Prop-valued derivability relation is convenient for mathematical statements. Provide an erasure/soundness bridge rather than forcing one representation to do both jobs.

15.4. Adding a finite certificate

15.4.1. Start with the mathematical predicate

Define a checker against arbitrary candidate data:

```
def checkCertificate (d : CertificateData) : Bool := ...

theorem checkCertificate_sound {d : CertificateData} :
  checkCertificate d = true -> MathematicalProperty d := by
  ...
```

Only then introduce the concrete certificate. If the soundness theorem is proved after data generation, there is a temptation to validate exactly the properties the candidate happens to satisfy rather than the properties the endpoint needs.

15.4.2. Choose an evaluator deliberately

Use `kernel decide` for bounded computations that fit. Use `native_decide` only with a documented trust decision. On Rocq, document `vm_compute` or other reflection machinery. Keep a slower small test certificate that can be checked by the strongest evaluator for regression.

15.4.3. Separate placement from meaning

For geometry:

integer placement check $\rightarrow$ real-set coverage $\rightarrow$ dissection theorem.

For machines:

trace replay $\rightarrow$ operational halting $\rightarrow$ score bound/classification.

For algebra:

list equality $\rightarrow$ evaluation equality $\rightarrow$ invariant/resolvent theorem.

Each arrow deserves a named theorem. This makes it impossible for a future refactor to silently weaken the checker while leaving the endpoint name unchanged.

15.5. Adding or changing a generator

15.5.1. Treat output format as an API

Generated Lean/Rocq source has consumers and compatibility constraints. Define a stable schema:

- canonical ordering of terms;
- duplicate-combination and zero-removal rules;
- integer/sign formatting;
- shard-size and naming policy;
- import direction;
- deterministic header containing generator version and input hash.

Changing canonicalization may be mathematically harmless but can invalidate every downstream merge certificate and create an enormous diff.

15.5.2. Emit a manifest

For each generated module, a manifest should record:

- logical module name and file path;
- content hash;
- parent shards;
- semantic root or source term range;
- expected certificate module;
- resource options needed to elaborate it.

Even if the repository does not yet enforce such a manifest globally, adding one to a new generator makes shard completeness and binary-search debugging much easier.

15.5.3. Regenerate all or prove locality

A partial regeneration is safe only if the generator can prove that unchanged shards do not depend on changed input. Otherwise regenerate the full family into a temporary directory and compare. Check both content and import graph.

15.5.4. Keep generator tests outside theorem claims

Unit tests can verify parser round trips, canonical ordering, and deterministic output. They improve engineering reliability but do not substitute for certificate theorems. README language should say “generator test” or “independent replay,” not “formal proof.”

15.6. Adding a Rocq port

15.6.1. Choose the parity level

Before coding, classify the goal:

- exact statement-level port;
- stronger/more general target theorem;
- theorem-surface port with a different encoding;
- partial dependency layer;
- declaration-free facade;
- complementary result, not a port.

Record that classification in the coverage ledger. A theorem with the same English title but an extra choice principle or standardness premise is not an exact port.

15.6.2. Map dependencies, not just declarations

For each source module, record:

- source imports;
- target modules providing analogous concepts;
- representation translations;
- additional logical assumptions;
- public declarations covered;

- deliberate omissions.

A port can be blocked by a missing semantic layer even if the target theorem itself looks short. FoundationPort's conservative ledger is the model.

15.6.3. Audit and kernel-check the target

Add `Print Assumptions` for headline declarations and a focused `coqchk/rocqchk` command. If the project uses a vendored library, state its commit and logical root.

15.7. Registering a new project

A new Lean project may need some or all of:

1. mathematical directory under the correct top-level subject;
2. `Lean/ProjectName.lean` facade;
3. implementation namespace and modules;
4. `Audit.lean`;
5. project README with status, layout, trust, and build;
6. local `lakefile.toml` if independent/focused builds matter;
7. root `lean_lib` mapping;
8. optional root facade import;
9. optional root default target.

These last three choices are independent. A heavyweight project can be registered but omitted from `ProveIt.lean`; a critical audit can be a default target without exposing all internals.

A new Rocq project needs a logical-root mapping, ordered source registration or `local_CoqProject`, audit, and kernel-check command.

15.8. Writing status documentation

A strong project README should answer the following in explicit headings:

Headline result

Literal mathematical claim and exact endpoint name.

Status

Unconditional, conditional, partial, or exploratory.

Architecture

Module path from definitions to endpoint.

Trust boundary

Kernel, evaluator, generated candidates, source transcription, imported theorems.

Lean/Rocq relationship

Independent, contract-parallel, complementary, or incomplete port.

Build

Focused commands and resource requirements.

Deliberate nonclaims

Nearby stronger statements not established.

Residue

Ordered remaining obligations for active projects.

Do not write “formalized” when only a definition or reduction is present. Do not write “verified by script” when the script is outside the proof assistant. Do not write “Lean and Coq proofs” when one side is conditional and the condition is the central theorem.

15.9. Commit and review hygiene

For hand-written changes:

- keep refactors separate from theorem changes when possible;
- include the focused build/audit command in the commit message;
- state any changed theorem type explicitly;
- update the project README and coverage ledger in the same commit.

For generated changes:

- separate generator-source changes from generated output only if the intermediate commits are clearly marked and not merged independently;
- record generator command and deterministic manifest;
- avoid hand-editing generated modules;
- include the first and final certificate targets validated;
- summarize shard frontier changes rather than listing thousands of filenames.

15.10. Pre-merge checklist

Extension checklist

1. The theorem statement has been reviewed with all implicit parameters visible.
2. The module is reachable from the intended facade or has a documented direct-build path.
3. The smallest leaf, endpoint, facade, and audit all build.
4. Generated data are consumed by a proved checker.
5. The evaluator and trust profile are documented.
6. Lean/Rocq parity claims match literal theorem types.
7. Conditional premises and open search claims remain visible.
8. The README status and residue are current.
9. Submodule pins and vendored provenance are unchanged or deliberately updated.
10. A clean checkout can reproduce the focused validation.

Primary files

[ProveIt.lean](#); [lakefile.toml](#); [_CoqProject](#); [Computability/TuringDegrees/COVERAGE.md](#); [Logic/FoundationPort/README.md](#); [Algebra/PolynomialFormulas/Lean/PolynomialFormulas/ComputableDummitCoefficientsCore.lean](#); [Combinatorics/SquaredSquare/README.md](#).

16. Status and active frontiers at the pinned commit

16.1. Three different meanings of “unfinished”

A large formal repository can have an unfinished component for three independent reasons.

Mathematical frontier

The desired statement is open or a historical exhaustive argument has not been formalized.

Formalization frontier

The mathematics is known or conjectured, but a bridge, translation, or certificate layer is missing.

Build/checkpoint frontier

The architecture is established and shards are being generated or kernel-checked incrementally.

These should not be collapsed into one progress percentage. A completed conditional reduction can be mathematically valuable while its premise remains open. A generated module can be physically present while its parent endpoint has not yet imported it. An entire theorem can be complete in Lean but only partially ported to Rocq.

16.2. Snapshot catalogue

Table 16.1.: Headline status at commit 675d0a2f.

Project	Established surface	Remaining or deliberately excluded surface	Frontier type
JacobianConjecture	Explicit dimension-three polynomial map; determinant -2 ; integral/rational collisions; no inverse; scaling, stabilization, and degree-lowered variants; Lean and Rocq audits.	No claim about the surviving dimension-two problem; broader consequences are outside this project.	Scope boundary.
PolynomialFormulas: classical core	Degrees one through four; exact radical relations; executable Gaussian radical syntax/approximation; explicit and generic Abel–Ruffini surfaces; quintic and sextic decision frameworks.	A general executable radical formula for arbitrary solvable quintics is not claimed merely from the decision theorem.	Mathematical/API boundary.
PolynomialFormulas: Lazard/Dummit	Large portions of finite group classification, resolvent bridges, sparse coefficient tables, term/-tail certificates, and a deterministic module plan are checked at the recorded checkpoint.	Later table terms, root-state data, and remaining planned certificate parents identified by the formalization report/commit checkpoint.	Build/checkpoint frontier.
PowerTowers A198683	Analytic and finite certificates for accepted small values through the stated OEIS surface; substantial $n = 12$ symbolic, overflow, magnitude, and endpoint work; near-1 endpoint bundle unconditional.	The main accepted data field deliberately withholds the final $n = 12$ value until the complete proof-quality certificate boundary is satisfied.	Formalization frontier.
SquaredSquare	Order-21 perfect dissection, scaling, and elementary lower bound excluding orders at most six; exact bracket $7 \leq m \leq 21$.	Historical exhaustive exclusion of orders 7–20, represented by <code>DuijvestijnSearchClaim</code> .	Finite-search frontier.
KlarnerConstant	All seventeen geometric recurrences, exact supersolution 9047/2000, pointwise bound, and asymptotic growth endpoint are unconditional.	Human transcription of paper diagrams remains a provenance boundary, not an open theorem premise.	Source boundary only.
BusyBeaver	Machine semantics, known-value API, and opt-in small-state certificate families; explicit generators and audits.	Some BB4 root/classification endpoint remains conditional or checkpointed as documented; expensive certificates remain outside the facade.	Build/formalization frontier.
CombinatoryLogic	Local lambda/SK/SKI/Iota simulations, coding, confluence components, and universality; external computability connections.	No full-abstraction, mutual-inverse, or unproved cross-semantics composition claim.	Deliberate nonclaim.
TuringDegrees	Lean quotient theory with core degree operations and cardinality results; Rocq setoid theory with additional jump, hierarchy, and incomparability directions; explicit coverage ledger.	Known deeper existence/classification theorems and exact cross-prover parity remain deliberately incomplete; some Rocq endpoints retain logical premises.	Coverage frontier.

Project	Established surface	Remaining or deliberately excluded surface	Frontier type
FoundationPort / Modal	A large independently checked Rocq theorem surface and a 455-module source inventory with reviewed port status.	Modules or specialized systems marked partial/unported ; green subtrees do not imply full Foundation parity.	Port frontier.
FirstOrder	Independent fixed-language soundness, completeness, model existence, consistency/model equivalence, and compactness.	Generic-language work is delegated to a separate Mathlib-backed facade, not an unfinished local generalization.	Deliberate scope.
PA ListCoding	Thirty audited formulas/predicates, including list operations and ε_0 -related arithmetic, in independent Lean/Rocq encodings.	Numeric code equality across assistants is not claimed.	Contract boundary.
Bounded PA consistency	Numeralwise all-occurrences bounded-consistency scheme in both assistants; stronger uniform endpoint available in Lean at this snapshot.	Rocq strongest uniform endpoint retains explicit compiler premises; exact parity is not claimed.	Cross-prover frontier.
Bounded ZFC consistency	Numeralwise all-occurrences bounded-consistency theorem proved over the local FirstOrder/ZF stack with no surviving project premise.	A single internal universal sentence is deliberately not the theorem.	Specification boundary.
ZFCinPA	Definability layers, exact target, eight-field certificate reduction, standard-index successor implications, and multiple semantic bridges.	Nonstandard-index successor implication <code>ZFCSuccessorImplicationsInAllModels</code> ; final uniform PA endpoint remains conditional.	Mathematical/formalization frontier.
PresburgerArithmetic	Lean end-to-end Cooper elimination and executable decision; Rocq normalized one-variable correctness layer.	Full identical Rocq end-to-end executable parity.	Port frontier.

16.3. How to read a checkpoint commit

A checkpoint commit in a generated proof program should answer:

1. Which leaf certificates are kernel-green?
2. Which parent certificates compose them?
3. Which semantic bridge consumes the parent?
4. Which files are generated but not yet reachable?
5. What is the next dependency-ordered frontier?
6. Which report or manifest was regenerated?

The pinned commit's Lazard message does this with term/tail numbers and a 563-module root plan. Those numbers are not theorem names; they are coordinates in a deterministic build DAG. A future walkthrough of a later commit should replace them rather than treating them as permanent architecture.

16.4. Status can regress without a theorem becoming false

Several ordinary changes can make a previous endpoint unavailable:

- an upstream Mathlib or Rocq theorem changes shape;
- a generated shard is renamed without updating its parent;
- a facade stops importing the endpoint;
- a toolchain upgrade makes a formerly feasible kernel reduction exceed resources;
- a coverage ledger tightens its parity criterion;
- a specification audit reveals that an earlier statement was weaker than intended.

This is why ProveIt keeps theorem status near build and trust information. “Was proved in some earlier commit” and “is part of the current public surface” are different facts.

16.5. Recommended status vocabulary

For future modules, the following terms match existing good practice:

Table 16.2.: Precise status terms.

Term	Use
<i>Unconditional</i>	The literal endpoint has no project-specific hypothesis; imported logical axioms are reported separately.
<i>Conditional</i>	The endpoint theorem has an explicit premise representing unfinished or external mathematics.
<i>Numeralwise</i>	A meta-level family of proofs indexed by each standard natural number.
<i>Uniform</i>	One object-language or model-internal theorem quantifies over the index.
<i>Kernel certificate</i>	Concrete computation is reduced by the kernel or converted to a kernel proof term.
<i>VM/native checked</i>	A faster evaluator is part of the operational trust path.
<i>Generated candidate</i>	Data were produced externally but do not become evidence until a formal checker accepts them.
<i>Focused green</i>	A named subset of modules built under a stated command/toolchain.
<i>Ported</i>	A reviewed theorem surface exists in the target assistant, under documented encoding and assumptions.
<i>Partial</i>	A meaningful theorem slice exists but module-level parity is not claimed.

Term	Use
<i>Deliberate nonclaim</i>	A nearby stronger property is explicitly outside the theorem surface.

Status

All entries in this chapter describe commit 675d0a2f. They are not predictions about the current branch after that commit. The commit-pinned READMEs and audits are the authoritative sources for this walkthrough.

Primary files

README.md; Algebra/PolynomialFormulas/README.md; Combinatorics/ExpressionEnumeration/PowerTowers/README.md; Combinatorics/SquaredSquare/README.md; Computability/BusyBeaver/README.md; Computability/TuringDegrees/COVERAGE.md; Logic/FoundationPort/README.md; Logic/Interpretability/ZFCinPA/README.md; Logic/PeanoArithmetic/BoundedConsistency/README.md; SetTheory/BoundedConsistency/README.md.

17. Architectural assessment and technical recommendations

17.1. What the repository already does exceptionally well

17.1.1. It names proof boundaries

The strongest recurring practice is the explicit separation among:

- a theorem proved by ordinary elaboration;
- a theorem closed by kernel computation;
- a VM/native computation boundary;
- an externally generated candidate;
- a source-transcription boundary;
- a retained mathematical premise.

Many formal repositories make these distinctions only discoverable by reading proof terms. ProveIt often puts them in the README, module name, and audit.

17.1.2. It avoids false cross-prover symmetry

TuringDegrees, PA list coding, bounded consistency, and CombinatoryLogic all document where Lean and Rocq use different representations or prove different surfaces. This is more trustworthy than forcing a two-column table in which every row is marked “done” despite hidden premises.

17.1.3. It scales exact computation by sharding

The Lazard/Dummit tree shows that kernel-checked exact algebra can be scaled far beyond one monolithic reduction. Term, row, tail, and merge certificates localize failures and let the import graph act as a proof DAG.

17.1.4. It separates reusable conditional theorems from unconditional endpoints

Klarner’s Main and GeometricComplete exemplify good layering: the abstract recurrence theorem remains reusable, while the concrete project also discharges its hypotheses. SquaredSquare and ZFCinPA make a different choice where the premise remains open and keep it in the public statement.

17.1.5. It treats status ledgers as artifacts

FoundationPort’s 455-module inventory and TuringDegrees’ claim ledger are first-class maintenance tools. They prevent local success from being misreported as full port completion.

17.2. Structural risks

17.2.1. Generated-source discoverability

Tens of thousands of Lean files make filesystem browsing a poor index. Module names encode local structure, but there is no single machine-readable map from:

headline theorem → semantic bridge → certificate roots → generated leaves → generator command.

The formalization reports fill part of this gap for Lazard, but each large generator currently establishes its own conventions.

17.2.2. Silent build no-ops

The ZFCinPA example demonstrates that a plausible `lake build +Module` command can schedule no work. In a repository where “file exists” and “target exists” are different, this is a serious validation hazard.

17.2.3. Documentation can drift faster than theorem types

A README may still describe an earlier checkpoint after generated modules or endpoint estimates have advanced. The A198683 $n = 12$ subtree, for example, contains stronger intermediate results than a simple “not yet accepted” summary conveys. Human status prose needs some generated support.

17.2.4. Resource feasibility is implicit

Scoped heartbeats and recursion limits are visible in source, but a reviewer cannot easily answer:

- expected wall-clock time;
- peak memory;
- whether parallel workers are safe;
- which evaluator was used in the last green build;
- which modules form the expensive critical path.

This matters for reproducibility on machines smaller than the original development host.

17.2.5. Root validation is not visibly automated

At the pinned commit there is no repository-level GitHub Actions workflow tree. Project READMEs provide exact commands, but no checked-in CI matrix shows which tiers run on every change, nightly, or only manually. With multiple toolchains and very large certificates, one all-or-nothing CI job would be impractical; the absence of a tiered machine-readable matrix remains a maintenance risk.

17.2.6. Submodule and compatibility complexity

Rocq builds combine submodules, opam dependencies, local wrappers, and a Windows compatibility overlay. This is technically justified but increases the chance that a successful build depends on an unrecorded mutable switch or patched working tree.

17.3. Recommendation 1: a machine-readable project registry

Add a root registry such as `projects.toml` generated or reviewed alongside `lakefile.toml`. One record per project could contain:

Listing 17.1: Illustrative project-registry schema.

```
[project.SquaredSquare]
subject = "Combinatorics"
lean_facade = "SquaredSquare"
lean_audit = "SquaredSquare.Audit"
rocq_root = "SquaredSquare"
readme = "Combinatorics/SquaredSquare/README.md"
status = "unconditional-bracket; sharp-minimality-conditional"
trust = ["lean-kernel-decide", "rocq-vm-compute", "manual-transcription"]
tier = "medium"
generated = false
```

For a generated project, add generator, manifest, certificate roots, and resource tier. This registry need not replace Lake or `_CoqProject`; it would join their build facts with human status/provenance facts.

Benefits:

- generate a repository project index and status table;
- validate that every facade/audit path exists;
- detect orphan generated modules;
- drive tiered CI;
- keep README summaries synchronized.

17.4. Recommendation 2: target preflight

Provide a script that fails if a requested Lean module is not scheduled:

```
pwsh Tools/Build-CheckedLeanTarget.ps1 `
-Module ZFCinPA.NumeralsOmega `
-RequireWork
```

The wrapper can query Lake's build graph if available, or fall back to direct `lake env lean` elaboration. It should print the physical source path, selected workspace, and output artifact.

Similarly, a Rocq wrapper can verify that each requested logical module is present in the generated Makefile before invoking `make`.

17.5. Recommendation 3: tiered validation

A practical validation matrix could have four tiers.

Table 17.1.: Suggested validation tiers.

Tier	Contents	Trigger
Smoke	Root facade parse/elaboration, micro-project facades, registry validation, format/provenance checks.	Every change.
Core	All hand-written endpoints and audits, excluding heavy generated certificate families.	Every change or merge queue.
Certificate	Changed generated leaves, their transitive certificate parents, semantic bridges, and audits.	Path-aware on generated/support changes.
Full	Root Lean default targets, all Rocq registered sources, all audits and kernel-checks under pinned environments.	Nightly/manual release checkpoint.

A path-to-target manifest from Recommendation 1 would let CI select certificate ancestors deterministically rather than rebuilding the entire DAG.

17.6. Recommendation 4: generated artifact manifests

Each generator should emit a JSON/TOML manifest with:

- generator source hash and version;
- input hashes;
- module/file hashes;
- dependency edges;
- semantic range (term numbers, rows, group classes);
- expected checker declaration;
- canonicalization version;
- last focused-green toolchain and resource observations.

A checker script can compare the manifest with both source imports and the filesystem. The proof assistant still establishes mathematical truth; the manifest establishes completeness and reproducibility of the generated DAG.

A Merkle root over leaf data would make it cheap to locate the first changed shard and to state exactly which data a parent certificate was built against.

17.7. Recommendation 5: generated status from literal theorem types

For conditional projects, maintain a small Lean module whose only purpose is to emit a structured status summary from `\#check` declarations. A script can parse or query the environment to verify that:

- the endpoint still has the documented premise;
- the unconditional wrapper exists when claimed;
- an audit names the same declaration;
- no implicit parameter appeared.

This would have caught several classes of documentation drift without trying to infer mathematical meaning automatically.

17.8. Recommendation 6: cross-prover coverage schema

Generalize the TuringDegrees/FoundationPort ledger into a common TSV or TOML schema:

```
source_claim = "FirstOrder.classical_complete"
lean_decl = "LeanProofs.FirstOrder.classical_complete"
rocq_decl = "FirstOrder.ClassicalCompleteness.classical_complete"
relation = "independent-equivalent"
lean_assumptions = ["Classical.choice", "propext", "Quot.sound"]
rocq_assumptions = ["ClassicalChoice"]
notes = "Fixed fMem language; same derivability convention"
```

Allowed relations could include exact, stronger, weaker, contract-equivalent, complementary, conditional, and unported. A renderer can produce human tables while a validator checks declaration existence.

17.9. Recommendation 7: source-transcription checksums

For projects based on paper diagrams or external codes, preserve:

- image/page identifier;
- normalized textual transcription;
- checksum of the source artifact when licensing permits local storage;
- independent parser/replay output;
- reviewer initials/date for manual visual comparison.

This does not make visual transcription a kernel theorem, but it turns an informal memory into an auditable provenance step.

17.10. Recommendation 8: resource contracts

Add a small metadata block per heavy target:

```
target = "PolynomialFormulas....Table0Tail22Certificate"
evaluator = "kernel-decide"
max_heartbeats = 20000000
max_rec_depth = 1000000
recommended_workers = 0
last_peak_memory_gib = 6.4
last_elapsed_seconds = 183
machine = "..."
```

The timing is not a correctness condition and will vary, but large regressions become visible. Contributors can select feasible targets for their hardware and avoid accidentally launching several memory-heavy shards concurrently.

17.11. Recommendation 9: stable public-facade tests

For every root facade, keep a tiny consumer module outside the implementation namespace that imports only the facade and checks the headline declarations. This catches accidental reliance on implementation imports available only in the monorepo root environment.

For example:

```
import JacobianConjecture

#check LeanProofs.JacobianCounterexample.jacobianConjecture_false_over_complex
```

The existing audits often serve this role, but a strict consumer test can avoid importing internal audit helpers and prove that the public facade alone is sufficient.

17.12. Recommendation 10: preserve the current honesty

The most important recommendation is conservative: do not sacrifice the repository's explicit status language while automating it. A generated green badge cannot express the difference between:

- an unconditional theorem;
- a complete reduction to an open premise;
- a numeralwise scheme;
- a kernel-green shard not yet connected to an endpoint;

- a theorem imported from an upstream library;
- an independent proof in another assistant.

Machine-readable metadata should make those distinctions harder to lose, not replace them with one “verified” bit.

17.13. Overall assessment

ProveIt’s architecture is unusually well suited to experimental formal mathematics: it permits small direct proofs, large generated calculations, independent ports, and conditional research frontiers in one repository while usually keeping their evidence types legible. Its main scaling problem is no longer theorem representation; it is indexing and validating the enormous proof DAG, build graph, and status graph as one coherent system.

The next engineering gains are therefore likely to come from manifests, target preflight, tiered validation, and generated crosswalks rather than from flattening projects into a more uniform source layout.

Primary files

README.md; lakefile.toml; _CoqProject; Computability/TuringDegrees/COVERAGE.md; Logic/FoundationPort/README.md; Algebra/PolynomialFormulas/LazardQuinticFormalization.tex; Tools/Rocq92Compat/README.md.

18. Reading itineraries

18.1. A one-hour architectural tour

For a fast but representative tour:

1. [README.md](#): read the trust-model and status conventions.
2. [ProveIt.lean](#): see the consumer surface.
3. [lakefile.toml](#): compare imported and registered libraries.
4. **QuantifierCommutation**: read an entire microproject.
5. **FirstOrder**: inspect the syntax–calculus–completeness ladder.
6. **SquaredSquare**: trace data → checker → real geometry.
7. One Dummit tail certificate: see generated sharding.
8. [Computability/TuringDegrees/COVERAGE.md](#): see cross-prover status done carefully.

This route samples every major architectural species without entering the largest generated files.

18.2. To audit one headline theorem

Use this route regardless of subject:

1. Find the literal declaration in the audit.
2. Print/check its fully elaborated type and assumptions.
3. Identify its endpoint module and direct imports.
4. Classify every direct dependency as hand-written theorem, imported theorem, checked computation, generated data, or explicit premise.
5. Follow the one dependency that carries semantic meaning, not the largest file.
6. Confirm the facade exposes the theorem.
7. Run the focused build and audit; read evaluator/trust notes.
8. Compare README paraphrase with literal theorem type.

For the Jacobian result, this means determinant and collision lemmas. For Klarnet, it means the concrete recurrence assembly and exact supersolution. For ZFCinPA, it means the surviving successor-implication premise.

18.3. To understand generated algebra

Read:

1. `ComputableDummitCoefficientsCore.lean`;
2. one small sparse term’s substitution/evaluation theorem;
3. one `Term...Data/Term...Certificate` pair;
4. one `Tail...Data/Tail...Certificate` pair;
5. the parent row/table certificate;
6. the invariant-theory bridge consuming the table;
7. the generator and formalization report last.

This order starts with semantics and uses the generated source as an instance. The opposite order produces a great deal of scrolling and very little understanding.

18.4. To understand formal metamathematics

Read:

1. FirstOrder Fol and Calculus;
2. FirstOrder Relativization;
3. ZF formula/model definitions;
4. one arithmetized syntax project such as PA ListCoding;
5. bounded PA or bounded ZFC proof-tree verifier;
6. partial truth and rule soundness;
7. ZFCinPA only after writing the standard/nonstandard gaps explicitly.

Do not begin with the final long theorem statement. Its identifiers refer to translations and coded predicates whose quantifier levels determine the mathematics.

18.5. To learn certificate engineering

Use three progressively richer examples:

SquaredSquare

Small fixed data, Boolean grid checker, clean geometric lifting.

BusyBeaver

Executable operational semantics, traces, normalization symmetries, and global classification.

PolynomialFormulas

Generated exact algebra, multi-level sharding, and semantic invariant bridges.

The three expose different completeness obligations. A placement checker must cover the whole region; a trace checker must be lifted to all machines for a maximum theorem; an algebraic checker must preserve evaluation and feed the right resolvent criterion.

18.6. To compare Lean and Rocq

Start with a genuinely small paired project such as QuantifierCommutation. Then inspect one contract-parallel project such as PA ListCoding. Finally read TuringDegrees or FoundationPort, where the coverage ledger is essential.

For every declaration pair, record:

(statement, encoding, assumptions, provenance, operational semantics).

Only the first component is visible from theorem names.

18.7. To contribute an independent result

A low-risk path is:

1. choose a project with a small facade and complete audit;
2. add one theorem in a focused module over existing definitions;
3. add a matching audit entry and README note;

4. validate both local and root environments;
5. only then consider a Rocq port or generated certificate.

Good starting projects include the analysis identities, integer sums, quantifier commutation, or small power-tower values. Projects involving nonstandard models or Lazard certificates have much higher architectural entry cost even when an individual theorem appears elementary.

18.8. To review mathematical status rather than code style

Read in this order:

1. status table/nonclaims;
2. literal endpoint and audit;
3. trust boundary;
4. coverage ledger or residue;
5. build command and pinned dependency versions;
6. source transcription/provenance;
7. implementation details.

A beautifully factored proof of the wrong endpoint is less useful than a messy but accurately scoped conditional theorem. ProveIt's best project documentation recognizes that priority.

Primary files

[README.md](#); [ProveIt.lean](#); [lakefile.toml](#); [Logic/QuantifierCommutation/README.md](#); [Logic/FirstOrder/README.md](#); [Combinatorics/SquaredSquare/README.md](#); [Computability/TuringDegrees/COVERAGE.md](#); [Algebra/PolynomialFormulas/LazardQuinticFormalization.tex](#).

Part IV.

Reference appendices

A. Root Lean workspace registry

The following table is an exact transcription of the library names and source roots in [lakefile.toml](#) at the pinned commit. “Root import” means a direct import in [ProveIt.lean](#); it does not count transitive imports. “Default” means the library itself or named audit modules appear in the root default-target list.

Table A.1.: Root lean_lib registry.

Library	srcDir	Root import	Default
JacobianConjecture	Algebra/JacobianConjecture/Lean	yes	-
PolynomialFormulas	Algebra/PolynomialFormulas/Lean	yes	-
ProveIt	repository root	-	yes
TrigonometricIdentities	Analysis/TrigonometricIdentities/Lean	yes	-
ExponentialIdentities	Analysis/ExponentialIdentities/Lean	yes	-
PowerTowers	Combinatorics/ExpressionEnumeration/PowerTowers/Lean	yes	-
A158415	Combinatorics/ExpressionEnumeration/RadicalExpressions/A158415/Lean	yes	-
A290268	Combinatorics/DerivativeExpansions/A290268/Lean	yes	-
SquaredSquare	Combinatorics/SquaredSquare/Lean	yes	-
KlarnerConstant	Combinatorics/Polyominoes/KlarnerConstant/Lean	yes	-
BusyBeaver	Computability/BusyBeaver/Lean	yes	-
CombinatoryLogic	Computability/CombinatoryLogic/Lean	yes	-
TuringDegrees	Computability/TuringDegrees/Lean	yes	-
FirstOrder	Logic/FirstOrder/Lean	yes	-
ArbitraryLanguageCompactness	Logic/FirstOrder/Compactness/Lean	yes	-
QuantifierCommutation	Logic/QuantifierCommutation/Lean	yes	-
ShefferStroke	Logic/Propositional/ShefferStroke/Lean	yes	-
NaturalDeduction	Logic/Propositional/NaturalDeduction/Lean	yes	-
FiniteMatrixNoncharacterizability	Logic/Propositional/FiniteMatrixNoncharacterizability/Lean	yes	-
MonotonicityOfEntailment	Logic/Propositional/MonotonicityOfEntailment/Lean	yes	-
PrincipleOfExplosion	Logic/Propositional/PrincipleOfExplosion/Lean	yes	-
EquationalLogic	Logic/Equational/Common/Lean	yes	-
BooleanAlgebra	Logic/Equational/BooleanAlgebra/Lean	yes	-
PAHF	Logic/Interpretability/PAHF/Lean	yes	-
NoFiniteModel	Logic/PeanoArithmetic/NoFiniteModel/Lean	yes	-
BoundedPAConsistency	Logic/PeanoArithmetic/BoundedConsistency/Lean	-	yes
PAListCoding	Logic/PeanoArithmetic/ListCoding/Lean	-	yes
PAFiniteBasisReduction	Logic/PeanoArithmetic/NotFinitelyAxiomatizable/Lean	yes	-
Foundation	lib/FormalizedFormalLogic-Foundation	-	-
PAUndecidable	Logic/PeanoArithmetic/Undecidable/Lean	-	yes
PresburgerArithmetic	Logic/PresburgerArithmetic/Lean	yes	-
DiophantineEquations	NumberTheory/DiophantineEquations/Lean	yes	-
IntegerSums	NumberTheory/IntegerSums/Lean	yes	-
RationalEnumeration	NumberTheory/RationalEnumeration/Lean	yes	-

Library	srcDir	Root import	Default
RiemannHypothesis	NumberTheory/RiemannHypothesis/PASentence/Lean	yes	–
ZF	SetTheory/ZF/Lean	yes	–
BoundedZFConsistency	SetTheory/BoundedConsistency/Lean	–	–
ZFCinPA	Logic/Interpretability/ZFCinPA/Lean	–	–
ClosureAxiomatization	SetTheory/ClosureAxiomatization/Lean	yes	–

The default-target list also names `PAListCoding.Audit`, `PAUndecidable.Audit`, and `BoundedPAConsistency.Audit`. This is why the final column is marked for the containing libraries even though audit modules are not separate `lean_lib` records.

B. Project and artifact index

Table B.1.: Where to start for each major project.

Project	Start path	First artifacts to inspect
JacobianConjecture	Algebra/JacobianConjecture	README.md; Lean Counterexample.lean; Audit.lean; Rocq Counterexample.v.
PolynomialFormulas	Algebra/PolynomialFormulas	README.md; Lean facade; Basic/Cubic/Quartic; ComputableDummitCoefficientsCore; formalization report.
ExponentialIdentities	Analysis/ExponentialIdentities	Facade; TinyExponentTower.lean; Rocq counterpart.
TrigonometricIdentities	Analysis/TrigonometricIdentities	Facade; arctangent and golden-ratio modules; audit.
PowerTowers	Combinatorics/ExpressionEnumeration/PowerTowers	Lean facade; Core; A198683; one small-value proof; $n = 12$ symbolic/endpoints.
A158415	Combinatorics/ExpressionEnumeration/RadicalExpressions/A158415	Facade; expression syntax; accepted-value certificates; research/support material.
A290268	Combinatorics/DerivativeExpansions/A290268	Facade; coefficient recurrence; auxiliary OEIS material.
SquaredSquare	Combinatorics/SquaredSquare	README; Basic; Duijvestijn; Minimality; MinimalOrder; audit.
KlarnConstant	Combinatorics/Polyominoes/KlarnConstant	README; Polyomino; Patterns; SeededPartition; one geometry family; Certificate; GeometricComplete.
BusyBeaver	Computability/BusyBeaver	README; facade; Core; KnownValues; smallest opt-in classification; generator only afterward.
CombinatoryLogic	Computability/CombinatoryLogic	README; Lambda; LambdaToSK; SKI; IotaToLambda; Universality; audit.
TuringDegrees	Computability/TuringDegrees	README; COVERAGE.md; Lean quotient core; Rocq jump/hierarchy modules; build wrapper.
QuantifierCommutation	Logic/QuantifierCommutation	Read the entire project: README, two proof modules, audit, and focused build.
FirstOrder	Logic/FirstOrder	README; Fol; Calculus; Completeness; ClassicalCompleteness; Compactness; audit.
ArbitraryLanguageCompactness	Logic/FirstOrder/Compactness	Facade and audit; identify the exact Mathlib theorem specialized.
FoundationPort	Logic/FoundationPort	README; ported.tsv; coverage.sh; Rocq Audit.v.
Modal	Logic/Modal	README/module coverage table; generic Kripke layer; one named-system chain; audit.
PAHF	Logic/Interpretability/PAHF	Lean/Rocq facades and audits; translation/-model modules.
ZFCinPA	Logic/Interpretability/ZFCinPA	README status table; exact target; eight-field certificate; standard-index bridge; conditional endpoint; companion audits.
PA ListCoding	Logic/PeanoArithmetic/ListCoding	README; code representation; formula registry; functionality theorems; audit.
Bounded PA consistency	Logic/PeanoArithmetic/BoundedConsistency	README; proof-tree/verifier; all-occurrences bound; partial truth; soundness; uniform endpoint/audit.

Project	Start path	First artifacts to inspect
NoFiniteModel	Logic/PeanoArithmetic/NoFiniteModel	Facade; semantic PA axioms; successor injection/non-surjection; audit.
PA finite-basis reduction	Logic/PeanoArithmetic/NotFinallyAxiomatizable	Statement and reduction modules before internal coding.
PA undecidability	Logic/PeanoArithmetic/Undecidable	Exact decision problem, reduction, computability bridge, audit.
PresburgerArithmetic	Logic/PresburgerArithmetic	README; Syntax; NormalForm; Cooper; Elimination; Decision; audit.
DiophantineEquations	NumberTheory/DiophantineEquations	README; tiny Lean wrapper; independent Rocq descent.
IntegerSums	NumberTheory/IntegerSums	Facade and paired finite-sum proofs.
RationalEnumeration	NumberTheory/RationalEnumeration	README; forward Calkin–Wilf map; Euclidean inverse; bijection; audit.
RiemannHypothesis	NumberTheory/RiemannHypothesis/PASentence	Exact arithmetic sentence and standard-model correspondence.
ZF	SetTheory/ZF	README; Lean facade; audit; search <code>Zf.lean</code> by declaration.
BoundedZFConsistency	SetTheory/BoundedConsistency	README; formula rank; restricted proof predicate; partial satisfaction; soundness; endpoint/audit.
ClosureAxiomatization	SetTheory/ClosureAxiomatization	README; forward/reverse semantic directions; deductive equivalence; article.
Rocq92Compat	Tools/Rocq92Compat	README; version pins; patch application; launchers/shims; no theorem claims.

C. Command cookbook

These commands are templates. Project READMEs remain authoritative for project-specific versions, worker settings, and dependencies.

C.1. Repository state

```
git rev-parse HEAD
git status --short
git submodule status
git diff --stat
git diff --check
```

C.2. Lean bootstrap and broad builds

```
lean --version
lake --version
lake exe cache get

# Root default targets.
lake build

# Consumer facade only.
lake build ProveIt
```

C.3. Focused Lean

```
# Directly elaborate a physical source file.
lake env lean path/to/Module.lean

# Build a registered library and audit.
lake build Project Project.Audit

# Use a project-local workspace.
lake --dir path/to/Project/Lean build
```

C.4. Lean inspection

```
import Project.Facade

#check Project.headlineTheorem
#print Project.headlineTheorem
#print axioms Project.headlineTheorem

set_option pp.all true in
#check Project.headlineTheorem
```

C.5. Lean resource controls

```
set_option maxHeartbeats 20000000 in
set_option maxRecDepth 1000000 in
theorem heavyCertificate : ... := by
  decide
```

```
# Serialize Lake jobs where project documentation recommends it.
$env:LEAN_NUM_THREADS = "0"
lake build HeavyTarget
Remove-Item Env:LEAN_NUM_THREADS -ErrorAction SilentlyContinue
```

C.6. Rocq focused build

```
coqc -Q path/to/Coq LogicalRoot path/to/Coq/Core.v
coqc -Q path/to/Coq LogicalRoot path/to/Coq/Endpoint.v
coqc -Q path/to/Coq LogicalRoot path/to/Coq/Audit.v

coqchk -silent -Q path/to/Coq LogicalRoot `
  LogicalRoot.Core LogicalRoot.Endpoint LogicalRoot.Audit
```

C.7. Rocq root build

```
git submodule update --init `
  lib/Coq-Synthetic-Computability `
  lib/MathComp-Abel

opam install --yes --deps-only `
  ./lib/MathComp-Abel/coq-mathcomp-abel.opam

pwsh -NoProfile -File `
  Computability/TuringDegrees/Coq/BuildSyntheticComputability.ps1

rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq
```

C.8. Rocq inspection

```
Check HeadlineTheorem.
Print HeadlineTheorem.
Print Assumptions HeadlineTheorem.
```

C.9. Foundation port coverage

```
bash Logic/FoundationPort/coverage.sh `
  > foundation-coverage.tsv
```

The script reports totals to standard error and emits one row per pinned Foundation source module.

C.10. Generated output comparison

```
$Out = Join-Path $env:TEMP "proveit-generated"
Remove-Item $Out -Recurse -Force -ErrorAction SilentlyContinue
New-Item $Out -ItemType Directory | Out-Null

python path/to/generator.py --output-root $Out

git diff --no-index -- `
    path/to/checked-in/generated `
    $Out
```

C.11. Find reachability

For a suspicious generated module:

```
# Find direct imports.
git grep -n "import .*ModuleName"

# Find facade and audit references.
git grep -n "ModuleName" -- "*Facade*.lean" "*Audit*.lean" "*.lean"

# Elaborate directly even if no target exists.
lake env lean path/to/ModuleName.lean
```

C.12. Clean rebuilds

Use clean operations only after preserving diagnostics; a clean can erase the evidence that distinguishes stale output from a source error.

```
lake clean
lake exe cache get
lake build Project Project.Audit

Remove-Item Makefile.coq -ErrorAction SilentlyContinue
rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq Target.vo
```

D. Trust and status checklist

D.1. For a direct theorem

1. Locate the literal theorem type.
2. Print all implicit parameters.
3. Read `\#print axioms` or `Print Assumptions`.
4. Identify imported headline theorems versus locally proved lemmas.
5. Confirm the facade imports the endpoint.
6. Confirm the audit names the endpoint.
7. Check the README paraphrase against the theorem type.

D.2. For a computed theorem

1. Identify the data type and concrete candidate.
2. Identify the executable checker.
3. Identify the checker-soundness theorem.
4. Identify the evaluator: kernel, VM, or native.
5. Check integer/arithmetic domains for overflow or coercion.
6. Confirm the endpoint uses checker soundness, not merely `check = true`.
7. Confirm generated data are reachable from the endpoint.

D.3. For a generated certificate

1. Record generator source and command.
2. Confirm deterministic/canonical output.
3. Compare leaf data and certificate modules.
4. Follow every parent shard to a semantic root.
5. Verify no generated theorem is admitted or asserted.
6. Verify regeneration does not leave orphan or stale modules.
7. Build leaf, parent, semantic bridge, facade, and audit.

D.4. For a conditional theorem

1. Name the surviving premise exactly.
2. Determine whether it is abstraction, source assumption, or open mathematical residue.
3. Check whether a concrete module discharges it.
4. Distinguish standard-index and all-model versions.
5. Ensure README status says *conditional*.
6. Ensure no unconditional alias hides the premise.

D.5. For a Lean/Rocq pair

1. Compare literal statements, not names.
2. Compare encodings and indexing.

3. Compare classical/constructive assumptions.
4. Compare evaluator/trust boundaries.
5. Determine whether one proof is imported or independently local.
6. Record exact parity relation in the coverage ledger.
7. Kernel-check both public module sets.

D.6. For a source transcription

1. Identify source page, figure, table, or code.
2. Preserve normalized transcription and checksum where possible.
3. Independently replay basic consistency conditions.
4. Distinguish transcription correctness from formal theorem correctness.
5. Document any manual visual comparison.

E. Glossary

Audit module

A Lean or Rocq module that imports public declarations and prints/checks their types and assumptions. It is executable documentation of the theorem surface.

Certificate

Concrete finite data intended to witness a decidable property. A certificate is evidence only through a proved checker/meaning theorem.

Checker

An executable formal function or predicate validating a certificate. Its soundness theorem connects acceptance to mathematical meaning.

Conditional endpoint

A theorem whose type retains a premise representing an unproved bridge, open claim, or intentionally abstract hypothesis.

Contract-level parity

Lean and Rocq use different representations but prove comparable public behavior such as decoding, functionality, or semantic equivalence.

Coverage ledger

A reviewed map from source claims/modules to target proof-assistant declarations and statuses.

Default target

A Lake target built by `lake build` with no explicit target. It need not be directly imported by the root facade.

Direct module elaboration

Running `lake env lean path/to/File.lean`; useful when no Lake target exposes the file.

Facade

A small module whose primary role is importing the intended public surface of a project.

Focused green

A named subset of modules that successfully built under a recorded toolchain/command. It is narrower than a full repository validation.

Generated data module

A source file containing literal exact data emitted by a support program. It should be consumed by a formal certificate theorem.

Kernel computation

Reduction performed within the proof assistant's trusted kernel, such as Lean `decide` on a closed decidable proposition.

Kernel checker

Lean's or Rocq's small trusted component that validates proof terms against the type theory.

Meaning theorem

A theorem that lifts a low-level checker result to the user-facing mathematical property.

Native/VM computation

Faster evaluation through a compiled or virtual-machine path. It can remain integrated with the proof assistant while having a broader operational trust base than direct kernel reduction.

Numeralwise scheme

For every external standard natural n , a separate formal derivation of the n -instance. This is not one internal universally quantified sentence.

Port

A theorem surface independently recreated in another proof assistant. Exactness, generality, assumptions, and encoding must be specified.

Proof boundary

The point at which data or computation becomes a theorem: kernel term, verified checker, imported theorem, or explicit premise.

Residue

The ordered set of remaining obligations in an active formalization.

Root facade

[ProveIt.lean](#), the broad user-facing Lean import. It is neither the complete library registry nor the root default-target list.

Setoid presentation

A type plus an explicit equivalence relation used instead of quotienting immediately. The Rocq Turing-degree project uses this style.

Shard

A bounded generated piece of a large certificate, often organized by term, row, tail, or class.

Source-transcription boundary

The human claim that formal data accurately reproduce an external diagram, table, or formula. Formal checking can validate consequences of the data without proving the visual transcription itself.

Standard versus nonstandard index

A standard index is an external natural/numeral. A nonstandard index is an arbitrary element satisfying arithmetic inside a nonstandard model. The latter requires model-internal uniformity.

Uniform theorem

One formal statement quantifying internally over all indices, rather than a meta-level family of instances.

Vendored source

Third-party code pinned or copied under `lib`, with provenance and licensing recorded.

Worker count

Lake process concurrency. It is independent of a declaration's heartbeat limit and can dominate peak memory for large certificate builds.

F. Snapshot methodology and source notes

F.1. Pinned repository state

All repository descriptions and hyperlinks refer to:

Repository	https://github.com/VladimirReshetnikov/ProveIt
Commit	675d0a2ff0fc624fba199e006049016ba806f40a
Short identifier	675d0a2f
Snapshot date	13 August 2026
Root tree	43b4ea9da61e61551f5b416a1b7c82ac2103897e

The main branch changed during inspection, including a reorganization into the current mathematical top-level taxonomy. Pinning prevents a path from silently referring to a later layout or theorem status.

F.2. Inspection method

The repository could not be cloned through the ordinary network path in the document-production environment. The exact commit was instead inspected through GitHub’s repository connector and Git-object/content APIs:

- root commit, branch, and recursive tree objects;
- root README, manifests, toolchain, facade, submodule map, and Rocq project;
- domain and project READMEs;
- representative implementation and audit modules;
- generated data/certificate pairs;
- support/generator inventories;
- recent commit metadata and checkpoint message.

Content links in the PDF use the full commit SHA, so they are equivalent to reading a local checkout at that commit for the cited paths.

F.3. What was and was not independently checked

The preparation process did:

- inspect the exact source text and module trees described above;
- trace representative public endpoints to definitions, checker layers, generated data, and audits;
- compare the root facade, Lake registry, and Rocq registry;
- symbolically spot-check the displayed Jacobian determinant and collision outside the proof assistant as a sanity check;
- compile this $\text{\LaTeX}$ document with $\text{\Lua\LaTeX}$;
- render and visually inspect the resulting PDF.

It did not:

- perform a full local checkout and complete `root lake build`;
- compile the entire Rocq federation and all submodules;
- regenerate every certificate family;

- independently peer-review the mathematical novelty or historical attribution of every project;
- certify that current main after 675d0a2ff0fc624fba199e006049016ba806f40a has the same status.

Accordingly, phrases such as “the project proves” mean that the pinned source contains the stated formal declaration and documented audit path. This walkthrough is not a substitute for running the project’s build and kernel checks on a clean checkout.

F.4. Current-event context

The Jacobian counterexample was an unusually recent event at the snapshot. The source code’s decisive identities are explicit and small, and the local formalization states them directly. For public context, the later self-contained account *Counterexamples to the Jacobian conjecture in dimensions greater than two* describes the July 2026 dimension-three refutation and subsequent generalizations. This external context is not a dependency of the Lean/Rocq proof.

F.5. Path and typography conventions

Clickable source paths are rendered in monospaced blue text. They point to `blob` URLs for files and `tree` URLs for directories at the pinned commit. Module names without a path are printed in monospaced type. Project names are bold sans serif.

Code excerpts are normalized for readability. In particular, some Lean Unicode symbols are rendered as ASCII pseudocode in listings. The surrounding text and pinned source link are authoritative where a normalized excerpt omits namespace qualification or notation details.

F.6. How to update this walkthrough

For a later commit:

1. replace the commit and root-tree identifiers;
2. diff `ProveIt.lean`, `lakefile.toml`, `_CoqProject`, and `.gitmodules`;
3. regenerate language/file scale figures;
4. revisit every status table and deliberate nonclaim;
5. update generated-certificate frontiers from the new report/commit;
6. re-run project coverage ledgers;
7. check whether new facades, audits, or CI workflows were added;
8. rebuild and visually inspect both source and PDF.

A purely path-based update is unsafe. In this repository, a theorem can keep its filename while its premise, evaluator, facade reachability, or cross-prover status changes.

F.7. Closing perspective

ProveIt is best read not as a pile of formal proofs but as a network of evidence-producing components. Some components derive a theorem directly. Some interpret syntax, compile one calculus into another, or build a model. Some check finite data emitted by search. Some record exactly where a port or uniformity argument stops. The repository’s coherence comes from making those transitions explicit.

The practical invariant for contributors and reviewers is given the final page to itself.

A closing invariant

Every headline claim should have a visible path from statement, through semantics and evidence, to the kernel—and every path that stops short should stop in a named premise or status entry.